

Tutorials Cross-validation

✓ Applications

We've got a splitter instance, what's next?

Remember that CV involves running a backtesting job on each range. Thanks to the ability of vectorbt to process two-dimensional data, we have two major ways of accomplishing that: either 1) split all arrays into chunks, merge them into two-dimensional arrays, and run one monolithic backtesting job, or 2) run a backtesting job on each chunk separately. We can also go a hybrid path: for example, build a two-dimensional array out of each set, and backtest it in isolation from other sets. Which approach to use depends on your requirements for RAM consumption and performance: two-dimensional arrays are faster to process, but there is a threshold of the number of stacked columns after which performance starts to downgrade (mostly because your system starts using swap memory).

Taking

Taking is a process of extracting one or more "slices" from an array-like object, which is implemented by the method `Splitter.take`. This method takes an object, iterates over each range in question, and takes the range from the object using `Splitter.take_range` - a dead-simple method that does `arr.iloc[range_]` on Pandas-like arrays and `arr[range_]` on NumPy arrays. Since many vectorbt classes inherit the indexing schema of Pandas, this method can even take a slice from vectorbt objects such as `Portfolio`! Once all slices have been collected, it can either stack them by rows (`stack_axis=0`) or columns (`stack_axis=1`, which is default), stack only splits and sets, or don't stack anything at all. When some slices shouldn't be stacked, they will be returned as a single Pandas Series that holds slices as values (may sound weird to have an array as a value of another array, but it's much easier for indexing than having a list).

Without stacking

Let's split the close price using default arguments:

```
>>> close_slices = splitter.take(data.close)
>>> close_slices
split_year  set
2018      train  Open time
2018-01-01 00:00:00+00:00  13380.0...
      test  Open time
2018-07-01 00:00:00+00:00  6356.81...
2019      train  Open time
2019-01-01 00:00:00+00:00  3797.1...
      test  Open time
2019-07-01 00:00:00+00:00  10624.9...
2020      train  Open time
2020-01-01 00:00:00+00:00  7200.85...
      test  Open time
2020-07-01 00:00:00+00:00  9232.0...
2021      train  Open time
2021-01-01 00:00:00+00:00  29331.6...
      test  Open time
2021-07-01 00:00:00+00:00  33504.6...
dtype: object
```

If you're wondering what this format is: a regular Pandas Series with the split and set labels as index and the close price slices as values - basically `pd.Series` within `pd.Series` 😊 Remember that array values can be any complex Python objects. For example, let's get the close price corresponding to the test set in 2020:

```
>>> close_slices[2020, "test"]
Open time
2020-07-01 00:00:00+00:00  9232.00
2020-07-02 00:00:00+00:00  9086.54
2020-07-03 00:00:00+00:00  9058.26
...
2020-12-29 00:00:00+00:00  27385.00
2020-12-30 00:00:00+00:00  28875.54
2020-12-31 00:00:00+00:00  28923.63
Freq: D, Name: Close, Length: 184, dtype: float64
```

Bingo!

And here's how simple is to apply a UDF on each range:

```
>>> def get_total_return(sr):
...     return sr.vbt.to_returns().vbt.returns.total()

>>> close_slices.apply(get_total_return) ❶
split_year  set
2018      train  -0.522416
      test  -0.417491
2019      train  1.858493
      test  -0.322797
2020      train  0.269093
      test  2.132976
2021      train  0.194783
      test  0.379417
dtype: float64
```

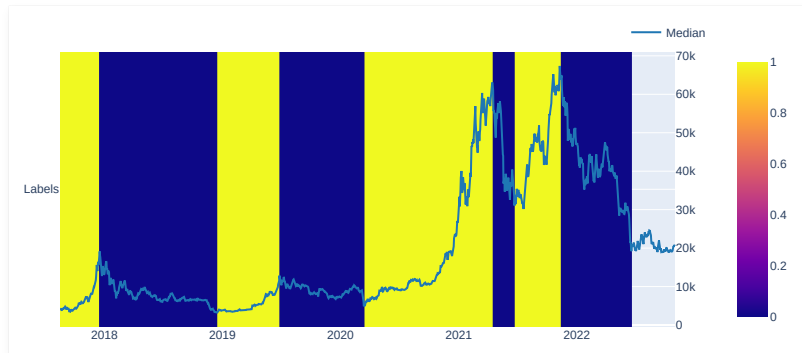
❶ Using `pandas.Series.apply`

Complex objects

One of the many unique features of vectorbt is the standardized behavior of its classes. Since this package is highly specialized on processing Pandas and NumPy arrays, most classes act as a proxy between the user and a set of such arrays; basically, each class enhances the feature set of Pandas and NumPy, and allows building connections between multiple arrays based on one common metadata stored in a **wrapper**. Since such a wrapper can be sliced just like a regular Pandas array, we can slice most vectorbt objects that hold this wrapper as well, including **Splitter**. And because we can slice them using the same indexing API as offered by Pandas arrays (thanks to **indexing**), we can pass most vectorbt objects directly to **Splitter.take**!

For example, let's analyze the performance of a portfolio during different market regimes. First, we'll use the forward-looking label generator **TRENDLB** to annotate each data point with either 1 (uptrend), 0 (downtrend), or NaN (cannot be classified). Given the volatility of our data, we'll register an uptrend once the price jumps by 100% from its previous low point, and a downtrend once the price falls by 50% from its previous high point:

```
>>> trendlb = data.run("trendlb", 1.0, 0.5)
>>> trendlb.plot().show()
```

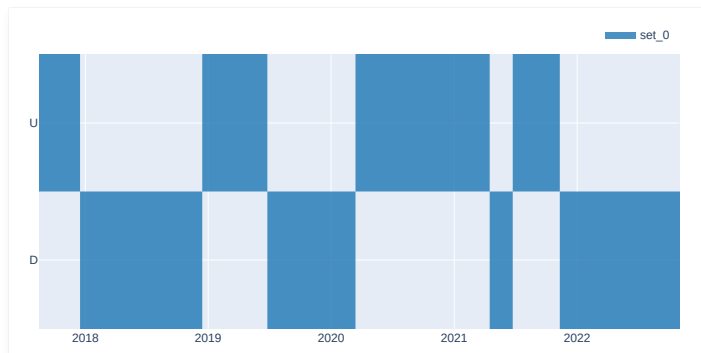


Hint

If you're not sure which pair of thresholds to use, look at the plot that the labeler produces and find the thresholds that work best for you.

Then, we'll build a splitter that converts the labels into splits:

```
>>> grouper = pd.Index(trendlb.labels.map({1: "U", 0: "D"}), name="trend")
>>> trend_splitter = vbt.Splitter.from_grouper(data.index, grouper)
>>> trend_splitter.plot().show()
```



In the next step, we'll run the full grid of parameter combinations on the entire period through column stacking, and extract the returns accessor of the type **ReturnsAccessor**, which will enable us in analyzing the returns. To make a comparison, we'll also do the same on our baseline model.

```
>>> hold_pf = vbt.Portfolio.from_holding(data)
>>> hold_returns_acc = hold_pf.returns_acc

>>> fast_sma, slow_sma = vbt.talib("SMA").run_combs(
...     data.close, np.arange(5, 50), short_names=["fast_sma", "slow_sma"])
>>> entries = fast_sma.real_crossed_above(slow_sma)
>>> exits = fast_sma.real_crossed_below(slow_sma)
>>> strat_pf = vbt.Portfolio.from_signals(
...     data, entries, exits, direction="both")
>>> strat_returns_acc = strat_pf.returns_acc
```

Now, take both slices from the accessor (remember that most vectorbt objects are indexable, including accessors) and plot the Sharpe heatmap for each market regime:

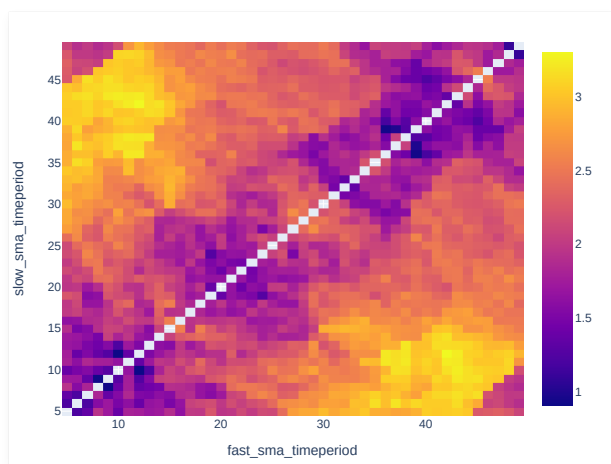
```
>>> hold_returns_acc_slices = trend_splitter.take(hold_returns_acc)
>>> strat_returns_acc_slices = trend_splitter.take(strat_returns_acc)
```

Uptrends

```
>>> hold_returns_acc_slices["U"].sharpe_ratio()
```

3.4490778178230763

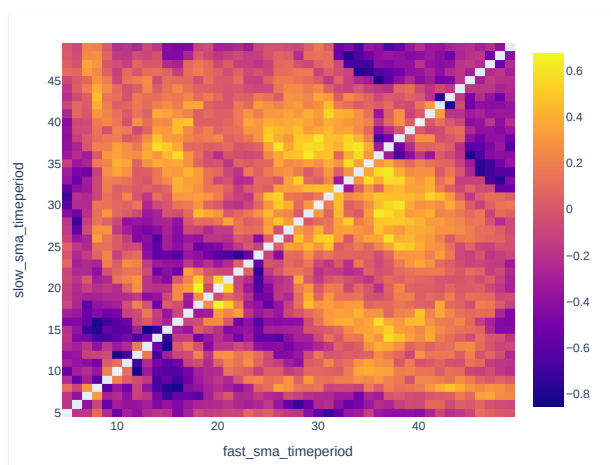
```
>>> strat_returns_acc_slices["U"].sharpe_ratio().vbt.heatmap(
...     x_level="fast_sma_timeperiod",
...     y_level="slow_sma_timeperiod",
...     symmetric=True
... ).show()
```



Downtrends

```
>>> hold_returns_acc_slices["D"].sharpe_ratio()
-1.329832516209626

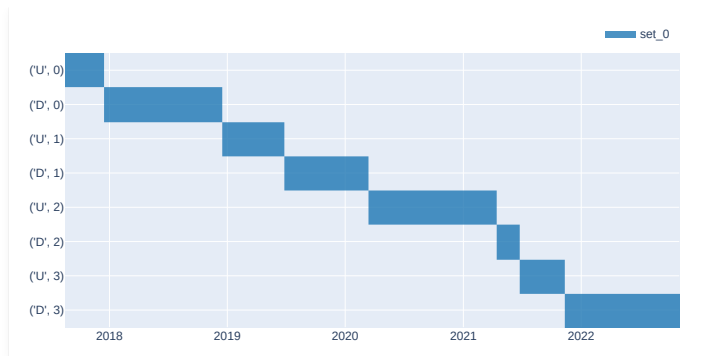
>>> strat_returns_acc_slices["D"].sharpe_ratio().vbt.heatmap(
...     x_level="fast_sma_timeperiod",
...     y_level="slow_sma_timeperiod",
...     symmetric=True
... ).show()
```



We can see that it takes a lot of effort (and some may say luck) to pick the right parameter combination and consistently beat the baseline. Since both pictures are completely different, we cannot rely on a single parameter combination; rather, we have to recognize the regime we're in and act accordingly, which is a massive challenge on its own. The above analysis should be used with caution though: there may be a position overflow from one market regime to another skewing the results since both periods are part of a single backtest. But at least we gained another bit of valuable information 🧐

But what if we try to take slices from the portfolio? The operation would fail because each split contains gaps, and portfolio cannot be indexed with gaps, only using non-interrupting ranges. Thus, we would need to break up each split into multiple smaller splits, also called "split parts". Luckily for us, there are two functionalities that make this possible: `Splitter.split_range` accepts an option `"by_gap"` as `new_split` to split a range by gap, while `Splitter.break_up_splits` can apply this operation on each split. This way, we can flatten the splits such that only one trend period is processed at once. We'll also sort the splits by their start index such that they come in the same temporal order as the labels:

```
>>> trend_splitter = trend_splitter.break_up_splits("by_gap", sort=True)
>>> trend_splitter.plot().show()
```



Another trick: instead of calling `Splitter.take`, we can call `split` on the object directly!

```
>>> strat_pf_slices = pf.split(trend_splitter)
>>> strat_pf_slices
trend split_part
U 0 Portfolio(\n wrapper=ArrayWrapper(\n ...
D 0 Portfolio(\n wrapper=ArrayWrapper(\n ...
U 1 Portfolio(\n wrapper=ArrayWrapper(\n ...
D 1 Portfolio(\n wrapper=ArrayWrapper(\n ...
U 2 Portfolio(\n wrapper=ArrayWrapper(\n ...
D 2 Portfolio(\n wrapper=ArrayWrapper(\n ...
U 3 Portfolio(\n wrapper=ArrayWrapper(\n ...
D 3 Portfolio(\n wrapper=ArrayWrapper(\n ...
dtype: object
```

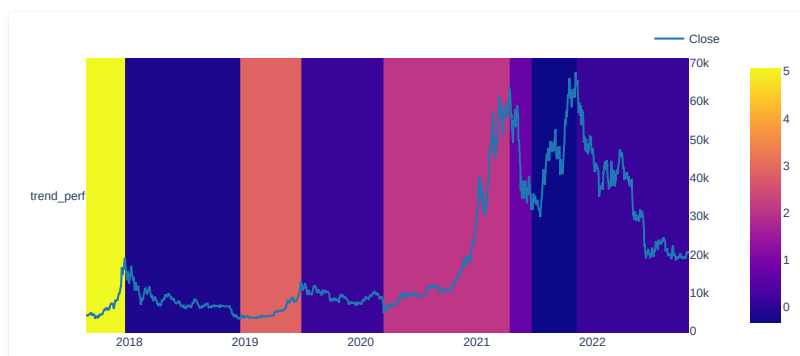
Let's analyze the median, that is, there are 50% of the parameter combinations with the same value or better, and, conversely, 50% with the same value or worse.

```
>>> trend_range_perf = strat_pf_slices.apply(lambda pf: pf.sharpe_ratio)
>>> median_trend_perf = trend_range_perf.median(axis=1)
>>> median_trend_perf
trend split_part
U 0 4.829429
D 0 -0.058159
U 1 2.874709
D 1 -0.510703
U 2 2.220633
D 2 0.917946
U 3 -0.736989
D 3 0.225841
dtype: float64
```

And now visually: replace the labels in `trend1b` by the freshly-computed values.

```
>>> trend_perf_ts = data.symbol_wrapper.fill().rename("trend_perf")
>>> for label, sr in trend_splitter.bounds.iterrows():
...     trend_perf_ts.iloc[sr["start"]:sr["end"]] = median_trend_perf[label]
>>> data.close.vbt.overlay_with_heatmap(trend_perf_ts).show()
```

1 Using `GenericAccessor.overlay_with_heatmap`



Can you spot something strange? Right, at least 50% of the parameter combinations during the last uptrend have a negative Sharpe 😬 At least my explanation is that moving averages are lagging indicators and by the time they reacted to the previous sharp decline, we already arrived at the next top; basically, the rebound took them off-guard such that any short positions from the previous decline couldn't close out on time.

Column stacking

This was a glimpse into the second approach that we mentioned at the beginning of this page, which involves applying a UDF on each range separately. But how about stacking all the slices into a single one and applying our `get_total_return` just once for a nice performance boost? Turns out, we can stack the slices manually using `pandas.concat`:

```
>>> close_stacked = pd.concat(
```

```
...     close_slices.values.tolist(),
...     axis=1, ①
...     keys=close_slices.index ②
... )
>>> close_stacked
split_year
set          2018      2019      2020      2021
Open time
2018-01-01 00:00:00+00:00 13380.00 NaN      NaN      NaN      NaN      NaN
2018-01-02 00:00:00+00:00 14675.11 NaN      NaN      NaN      NaN      NaN
2018-01-03 00:00:00+00:00 14919.51 NaN      NaN      NaN      NaN      NaN
...
2021-12-29 00:00:00+00:00      NaN      NaN      NaN      NaN      NaN 46464.66
2021-12-30 00:00:00+00:00      NaN      NaN      NaN      NaN      NaN 47120.87
2021-12-31 00:00:00+00:00      NaN      NaN      NaN      NaN      NaN 46216.93

[1461 rows x 8 columns]
```

① Along columns

② Slice labels become column labels

As we can see, even though the operation produced a lot of NaNs, we now have a format that is perfectly acceptable by vectorbt. Let's apply our UDF to this array:

```
>>> get_total_return(close_stacked)
split_year  set
2018        train  -0.522416
           test   -0.417491
2019        train   1.858493
           test   -0.322797
2020        train   0.269093
           test    2.132976
2021        train   0.194783
           test    0.379417
Name: total_return, dtype: float64
```

Pure magic ✨

But the stacking approach above works only if we're splitting Pandas objects, but what about NumPy arrays and complex vectorbt objects? We can instruct `Splitter.take` to do the stacking job for us. The method will use `column_stack_merge` for stacking any array-like objects in the most efficient way. For example, let's replicate the above format using a single line of code:

```
>>> close_stacked = splitter.take(data.close, into="stacked")
>>> close_stacked.shape
(1461, 8)
```

To get rid of many NaNs in the stacked data, we can reset the index of each slice prior to stacking by adding a prefix "reset_" to any "stacked" option provided as `into`:

```
>>> close_stacked = splitter.take(data.close, into="reset_stacked")
>>> close_stacked
split_year      2018      2019      2020      \
set      train  test  train  test  train  test
0      13380.00  6356.81  3797.14  10624.93  7200.85  9232.00
1      14675.11  6615.29  3858.56  10842.85  6965.71  9086.54
2      14919.51  6513.86  3766.78  11940.00  7344.96  9058.26
..      ...      ...      ...      ...      ...      ...
181      NaN    3695.32      NaN    7388.24  9138.55  27385.00
182      NaN    3801.91      NaN    7246.00      NaN  28875.54
183      NaN    3702.90      NaN    7195.23      NaN  28923.63

split_year      2021
set      train  test
0      29331.69  33504.69
1      32178.33  33786.55
2      33000.05  34669.13
..      ...      ...
181      NaN    46464.66
182      NaN    47120.87
183      NaN    46216.93

[184 rows x 8 columns]
```

We can also instruct the method to align each slice by the end point rather than the start point, which will push NaNs to the beginning of the array:

```
>>> close_stacked = splitter.take(data.close, into="from_end_stacked")
>>> close_stacked
split_year      2018      2019      2020      \
set      train  test  train  test  train  test
0      NaN    6356.81      NaN  10624.93      NaN  9232.00
1      NaN    6615.29      NaN  10842.85      NaN  9086.54
2      NaN    6513.86      NaN  11940.00  7200.85  9058.26
..      ...      ...      ...      ...      ...      ...
181    5853.98  3695.32  12400.63  7388.24  9116.35  27385.00
182    6197.92  3801.91  11903.13  7246.00  9192.56  28875.54
183    6390.07  3702.90  10854.10  7195.23  9138.55  28923.63

split_year      2021
set      train  test
0      NaN    33504.69
1      NaN    33786.55
```

```

2          NaN  34669.13
..          ...      ...
181       34494.89  46464.66
182       35911.73  47120.87
183       35045.00  46216.93

```

[184 rows x 8 columns]

Hint

If all slices have the same length, both alignments will produce the same array.

As we can see, there are two potential issues associated with this operation: the final array will have no datetime index, and the slices may have different lengths such that there will still be some NaNs present in the array. But also, we are rarely interested in stacking training and test sets together since they are bound to different pipelines, thus let's only stack the splits that belong to the same set, which will also produce slices of roughly the same length, by using "reset_stacked_splits" as `into`:

```

>>> close_stacked = splitter.take(data.close, into="reset_stacked_by_set")
>>> close_stacked
set
train  split_year    2018    2019    2020    2...
test   split_year    2018    2019    2020    2...
dtype: object

```

We've got some weird-looking format once again, but it's nothing more than a Series with the set labels as index and the stacked close price slices as values:

```

>>> close_stacked["train"]
split_year    2018    2019    2020    2021
0      13380.00  3797.14  7200.85  29331.69
1      14675.11  3858.56  6965.71  32178.33
2      14919.51  3766.78  7344.96  33000.05
..          ...      ...      ...      ...
179     6197.92  11903.13  9116.35  35911.73
180     6390.07  10854.10  9192.56  35045.00
181          NaN      NaN   9138.55      NaN

```

[182 rows x 4 columns]

By resetting the index, we save a tremendous amount of RAM: our arrays hold only $182 * 8 = 1456$ instead of $1461 * 8 = 11688$ (i.e., 88% less) values in memory. But how do we access the index associated with each column? We can slice it as a regular array!

```

>>> index_slices = splitter.take(data.index)
>>> index_slices
split_year  set
2018      train  DatetimeIndex(['2018-01-01 00:00:00+00:00', '2...
           test   DatetimeIndex(['2018-07-01 00:00:00+00:00', '2...
2019      train  DatetimeIndex(['2019-01-01 00:00:00+00:00', '2...
           test   DatetimeIndex(['2019-07-01 00:00:00+00:00', '2...
2020      train  DatetimeIndex(['2020-01-01 00:00:00+00:00', '2...
           test   DatetimeIndex(['2020-07-01 00:00:00+00:00', '2...
2021      train  DatetimeIndex(['2021-01-01 00:00:00+00:00', '2...
           test   DatetimeIndex(['2021-07-01 00:00:00+00:00', '2...
dtype: object

```

Another way of getting the index information as by attaching the bounds to the range labels using `attach_bounds`, which can be `True` and "index" to attach the bounds in the integer and datetime format respectively. Let's use the latter but also enable `right_inclusive` to make the right bound inclusive if we want to use it in indexing with `loc`:

```

>>> close_stacked_wb = splitter.take(
...     data.close,
...     into="reset_stacked_by_set",
...     attach_bounds="index",
...     right_inclusive=True
... )
>>> close_stacked_wb["train"]
split_year    2018    2019 \
start  2018-01-01 00:00:00+00:00  2019-01-01 00:00:00+00:00
end    2018-06-30 00:00:00+00:00  2019-06-30 00:00:00+00:00
0      13380.00      3797.14
1      14675.11      3858.56
2      14919.51      3766.78
..          ...      ...
179     6197.92     11903.13
180     6390.07     10854.10
181          NaN      NaN

split_year    2020    2021
start  2020-01-01 00:00:00+00:00  2021-01-01 00:00:00+00:00
end    2020-06-30 00:00:00+00:00  2021-06-30 00:00:00+00:00
0      7200.85      29331.69
1      6965.71      32178.33
2      7344.96      33000.05
..          ...      ...
179     9116.35      35911.73
180     9192.56      35045.00
181     9138.55      NaN

```

[182 rows x 4 columns]

Though, to keep the index clean, we will go with the arrays without bounds first. We've established two arrays: one for training and another one for testing purposes. Let's modify our pipeline `sma_crossover_perf` from the first page to be run on a single set: replace the argument `data` with `close`, and add another argument for the frequency required by Sharpe since our index isn't datetime-like anymore.

```
>>> @vbt.parameterized(merge_func="concat")
... def set_sma_crossover_perf(close, fast_window, slow_window, freq):
...     fast_sma = vbt.talib("sma").run(
...         close, fast_window, short_name="fast_sma", hide_params=True) ①
...     slow_sma = vbt.talib("sma").run(
...         close, slow_window, short_name="slow_sma", hide_params=True)
...     entries = fast_sma.real_crossed_above(slow_sma)
...     exits = fast_sma.real_crossed_below(slow_sma)
...     pf = vbt.Portfolio.from_signals(
...         close, entries, exits, freq=freq, direction="both")
...     return pf.sharpe_ratio
```

① Do not add parameter to the index as it will be added by the `parameterized` decorator

Apply it on the training set to get the performance per parameter combination and split:

```
>>> train_perf = set_sma_crossover_perf(
...     close_stacked["train"],
...     vbt.Param(np.arange(5, 50), condition="x < slow_window"),
...     vbt.Param(np.arange(5, 50)),
...     data.index.freq,
...     _execute_kwargs=dict(
...         clear_cache=50,
...         collect_garbage=50
...     )
... )
```

Combination 990/990

```
>>> train_perf
fast_window  slow_window  split_year
5            6            2018        1.158471
                    2019        1.901410
                    2020       -0.426441
                    2021        0.052654
                    2018        2.231909
                    ....
47           49            2021       -0.446099
48           49            2018       -1.169584
                    2019        3.727154
                    2020       -1.913321
                    2021       -0.400270
Name: sharpe_ratio, Length: 3960, dtype: float64
```

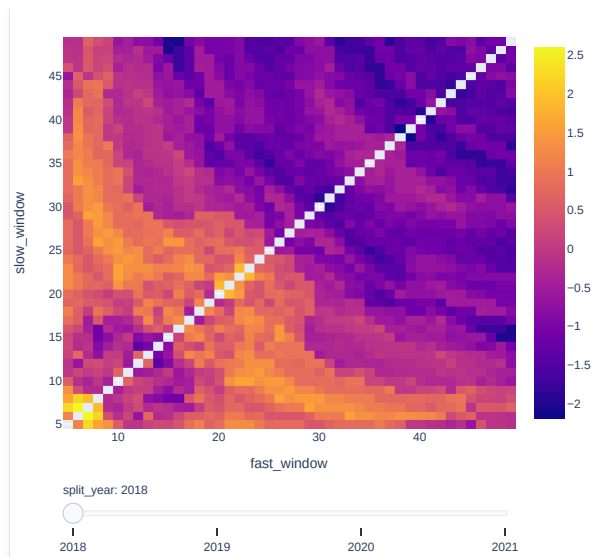
A total of 990 parameter combinations were run in 20 seconds, or 20ms per run 🚀

Let's generate the performance heatmap for each split:

```
>>> train_perf.vbt.heatmap(
...     x_level="fast_window",
...     y_level="slow_window",
...     slider_level="split_year",
...     symmetric=True
... ).show()
```

 **Dashboard**

Run the notebook to view the dashboard!



By looking at the plot above, we can identify a number of yellow points that may be good candidates for our strategy. But, in contrast to the approach from the first page where we selected the best parameter combination, let's conduct a neighborhood analysis by finding a Sharpe ratio that's surrounded by high Sharpe ratios, so that we can diminish the effect of outliers and bring in more robustness into our optimizer.

This query can be well quantified by transforming the `train_perf` Series into a DataFrame using the accessor method `BaseAccessor.unstack_to_df` and running the accessor method `GenericAccessor.proximity_apply`, which rolls a two-dimensional window over the entire matrix and reduces this window using a UDF. This mini-pipeline must be applied separately to each split. We will use a window of 2, that is, the two rows and columns that surround the central point, which yields a total of $(n * 2 + 1) ** 2 = 25$ values. As a UDF we'll take the median, that is, each value will mean that 50% or more of the surrounding values are better than it. Our UDF will also filter out all the windows that have less than 20 valid values.

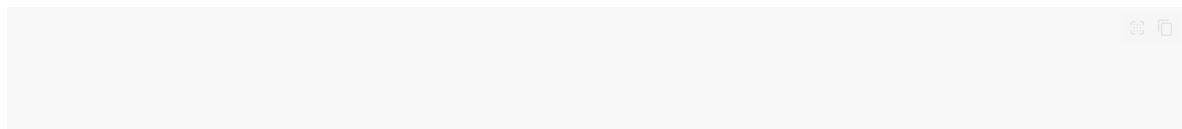
```
>>> @njit
... def prox_median_nb(arr): ❶
...     if (~np.isnan(arr)).sum() < 20:
...         return np.nan
...     return np.nanmedian(arr) ❷

>>> prox_perf_list = []
>>> for split_label, perf_sr in train_perf.groupby("split_year"): ❸
...     perf_df = perf_sr.vbt.unstack_to_df(0, [1, 2]) ❹
...     prox_perf_df = perf_df.vbt.proximity_apply(2, prox_median_nb) ❺
...     prox_perf_sr = prox_perf_df.stack([0, 1]) ❻
...     prox_perf_list.append(prox_perf_sr.reindex(perf_sr.index)) ❼

>>> train_prox_perf = pd.concat(prox_perf_list) ❽
>>> train_prox_perf
fast_window  slow_window  split_year
5            6           2018      NaN
              6           2019      NaN
              6           2020      NaN
              6           2021      NaN
              7           2018      NaN
              ..
47           49           2021      NaN
48           49           2018      NaN
              49           2019      NaN
              49           2020      NaN
              49           2021      NaN
Length: 3960, dtype: float64
```

- ❶ This function will take the median out of the proximity windows. Argument `arr` is a two-dimensional window flattened to one dimension.
- ❷ Since our window still may contain NaN values, use the NaN-resistant median
- ❸ Run the following per split
- ❹ Make a DataFrame with one level as rows, and two other levels as columns. We could have also used level names.
- ❺ Run our smoothing window over the entire DataFrame
- ❻ Make a Series by stacking all columns. Again, we could have also used level names.
- ❼ Reindex the Series to match the original index and append to list
- ❽ Stack all Series in the list into one big Series

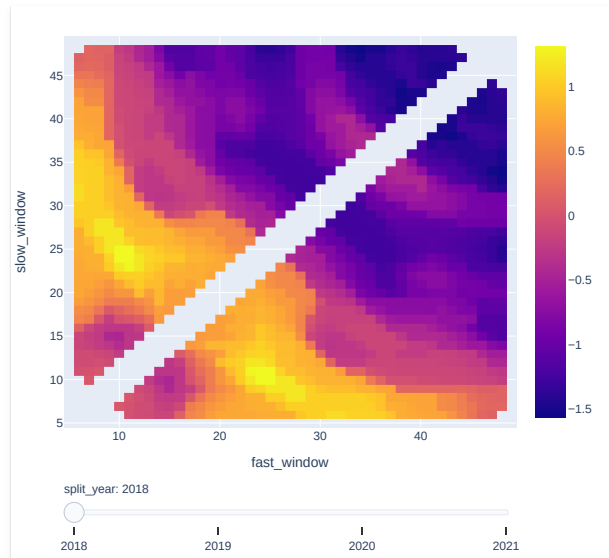
At first, it may appear that all values are NaN, but let's prove otherwise:




```
>>> train_prox_perf.vbt.heatmap(
...     x_level="fast_window",
...     y_level="slow_window",
...     slider_level="split_year",
...     symmetric=True
... ).show()
```

Dashboard

Run the notebook to view the dashboard!



Fans of [computer vision](#) will recognize what we did above: we used a 5x5 neighboring window acting as a [filter](#) that replaces each pixel with the median pixel value of it and a neighborhood window of adjacent pixels. The effect is a more smooth image with sharp features removed (in our case Sharpe outliers). For example, in the first split, the highest Sharpe fell from 2.5 to 1.3, which is still extraordinary since it means that there are points that have at least 50% of surrounding points with a Sharpe 1.3 or higher! We can now search in each split for the parameter combination that has the highest proximity performance:

```
>>> best_params = train_prox_perf.groupby("split_year").idxmax()
>>> best_params = train_prox_perf[best_params].index
>>> train_prox_perf[best_params]
fast_window  slow_window  split_year
10           24          2018        1.311910
9            39          2019        3.801643
14           19          2020        2.077684
31           41          2021        2.142695
dtype: float64
```

What we're waiting for? Let's test those combinations on our test set! But wait, how do we apply each parameter combination on each column in the test array? Isn't each parameter combination being applied on the entire input? That's right, but there's a trick: use templates to instruct the `parameterized` decorator to pass only one column at a time, depending on the parameter combination being processed.

```
>>> test_perf = set_sma_crossover_perf(
...     vbt.RepEval(
...         "test_close.iloc[:, [config_idx]]", ❶
...         context=dict(test_close=close_stacked["test"])
...     ),
...     vbt.Param(best_params.get_level_values("fast_window"), level=0), ❷
...     vbt.Param(best_params.get_level_values("slow_window"), level=0),
...     data.index.freq
... )
>>> test_perf
fast_window  slow_window  split_year
10           24          2018        -0.616204
9            39          2019         0.017269
14           19          2020         4.768589
31           41          2021        -0.363900
Name: sharpe_ratio, dtype: float64
```

- ❶ This template will be substituted during processing of each parameter combination (also called a parameter config). Here, we select the column from our test data that has the same index as the parameter config. We wrap the index with a list to run the pipeline on a DataFrame with meaningful columns.
- ❷ Extract the parameter values and assign both `fast_window` and `slow_window` the same level to avoid building a product between them

Let's compare these values against the baseline:

```
>>> def get_index_sharpe(index):
...     return data.loc[index].run("from_holding").sharpe_ratio
>>> index_slices.xs("test", level="set").apply(get_index_sharpe) ❶
```

```
split_year
2018    -1.327655
2019    -0.788038
2020     4.425057
2021     1.304871
dtype: float64
```

- ① Each value in `index_slices` is an index, so we simply apply `get_index_sharpe` on it!

Not bad! Our model beats the baseline for three years in a row.

Row stacking

The same way as we stacked ranges along columns, we can stack ranges along rows. The major difference between both approaches is that column stacking is meant for producing independent tests while row stacking puts ranges into the same test and thus introduces a temporal dependency between them.

What row stacking is perfect for is block resampling required for time-series bootstrapping. The bootstrap is a flexible and powerful statistical tool that can be used to quantify the uncertainty. Rather than repeatedly obtaining independent datasets (which are limited in finance), we instead obtain distinct datasets by repeatedly sampling observations from the original dataset. Each of these "bootstrap datasets" is created by sampling *with replacement*, and is the same size as our original dataset. As a result some observations may appear more than once and some not at all (about two-thirds of the original data points appear in each bootstrap sample). Since we're working on time series, we can't simply sample the observations with replacement; rather, we should create blocks of consecutive observations, and sample those. Then we paste together sampled blocks to obtain a bootstrap dataset (learn more [here](#)).

For our example, we'll use the **moving block bootstrap**, which involves rolling a fixed-size window with an offset of just one bar:

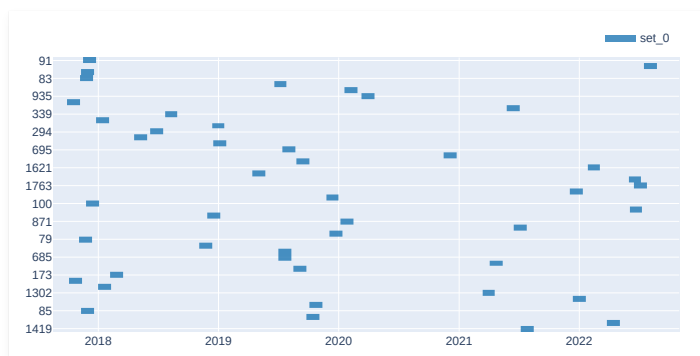
```
>>> block_size = int(3.15 * len(data.index) ** (1 / 3)) ①
>>> block_size
39

>>> block_splitter = vbt.Splitter.from_rolling(
...     data.index,
...     length=block_size,
...     offset=1,
...     offset_anchor="prev_start" ②
... )
>>> block_splitter.n_splits
1864
```

- ① Default block length used in the R package
- ② By default, the anchor is the right bound of the previous window

We've generated 1864 blocks. The next step is sampling. To generate a single sample, shuffle the blocks (i.e., splits) with replacement, which can be easily done using `Splitter.shuffle_splits`. We also need to limit the number of blocks such that they have roughly the same number of data points as our original data:

```
>>> size = int(block_splitter.n_splits / block_size)
>>> sample_splitter = block_splitter.shuffle_splits(size=size, replace=True)
>>> sample_splitter.plot().show()
```



Let's compute the returns, "take" the slices corresponding to the blocks in the splitter, and stack them along rows:

```
>>> returns = data.returns
>>> sample_rets = sample_splitter.take(
...     returns,
...     into="stacked", ①
...     stack_axis=0 ②
... )
>>> sample_rets
split  Open time
920    2020-02-23 00:00:00+00:00    0.029587
      2020-02-24 00:00:00+00:00   -0.028206
      2020-02-25 00:00:00+00:00   -0.035241
      2020-02-26 00:00:00+00:00   -0.056956
      2020-02-27 00:00:00+00:00    0.004321
      ...
1617   2022-02-23 00:00:00+00:00   -0.025642
      2022-02-24 00:00:00+00:00    0.028918
      2022-02-25 00:00:00+00:00    0.023272
      2022-02-26 00:00:00+00:00   -0.002612
```

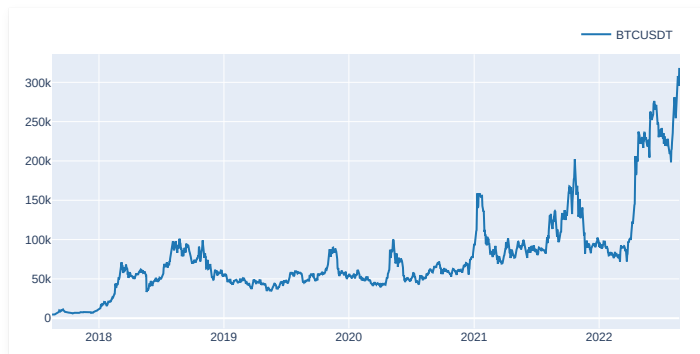
```
2022-02-27 00:00:00+00:00 -0.036242
Name: Close, Length: 1833, dtype: float64
```

- 1 Stack all slices into one Series
- 2 Along rows

We've created a "frankenstein" price series!

```
>>> sample_rets.index = data.index[:len(sample_rets)] 1
>>> sample_cumrets = data.close[0] * (sample_rets + 1).cumprod() 2
>>> sample_cumrets.vbt.plot().show()
```

- 1 Plotting requires a proper index
- 2 Compute the cumulative returns and project them from the start of our data



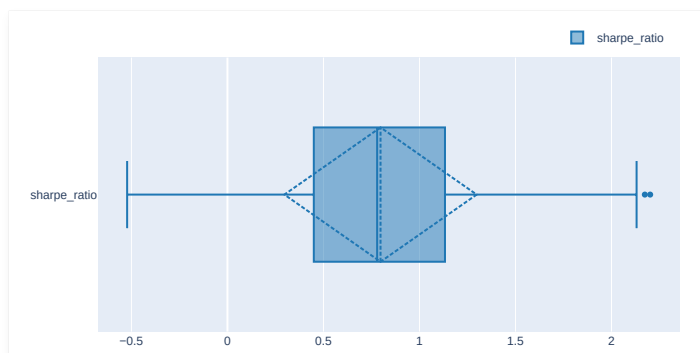
But one sample is not enough: we need to generate 100, 1000, or even 10000 samples for our estimates to be as accurate as possible.

```
>>> samples_rets_list = []
>>> for i in vbt.ProgressBar(range(1000)):
...     sample_spl = block_splitter.shuffle_splits(size=size, replace=True)
...     sample_rets = sample_spl.take(returns, into="stacked", stack_axis=0)
...     sample_rets.index = returns.index[:len(sample_rets)]
...     sample_rets.name = i
...     samples_rets_list.append(sample_rets)
>>> sample_rets_stacked = pd.concat(samples_rets_list, axis=1)
```

Sample 1000/1000

We can then analyze the distribution of a statistic of interest:

```
>>> sample_sharpe = sample_rets_stacked.vbt.returns.sharpe_ratio()
>>> sample_sharpe.vbt.boxplot(horizontal=True).show()
```



This histogram provides an estimate of the shape of the distribution of the sample Sharpe from which we can answer questions about how much the Sharpe varies across samples. Since this particular bootstrap distribution is symmetric, we can use percentile-based confidence intervals. For example, a 95% confidence interval translates to the 2.5th and 97.5th percentiles:

```
>>> sample_sharpe.quantile(0.025), sample_sharpe.quantile(0.975)
(-0.13636235254958026, 1.726050620753774)
```

The method here can be applied to almost any other statistic or estimator, including more complex backtesting metrics.

Applying

If the "taking" approach provides us with object slices that we can work upon, the "applying" approach using `Splitter.apply` runs a UDF on each range, and can not only do the "taking" part for us, but also easily merge the outputs of the UDF. First, it resolves the ranges it needs to iterate over; similarly to other methods, it takes

`split_group_by` and `set_group_by` to merge ranges, but also the arguments `split` and `set_` that can be used to select specific (merged) ranges. Then, while iterating over each range, it substitutes any templates and other instructions in the positional and keyword arguments meant to be passed to the UDF. For example, by wrapping any argument with the class `Takeable`, the method will select a slice from it and substitute the instruction with that slice. The arguments prepared in each iteration are saved in a list and passed to the executor - `execute`, which you're probably already familiar with. The executor lazily executes all the iterations and (optionally) merges the outputs. *Lazily* here means that none of the arrays will be sliced until the range is executed - good for memory ❤️

Let's run a simple example of calculating the total return of each close slice:

```
>>> splitter.apply(
...     get_total_return,
...     vbt.Takeable(data.close), ❶
...     merge_func="concat" ❷
... )
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test    2.132976
2021      train   0.194783
          test    0.379417
dtype: float64
```

❶ Any positional arguments passed after the function will be passed to the function. We could have also passed it as a keyword argument

```
sr=vbt.Takeable(data.close).
```

❷ See [resolve_merge_func](#)

That's how easy it is!

We could have also used templates:

```
>>> splitter.apply(
...     get_total_return,
...     vbt.RepFunc(lambda range_: data.close[range_]), ❶
...     merge_func="concat"
... )
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test    2.132976
2021      train   0.194783
          test    0.379417
dtype: float64
```

❶ `range_` will be substituted with the current range

Or, by manually selecting the range inside the function:

```
>>> def get_total_return(range_, data):
...     return data.returns[range_].vbt.returns.total()

>>> splitter.apply(
...     get_total_return,
...     vbt.Rep("range_"),
...     data,
...     merge_func="concat"
... )
split_year  set
2018      train  -0.534128
          test   -0.420523
2019      train   1.931243
          test   -0.337096
2020      train   0.270084
          test    2.165013
2021      train   0.211639
          test    0.318788
dtype: float64
```

Hint

Results are slightly different because slicing returns is more accurate than slicing the price.

Taking from complex vectorbt objects works too:

```
>>> def get_total_return(data):
...     return data.returns.vbt.returns.total()

>>> splitter.apply(
...     get_total_return,
...     vbt.Takeable(data), ❶
...     merge_func="concat"
... )
split_year  set
2018      train  -0.522416
```

```

2019    test    -0.417491
        train    1.858493
2020    test    -0.322797
        train    0.269093
2021    test    2.132976
        train    0.194783
        test    0.379417
dtype: float64

```

1 Splits the `Data` instance

Let's run the function above on the entire split:

```

>>> splitter.apply(
...     get_total_return,
...     vbt.Takeable(data),
...     set_group_by=True,
...     merge_func="concat"
... )
split_year
2018    -0.723251
2019     0.894908
2020     3.016697
2021     0.575665
dtype: float64

```

If we need to select specific ranges to run the pipeline upon, use `split` and `set`, which can be an integer, a label, or a sequence of such:

```

>>> splitter.apply(
...     get_total_return,
...     vbt.Takeable(data),
...     split=[2020, 2021],
...     set="train",
...     merge_func="concat"
... )
split_year
2020     0.270084
2021     0.211639
dtype: float64

```

Let's apply this approach to cross-validate our SMA crossover strategy:

```

>>> train_perf = splitter.apply(
...     sma_crossover_perf,
...     vbt.Takeable(data),
...     vbt.Param(np.arange(5, 50), condition="x < slow_window"), ①
...     vbt.Param(np.arange(5, 50)),
...     _execute_kwargs=dict( ②
...         clear_cache=50,
...         collect_garbage=50
...     ),
...     set="train", ③
...     merge_func="concat"
... )

```

① Parameters are processed in `sma_crossover_perf`, not in `splitter.apply`

② This dictionary is also getting passed to `sma_crossover_perf`. We'll hide the progress bar for parameter combinations and display the one for splits below.

③ We could have also selected this set using `splitter.loc["train"]`

Split 4/4

```

>>> train_perf
split_year  fast_window  slow_window
2018        5           6           1.161661
              7           2.238117
              8           1.583246
              9           1.369400
             10          0.563251
              ...
2021       46          48          -0.238667
              49          -0.437822
              47          48          -0.799317
              49          -0.447324
              48          49          -0.401369
Length: 3960, dtype: float64

```

Let's find the best parameter combinations and pass them to the test set using templates:

```

>>> best_params = train_perf.groupby("split_year").idxmax()
>>> best_params = train_perf[best_params].index
>>> train_perf[best_params]
split_year  fast_window  slow_window
2018        6           7           2.612006
2019       35          36           4.246530
2020       16          17           2.605763
2021       32          45           3.073246
dtype: float64

>>> best_fast_windows = best_params.get_level_values("fast_window")

```

```

>>> best_slow_windows = best_params.get_level_values("slow_window")

>>> test_perf = splitter.apply(
...     sma_crossover_perf,
...     vbt.Takeable(data),
...     vbt.RepFunc(lambda split_idx: best_fast_windows[split_idx]), ❶
...     vbt.RepFunc(lambda split_idx: best_slow_windows[split_idx]),
...     set_="test",
...     merge_func="concat"
... )
>>> test_perf
split_year
2018    1.501380
2019    0.152723
2020    4.473387
2021   -1.147178
dtype: float64

```

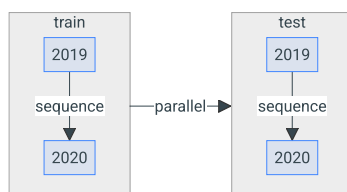
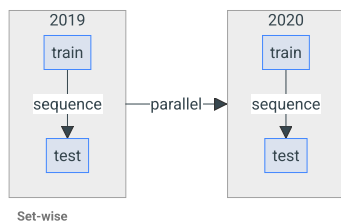
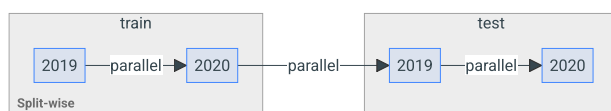
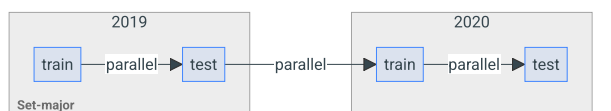
❶ `split_idx` will be substituted by the split number. Make sure that the values in `best_fast_windows` and `best_slow_windows` appear in the same order as the splits in the splitter. Or, use `split_label` if the order is different.

We've effectively outsourced the range iteration, taking, and execution steps.

Iteration schemes

The method `Splitter.apply` supports multiple iteration schemes. For instance, it can iterate over the selected ranges in the split-major or set-major order as a single sequence. It can also pack the splits/sets of the same set/split into the same bucket and run as a single iteration - split-wise and set-wise respectively. Why does it matter? Consider a scenario where a UDF needs to write the results to an object and then access these results in a specific order. For example, we may want a OOS range to read the results of the previous IS range, which is a requirement of parameterized CV; in such a case, the iteration can be in any order if the execution is sequential, in any major order if it (carefully) spans across multiple threads, and in the split-wise order if the execution spans across multiple processes since the sets need to share the same memory.

Split-major



Let's cross-validate our SMA crossover strategy using a single call! When an IS range is processed, find out the parameter combination with the highest Sharpe ratio and return it. When an OOS range is processed, access the parameter combination from the previous IS range and use it for testing. If you take a closer look at the available context at each iteration, there is no variable that holds the previous results: we need to store, write, and access the results manually. We'll iterate in a set-major order such that the splits in the training set are processed first.

```

>>> def cv_sma_crossover(
...     data,
...     fast_windows,
...     slow_windows,
...     split_idx, ❶
...     set_idx, ❷
...     train_perf_list ❸
... ):
...     if set_idx == 0: ❹
...         train_perf = sma_crossover_perf(
...             data,

```

```

...     vbt.Param(fast_windows, condition="x < slow_window"),
...     vbt.Param(slow_windows),
...     _execute_kwargs=dict(
...         clear_cache=50,
...         collect_garbage=50
...     )
... )
... train_perf_list.append(train_perf) ❸
... best_params = train_perf.idxmax()
... return train_perf[[best_params]] ❹
... else:
...     train_perf = train_perf_list[split_idx] ❺
...     best_params = train_perf.idxmax()
...     test_perf = sma_crossover_perf(
...         data,
...         vbt.Param([best_params[0]]), ❻
...         vbt.Param([best_params[1]]),
...     )
...     return test_perf ❼

```

```

>>> train_perf_list = []
>>> cv_perf = splitter.apply(
...     cv_sma_crossover,
...     vbt.Takeable(data),
...     np.arange(5, 50),
...     np.arange(5, 50),
...     vbt.Rep("split_idx"),
...     vbt.Rep("set_idx"),
...     train_perf_list,
...     iteration="set_major",
...     merge_func="concat"
... )

```

- ❶ Substituted by the current split number
- ❷ Substituted by the current set number
- ❸ List with the results from the training set
- ❹ If we're currently in the training set, run the optimization
- ❺ Append the results to the list
- ❻ Return the best result, which will be merged with the test results. Wrap with a list to return a Series rather than a single value (we want a nice index).
- ❼ Access the training results of the current split and test the best result
- ❽ Wrap with `Param` for `sma_crossover_perf` to return a Series rather than a single value (we want a nice index)
- ❼ Return the test result

Iteration 8/8

We've got 8 iterations, one per range: the first 4 iterations correspond to the training ranges, which took the most time to execute, while the last 4 iterations correspond to the test ranges, which executed almost instantly because of just one parameter combination. Let's concatenate the training results from `train_perf_dict` and see what's inside `cv_perf`:

```

>>> train_perf = pd.concat(train_perf_list, keys=splitter.split_labels)
>>> train_perf

```

split_year	fast_window	slow_window	
2018	5	6	1.161661
		7	2.238117
		8	1.583246
		9	1.369400
		10	0.563251
			...
2021	46	48	-0.238667
		49	-0.437822
		48	-0.799317
		49	-0.447324
		48	-0.401369

Length: 3960, dtype: float64

```

>>> cv_perf

```

set	split_year	fast_window	slow_window	
train	2018	6	7	2.612006
	2019	35	36	4.246530
	2020	16	17	2.605763
	2021	32	45	3.073246
test	2018	6	7	1.501380
	2019	35	36	0.152723
	2020	16	17	4.473387
	2021	32	45	-1.147178

dtype: float64

Same results, awesome!

Merging

Another great feature of this method is that it can merge arbitrary outputs: from single values and Series, to DataFrames and even tuples of such. There are two main merging options available: merging all outputs into a single object (`merge_all=True`) and merging by the main unit of iteration (`merge_all=False`). The first option does the following: it flattens all outputs into a single sequence, resolves the merging function using `resolve_merge_func`, and calls the merging function

on that sequence. If the merging function is not specified, it wraps the sequence into a Pandas Series (even if each output is a complex object). If each output is a tuple, returns multiple of such Series. Let's illustrate this by returning entries and exits, and stacking them along columns:

```
>>> def get_entries_and_exits(data, fast_window, slow_window):
...     fast_sma = data.run("sma", fast_window, short_name="fast_sma")
...     slow_sma = data.run("sma", slow_window, short_name="slow_sma")
...     entries = fast_sma.real_crossed_above(slow_sma)
...     exits = fast_sma.real_crossed_below(slow_sma)
...     return entries, exits

>>> entries, exits = splitter.apply(
...     get_entries_and_exits,
...     vbt.Takeable(data),
...     20,
...     30,
...     merge_func="column_stack"
... )

>>> entries
split_year
set
2018
2019
2020
2021
train test train test train test train test
Open time
2018-01-01 00:00:00+00:00 False NaN NaN NaN NaN NaN NaN NaN
2018-01-02 00:00:00+00:00 False NaN NaN NaN NaN NaN NaN NaN
2018-01-03 00:00:00+00:00 False NaN NaN NaN NaN NaN NaN NaN
...
2021-12-29 00:00:00+00:00 NaN NaN NaN NaN NaN NaN NaN False
2021-12-30 00:00:00+00:00 NaN NaN NaN NaN NaN NaN NaN False
2021-12-31 00:00:00+00:00 NaN NaN NaN NaN NaN NaN NaN False

[1461 rows x 8 columns]
```

We can then replace NaN with `False` and backtest them. If you don't want to have that many NaNs, use `"reset_column_stack"` as `merge_func`. We can also provide multiple merging functions (as a tuple) in case the outputs of our UDF have different formats.

Note

Even though you can return multiple different formats, the formats must remain the same across all ranges!

As said previously, we can also merge outputs by the main unit of iteration. Let's run the same UDF but only stack the masks that belong to the same split:

```
>>> entries, exits = splitter.apply(
...     get_entries_and_exits,
...     vbt.Takeable(data),
...     50,
...     200,
...     merge_all=False,
...     merge_func="row_stack"
... )

>>> entries.loc[2018]
set Open time
train 2018-01-01 00:00:00+00:00 False
2018-01-02 00:00:00+00:00 False
2018-01-03 00:00:00+00:00 False
2018-01-04 00:00:00+00:00 False
2018-01-05 00:00:00+00:00 False
...
test 2018-12-27 00:00:00+00:00 False
2018-12-28 00:00:00+00:00 False
2018-12-29 00:00:00+00:00 False
2018-12-30 00:00:00+00:00 False
2018-12-31 00:00:00+00:00 True
Length: 365, dtype: bool
```

This way, each mask covers an entire year and can be backtested as a whole. The additional level `set` provides us with the information on which set each timestamp belongs to.

In a case where a range's start and end date cannot be inferred from the merged data alone, we can instruct the method to attach this information. Let's get the total number of signals:

```
>>> def get_signal_count(*args, **kwargs):
...     entries, exits = get_entries_and_exits(*args, **kwargs)
...     return entries.vbt.signals.total(), exits.vbt.signals.total()

>>> entry_count, exit_count = splitter.apply(
...     get_signal_count,
...     vbt.Takeable(data),
...     20,
...     30,
...     merge_func="concat",
...     attach_bounds="index" ①
... )

>>> entry_count
split_year set start end
2018 train 2018-01-01 00:00:00+00:00 2018-07-01 00:00:00+00:00 2
test 2018-07-01 00:00:00+00:00 2019-01-01 00:00:00+00:00 4
2019 train 2019-01-01 00:00:00+00:00 2019-07-01 00:00:00+00:00 2
test 2019-07-01 00:00:00+00:00 2020-01-01 00:00:00+00:00 3
2020 train 2020-01-01 00:00:00+00:00 2020-07-01 00:00:00+00:00 1
test 2020-07-01 00:00:00+00:00 2021-01-01 00:00:00+00:00 2
```



```

2021      train  2021-01-01 00:00:00+00:00  2021-07-01 00:00:00+00:00  4
      test  2021-07-01 00:00:00+00:00  2022-01-01 00:00:00+00:00  1
dtype: int64

```

- 1 Calculate the integer bounds, map them to `data.index`, and attach them to the output's index

Finally, to demonstrate the power of merging functions, let's create our own merging function that plots the returned signals depending on the set!

```

>>> def plot_entries_and_exits(results, data, keys):
...     set_labels = keys.get_level_values("set") 1
...     fig = data.plot(plot_volume=False) 2
...     train_seen = False
...     test_seen = False
...
...     for i in range(len(results)): 3
...         entries, exits = results[i] 4
...         set_label = set_labels[i]
...         if set_label == "train":
...             entries.vbt.signals.plot_as_entries( 5
...                 data.close,
...                 trace_kwargs=dict( 6
...                     marker=dict(color="limegreen"),
...                     name=f"Entries ({set_label})",
...                     legendgroup=f"Entries ({set_label})", 7
...                     showlegend=not train_seen 8
...                 ),
...                 fig=fig
...             ),
...             exits.vbt.signals.plot_as_exits(
...                 data.close,
...                 trace_kwargs=dict(
...                     marker=dict(color="orange"),
...                     name=f"Exits ({set_label})",
...                     legendgroup=f"Exits ({set_label})",
...                     showlegend=not train_seen
...                 ),
...                 fig=fig
...             )
...             train_seen = True
...         else:
...             entries.vbt.signals.plot_as_entries(
...                 data.close,
...                 trace_kwargs=dict(
...                     marker=dict(color="skyblue"),
...                     name=f"Entries ({set_label})",
...                     legendgroup=f"Entries ({set_label})",
...                     showlegend=not test_seen
...                 ),
...                 fig=fig
...             ),
...             exits.vbt.signals.plot_as_exits(
...                 data.close,
...                 trace_kwargs=dict(
...                     marker=dict(color="magenta"),
...                     name=f"Exits ({set_label})",
...                     legendgroup=f"Exits ({set_label})",
...                     showlegend=not test_seen
...                 ),
...                 fig=fig
...             )
...             test_seen = True
...     return fig 9

>>> splitter.apply(
...     get_entries_and_exits,
...     vbt.Takeable(data),
...     20,
...     30,
...     merge_func=plot_entries_and_exits,
...     merge_kwargs=dict(data=data, keys=vbt.Rep("keys")),
... ).show()

```

- 1 `keys` is a multi-index for `results` that contains split labels, set labels, and (optionally) bounds
- 2 Plot OHLC
- 3 Whenever `merge_all` is `True`, the results are flattened into a single list. To figure out which results belong to which set, we can read `keys`.
- 4 Each result is a tuple of entries and exits, exactly how returned by `get_entries_and_exits`
- 5 Use `SignalsAccessor.plot_as_entries` and `SignalsAccessor.plot_as_exits` to plot the signals as entry and exit markers respectively
- 6 Change the appearance of the markers depending on the signal type and set
- 7 By making the legend group the same for multiple traces, we can make them react to any user interactions in the same way
- 8 Show the legend only if there isn't any trace with the same name already plotted, otherwise there will be duplicates
- 9 Our merging function returns a Plotly figure



As we can see, `Splitter.apply` is a flexible method that can execute any UDF on each range in the splitter. Not only it can return arrays in an analysis-friendly format, but it can also post-process and merge the outputs using another UDF, which makes it ideal for quick CV.

Decorators

But even the method above isn't the end of automation that vectorbt offers: similarly to the decorator `@parameterized`, which can enhance any function with a parameter processing logic, there is a decorator `@split` that can enhance just about any function with a split processing logic. The workings of this decorator are dead-simple: wrap a function, resolve a splitting specification into a splitter, and forward all arguments down to `Splitter.apply`. This way, we're making CV pipeline-centric and not splitter-centric.

There are several ways how to make a function splittable:

```
>>> @vbt.split(splitter=splitter) ❶
... def get_split_total_return(data):
...     return data.returns.vbt.returns.total()

>>> get_split_total_return(vbt.Takeable(data))
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test    2.132976
2021      train   0.194783
          test    0.379417
dtype: float64

>>> def get_total_return(data):
...     return data.returns.vbt.returns.total()

>>> get_split_total_return = vbt.split( ❷
...     get_total_return,
...     splitter=splitter
... )
>>> get_split_total_return(vbt.Takeable(data))
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test    2.132976
2021      train   0.194783
          test    0.379417
dtype: float64
```

- ❶ Through decoration
- ❷ Through passing the function as the first argument

If we didn't pass any argument to the decorator, or if we want to override some argument, we can add a prefix `_` to an argument to pass it to `@split` rather than to the function itself:

```
>>> @vbt.split
... def get_split_total_return(data):
...     return data.returns.vbt.returns.total()

>>> get_split_total_return(vbt.Takeable(data), _splitter=splitter)
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test    2.132976
2021      train   0.194783
          test    0.379417
dtype: float64
```

A potential inconvenience of the approach above is that for each data we pass, we need to separately construct a splitter using the index of the same data. To solve this, the decorator can also take an instruction on how to create a splitter from the passed data. The instruction consists of a method name (or the actual callable) provided as `splitter`, such as "from_rolling", and keyword arguments passed to this method as `splitter_kwargs`. Let's roll a window of 30 days of the year 2020:

```
>>> get_split_total_return(
...     vbt.Takeable(data.loc["2020":"2020"]),
...     _splitter="from_rolling",
...     _splitter_kwargs=dict(length="30d")
... )
split
0      0.321123
1     -0.088666
2     -0.250531
3      0.369418
4      0.093628
5     -0.059949
6      0.186424
7      0.020706
8     -0.069256
9      0.211424
10     0.372754
11     0.441005
dtype: float64
```

To avoid wrapping each object to take slices from with `Takeable`, we can also specify a list of such arguments as `takeable_args`:

```
>>> get_total_return_by_month = vbt.split(
...     get_total_return,
...     splitter="from_grouper",
...     splitter_kwargs=dict(by=vbt.RepEval("index.to_period('M')")),
...     takeable_args=["data"]
... )

>>> get_total_return_by_month(data.loc["2020":"2020"])
Open time
2020-01      0.298859
2020-02     -0.091746
2020-03     -0.248649
2020-04      0.297622
2020-05      0.070388
2020-06     -0.104131
2020-07      0.227844
2020-08     -0.012851
2020-09     -0.096073
2020-10      0.298694
2020-11      0.431230
2020-12      0.541364
Freq: M, dtype: float64
```

In the example above, a new splitter is built from every instance of data that we pass.

Furthermore, we can combine multiple decorators. For example, let's decorate the function `sma_crossover_perf` from the first page that we've already decorated with `@parameterized`, and make it split the entire period into 60% for the training set and 40% for the test set:

```
>>> cv_sma_crossover_perf = vbt.split(
...     sma_crossover_perf,
...     splitter="from_single", ①
...     splitter_kwargs=dict(split=0.6, set_labels=["train", "test"]),
...     takeable_args=["data"],
...     merge_func="concat", ②
... )
```

- ① Use `Splitter.from_single` to build a single split
- ② Any other argument passed to `@split` will be forwarded down to `Splitter.apply`

We'll run the full parameter grid on the training set only:

```
>>> train_perf = cv_sma_crossover_perf(
...     data.loc["2020":"2021"],
...     vbt.Param(np.arange(5, 50), condition="x < slow_window"),
...     vbt.Param(np.arange(5, 50)),
...     p_execute_kwargs=dict(
...         clear_cache=50,
...         collect_garbage=50
...     ),
...     _forward_kwargs_as={
...         "p_execute_kwargs": "_execute_kwargs" ①
...     },
...     _apply_kwargs=dict(set_="train") ②
... )
```

- ① Both `@split` and `@parameterized` take `execute_kwargs`. To differentiate between them, we'll pass `p_execute_kwargs`, let `@split` rename it to `_execute_kwargs` using `forward_kwargs_as`, and forward the argument down to `@parameterized`
- ② If we need to change any argument related to `Splitter.apply`, we can pass it directly to `@split`, or include it in `_apply_kwargs` when passed to an already decorated function

```
>>> train_perf
fast_window  slow_window
5            6            1.160003
            7            1.122994
            8            2.054193
            9            1.880043
           10            1.632951
            ...
46           48            1.362618
           49            1.415010
47           48            1.317795
           49            1.250835
48           49            0.160916
Length: 990, dtype: float64
```

We can then validate the optimization performance on the test set:

```
>>> test_perf = cv_sma_crossover_perf(
...     data.loc["2020":"2021"],
...     train_perf.idxmax()[0],
...     train_perf.idxmax()[1],
...     _apply_kwargs=dict(set_="test")
... )
```

Split 1/1

```
>>> test_perf
1.2596407796960982
```

Hint

If you want to have a proper Series instead of a single value returned, disable `squeeze_one_split` and `squeeze_one_set` in `Splitter.apply` using `_apply_kwargs`.

But even this decorator isn't the final form of automation: there's a special decorator `@cv_split` that combines `@split` and `@parameterized` to run the full parameter grid on the first set and the best parameter combination on the remaining sets. How is the best parameter combination defined? There's an argument `selection` that can be a template taking the previous results (available as `grid_results` in the context) and returning the integer position of the parameter combination that the user defines as the best. Moreover, the decorator can either return the best results only (`return_grid=False`), or additionally include the grid results from the training set (`return_grid=True`) or even the grid run on all sets (`return_grid="all"`).

Since we're waiting too much time for the entire grid of parameter combinations to complete, let's rewrite our pipeline with Numba to return the Sharpe ratio of a single parameter combination:

```
>>> @njit(nogil=True) ①
... def sma_crossover_perf_nb(close, fast_window, slow_window, ann_factor):
...     fast_sma = vbt.nb.ma_nb(close, fast_window) ②
...     slow_sma = vbt.nb.ma_nb(close, slow_window)
...     entries = vbt.nb.crossed_above_nb(fast_sma, slow_sma) ③
...     exits = vbt.nb.crossed_above_nb(slow_sma, fast_sma)
...     sim_out = vbt.pf_nb.from_signals_nb( ④
...         target_shape=close.shape,
...         group_lens=np.full(close.shape[1], 1),
...         close=close,
...         long_entries=entries,
...         short_entries=exits,
...         save_returns=True
...     )
...     return vbt.ret_nb.sharpe_ratio_nb( ⑤
...         sim_out.in_outputs.returns,
...         ann_factor
...     )
```

① Disable GIL for multithreading

② Use `ma_nb`

③ Use `crossed_above_nb`

④ Use `from_signals_nb`

⑤ Use `sharpe_ratio_nb`

Test the function on the full history:

```
>>> sma_crossover_perf_nb(vbt.to_2d_array(data.close), 20, 30, 365)
array([1.04969317])
```

Note

All Numba functions that we use are expecting a two-dimensional NumPy array as input.

Finally, let's define and run CV in a parallel manner:

```
>>> cv_sma_crossover_perf = vbt.cv_split(
...     sma_crossover_perf_nb,
...     splitter="from_rolling",
...     splitter_kwargs=dict(
```

```

...     length=360,
...     split=0.5,
...     set_labels=["train", "test"]
... ),
...     takeable_args=["close"],
...     merge_func="concat", ❶
...     parameterized_kwargs=dict( ❷
...         engine="dask",
...         chunk_len="auto",
...     )
... )

>>> grid_perf, best_perf = cv_sma_crossover_perf(
...     vbt.to_2d_array(data.close),
...     vbt.Param(np.arange(5, 50), condition="x < slow_window"),
...     vbt.Param(np.arange(5, 50)),
...     pd.Timedelta(days=365) // data.index.freq, ❸
...     _merge_kwargs=dict(wrapper=data.symbol_wrapper), ❹
...     _index=data.index, ❺
...     _return_grid="all" ❻
... )

```

- ❶ Keyword arguments passed to `@parameterized`
- ❷ Chunk length will be based on the number of CPUs
- ❸ The merging function passed to `@split`, will be automatically passed to `@parameterized` as well (and yes, both decorators can have different merging functions)
- ❹ Get the annualization factor by dividing the year frequency by the data frequency
- ❺ Our Numba function returns a one-dimensional NumPy array (i.e., one value per column in the data), thus we need to pass the wrapper to assign each value in the returned array with the corresponding column name
- ❻ `index` is an argument of `@split` and can be used to pass an index if no index can be extracted from the "takeable" arrays
- ❼ Run the entire grid on each set. Will return both the grid results and the best results. Note that the test set will still use the previous grid, not its own.

Info

By default, the highest value is selected. To select the lowest value, set `selection="min"` in `@split`. Also, make sure to change the selection template if any other merging function is being used.

Split 9/9

That was fast! 🐢

Question

Why is the performance so different compared to the previous version, which, by the way, uses the same Numba functions under the hood? Remember that when running one function a thousand of times, even a 1-millisecond longer execution time translates into a 1-second longer **total** execution time.

Let's take a look at the CV results:

```

>>> grid_perf
split  set  fast_window  slow_window  symbol
0      train  5          6          BTCUSDT  1.771782
          7          BTCUSDT  2.206458
          8          BTCUSDT  2.705892
          9          BTCUSDT  1.430768
         10         BTCUSDT  0.851692
...
8      test  46         48          BTCUSDT  0.637332
          49         BTCUSDT -0.424650
          47         48          BTCUSDT -0.214946
          49         BTCUSDT  0.231712
          48         49          BTCUSDT -0.351245
Length: 17820, dtype: float64

>>> best_perf
split  set  fast_window  slow_window  symbol
0      train  19         34          BTCUSDT  4.392966
      test  19         34          BTCUSDT  0.535497
1      train  6          7          BTCUSDT  2.545991
      test  6          7          BTCUSDT -1.101692
2      train  18         20          BTCUSDT  4.363491
      test  18         20          BTCUSDT  1.692070
3      train  14         18          BTCUSDT  3.615833
      test  14         18          BTCUSDT  0.035444
4      train  18         21          BTCUSDT  3.236440
      test  18         21          BTCUSDT  1.882290
5      train  20         27          BTCUSDT  3.371474
      test  20         27          BTCUSDT  2.574914
6      train  11         18          BTCUSDT  4.657549
      test  11         18          BTCUSDT -2.067505
7      train  29         30          BTCUSDT  3.388797
      test  29         30          BTCUSDT  0.968127
8      train  25         28          BTCUSDT  2.149624
      test  25         28          BTCUSDT  1.256857
dtype: float64

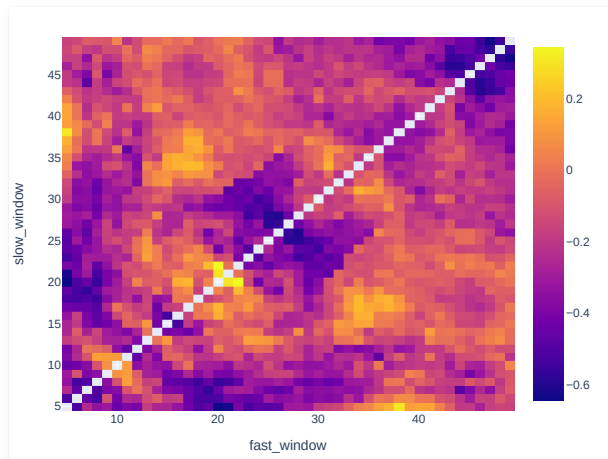
```

For instance, the test results negatively correlate with the training results, meaning that the parameter combinations with the highest Sharpe tend to underperform when they overperformed previously. This actually makes sense because BTC market regimes tend to switch frequently.

```
>>> best_train_perf = best_perf.xs("train", level="set")
>>> best_test_perf = best_perf.xs("test", level="set")
>>> best_train_perf.corr(best_test_perf)
-0.21641517083891232
```

To dig deeper, let's analyze the cross-set correlation of each parameter combination, that is, how the performance of a parameter combination in the training set correlates with the performance of the same parameter combination in the test set:

```
>>> param_cross_set_corr = grid_perf\
...     .unstack("set")\
...     .groupby(["fast_window", "slow_window"])\
...     .apply(lambda x: x["train"].corr(x["test"]))
>>> param_cross_set_corr.vbt.heatmap(symmetric=True).show()
```



Another perspective can be added by analyzing the test performance of the best parameter combinations in relation to the test performance of all parameter combinations. Let's get the relative number of parameter combinations that are better than the selected one in each split:

```
>>> grid_test_perf = grid_perf.xs("test", level="set")
>>> grid_df = grid_test_perf.rename("grid").reset_index() ❶
>>> del grid_df["fast_window"] ❷
>>> del grid_df["slow_window"]
>>> best_df = best_test_perf.rename("best").reset_index()
>>> del best_df["fast_window"]
>>> del best_df["slow_window"]
>>> merged_df = pd.merge(grid_df, best_df, on=["split", "symbol"]) ❸
>>> grid_better_mask = merged_df["grid"] > merged_df["best"] ❹
>>> grid_better_mask.index = grid_test_perf.index
>>> grid_better_cnt = grid_better_mask.groupby(["split", "symbol"]).mean() ❺
>>> grid_better_cnt
split  symbol
0      BTCUSD  0.242424
1      BTCUSD  0.988889
2      BTCUSD  0.214141
3      BTCUSD  0.404040
4      BTCUSD  0.359596
5      BTCUSD  0.963636
6      BTCUSD  0.908081
7      BTCUSD  0.342424
8      BTCUSD  0.250505
dtype: float64
```

- ❶ Make columns out of the index
- ❷ Remove the columns that we don't want to merge by
- ❸ Merge both DataFrames by split and symbol
- ❹ Get the mask of the grid results being better than the best results
- ❺ Use `sum()` to get the absolute count and `mean()` to get the relative count

The selected parameter combinations seem to beat the most other parameter combinations tested during the same time period, but some results are particularly disappointing: in the splits 1, 5, and 6, the selected parameter combination performed worse than other 90%.

Finally, let's compare the results against our buy-and-hold baseline. For this, we need to extract the price that belongs to each split, but how do we do that without a splitter? Believe it or not, `@split` has an argument to return the splitter without running the pipeline 😊

```
>>> cv_splitter = cv_sma_crossover_perf(
...     _index=data.index, ❶
...     _return_splitter=True
... )
>>> stacked_close = cv_splitter.take(
...     data.close,
```

```

...     into="reset_stacked",
...     set_="test" ❷
... )
>>> hold_pf = vbt.Portfolio.from_holding(stacked_close, freq="daily")
>>> hold_perf = hold_pf.sharpe_ratio
>>> hold_perf
split
0    -0.430642
1    -1.741407
2     3.408079
3    -0.556471
4     0.954291
5     3.241618
6     0.686198
7    -0.038013
8    -0.917722
Name: sharpe_ratio, dtype: float64

```

- ❶ We either need to pass the index or all arguments for parsing
- ❷ Select the test set only

As we can see, the "taking" and "applying" approaches can be safely combined since the underlying splitter is guaranteed to be built in the same way and thus produce the same results (unless the splitter method has some random component and hasn't been provided with a seed).

Modeling

The class [Splitter](#) can also be helpful in cross-validating ML models. In particular, you can casually step upon a class [SplitterCV](#) that acts as a regular cross-validator from scikit-learn by subclassing `BaseCrossValidator`. We'll demonstrate its usage on a simple classification problem of predicting the best entry and exit timings.

Before we start, we need to decide on features and labels that should act as predictor and response variables respectively. Features are usually multi-columnar time-series DataFrames where each row contains multiple data points (one per column) that should predict the same row in labels. Labels are usually a single-columnar time-series Series that should be predicted. Ask yourself the following questions to easily come up with a decision:

1. "How can the future performance be represented, preferably as a single number? Should it be the price at the next bar, the average price change over the next week, a vector of weights for rebalancing, a boolean containing a signal, or something else?"
2. "What kind of data that encompasses the past performance is likely to predict the future performance? Should it be indicators, news sentiment index, past backtesting results, or something else?"
3. "Which ML model can handle such a task?" (remember that most models are limited to just a couple of specific feature and label formats!)

For the sake of an example, we'll fit a [random forest classifier](#) on all [TA-Lib](#) indicators stacked along columns to predict the binary labels generated by the label generator [TRENDLB](#), where 1 means an uptrend and 0 means a downtrend. Sounds like fun 😊

First, run all the TA-Lib indicators on the entire data to get the feature set `X`:

```

>>> X = data.run("talib")
>>> X.shape
(1902, 174)

```

We've got 1902 rows (dates) and 174 columns (features).

Next, generate the labels `y` (we'll use the same configuration as previously):

```

>>> trendlb = data.run("trendlb", 1.0, 0.5, mode="binary")
>>> y = trendlb.labels
>>> y.shape
(1902,)

```

Both the features and the labels contain NaNs, which we need to carefully take care of. If we remove the rows with at least one NaN, we'll remove all the data. Instead, we'll first remove the columns that consist entirely of NaNs or a single unique value. Also, because `X` and `y` should have the same length, we need to do the row-filtering operation on both datasets simultaneously:

```

>>> X = X.replace([-np.inf, np.inf], np.nan)
>>> invalid_column_mask = X.isnull().all(axis=0) | (X.nunique() == 1)
>>> X = X.loc[:, ~invalid_column_mask]
>>> invalid_row_mask = X.isnull().any(axis=1) | y.isnull()
>>> X = X.loc[~invalid_row_mask]
>>> y = y.loc[~invalid_row_mask]
>>> X.shape, y.shape
((1773, 144), (1773,))

```

Warning

If you worked with ML before, you'll quickly feel the danger coming from the logical operation in the first cell: we're checking for a condition across the entire column, thus potentially catching the [look-ahead bias](#). Even though our operation isn't too dangerous because we remove only the columns that are likely to stay irrelevant in the future, other transformations such as data normalization should always be included in a [Pipeline](#) that's executed per split rather than once and globally.

We've successfully removed a total of 129 rows and 30 columns.

Next, we'll establish our classifier that will learn `X` to predict `y`:

```
>>> from sklearn.ensemble import RandomForestClassifier ❶
>>> clf = RandomForestClassifier(random_state=42)
```

❶ Use `RandomForestClassifier`

❷ Question

Why haven't we rescaled, normalized, or reduced the dimensionality of the features? Random forests are very robust modeling techniques and can handle high noise levels as well as a high number of features.

To cross-validate the classifier, let's create an `SplitterCV` instance that splits the entire period into expanding windows with non-overlapping test periods of 180 bars:

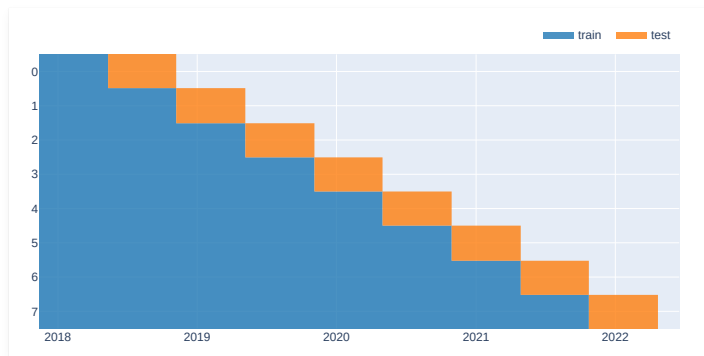
```
>>> cv = vbt.SplitterCV(
...     "from_expanding", ❶
...     min_length=360, ❷
...     offset=180,
...     split=-180,
...     set_labels=["train", "test"]
... )

>>> cv_splitter = cv.get_splitter(X) ❸
>>> cv_splitter.plot().show()
```

❶ Method of `Splitter`

❷ Other keyword arguments are forwarded down to this method

❸ Get the actual splitter for plotting



Finally, run the classifier on each training period and check the accuracy of its predictions on the respective test period. Even though the accuracy score is the most basic of all classification scores and has its own flaws, we'll keep things simplified for now:

```
>>> from sklearn.model_selection import cross_val_score ❶

>>> cross_val_score(clf, X, y, cv=cv, scoring="accuracy")
array([1.         , 0.20555556, 0.88333333, 0.72777778, 1.         ,
       0.92777778, 0.53333333, 0.30555556])
```

❶ Use `cross_val_score`

We can see that there are only two underperforming splits, and even two splits with 100% accuracy, how is this possible? Let's find out! What we need are raw predictions: we'll use the actual splitter to take the slices from `X` and `y`, generate the predictions on each test set using our classifier, and concatenate all the predictions into a single Series.

```
>>> X_slices = cv_splitter.take(X)
>>> y_slices = cv_splitter.take(y)

>>> test_labels = []
>>> test_preds = []
>>> for split in X_slices.index.unique(level="split"): ❶
...     X_train_slice = X_slices[(split, "train")] ❷
...     y_train_slice = y_slices[(split, "train")]
...     X_test_slice = X_slices[(split, "test")]
...     y_test_slice = y_slices[(split, "test")]
...     slice_clf = clf.fit(X_train_slice, y_train_slice) ❸
...     test_pred = slice_clf.predict(X_test_slice) ❹
...     test_pred = pd.Series(test_pred, index=y_test_slice.index)
...     test_labels.append(y_test_slice)
...     test_preds.append(test_pred)

>>> test_labels = pd.concat(test_labels).rename("labels") ❺
>>> test_preds = pd.concat(test_preds).rename("preds")
```

❶ Iterate over the split labels

❷ Get the arrays under the current split

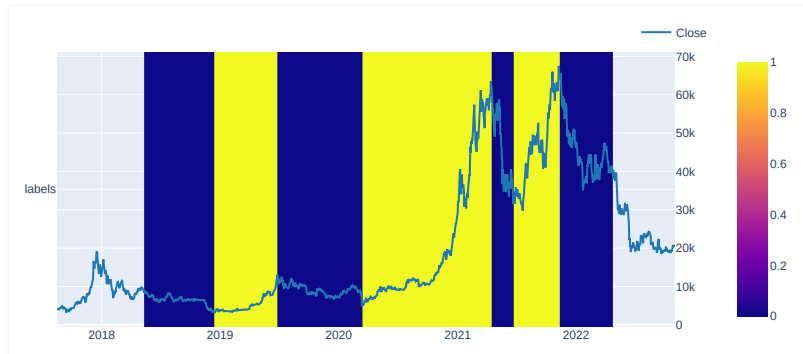
❸ Fit the classifier to the training data

- 4 Generate the predictions and wrap into Series
- 5 Concatenate all the prediction Series along rows

Let's compare the actual labels (left tab) to the predictions (right tab):

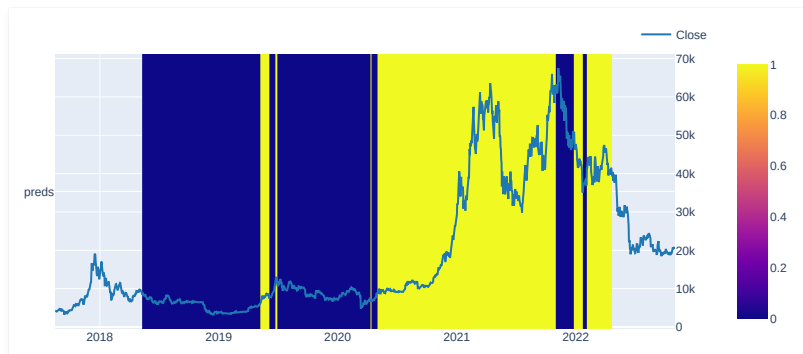
Labels

```
>>> data.close.vbt.overlay_with_heatmap(test_labels).show()
```

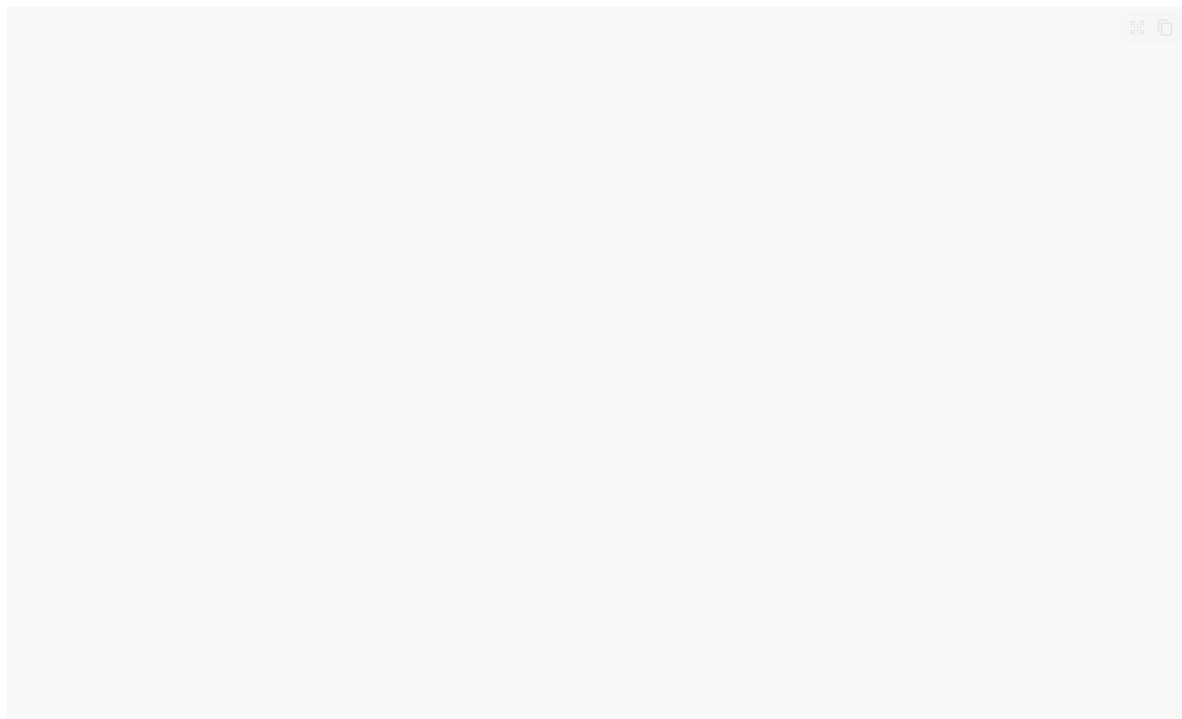


Predictions

```
>>> data.close.vbt.overlay_with_heatmap(test_preds).show()
```



The model seems to correctly classify many bigger uptrends and even issue an exit signal at the latest peak on time! Nevertheless, we shouldn't just rely on our visual intuition: let's backtest the predictions.



```

>>> pf = vbt.Portfolio.from_signals(
...     data.close[test_preds.index],
...     test_preds == 1,
...     test_preds == 0,
...     direction="both" ❶
... )
>>> pf.stats()
Start                2018-05-12 00:00:00+00:00
End                  2022-04-20 00:00:00+00:00
Period              1440 days 00:00:00
Start Value          100.0
Min Value            55.079685
Max Value            1238.365833
End Value            719.483655
Total Return [%]     619.483655
Benchmark Return [%] 388.524488
Total Time Exposure [%] 100.0
Max Gross Exposure [%] 532.441444
Max Drawdown [%]     71.915142
Max Drawdown Duration 244 days 00:00:00
Total Orders         20
Total Fees Paid       0.0
Total Trades         20
Win Rate [%]         52.631579
Best Trade [%]        511.777894
Worst Trade [%]       -27.856728
Avg Winning Trade [%] 66.38609
Avg Losing Trade [%]  -11.784147
Avg Winning Trade Duration 104 days 16:48:00
Avg Losing Trade Duration  35 days 05:20:00
Profit Factor         1.999834
Expectancy           32.802227
Sharpe Ratio          0.983656
Calmar Ratio          0.902507
Omega Ratio           1.202685
Sortino Ratio         1.627438
dtype: object

```

❶ It's important to have any false-positive exit signals be as painful as false-positive entry signals

We've got some pretty solid statistics 🌟

If you're willing to accept a challenge: build a pipeline to impute and (standard-)normalize the data, **reduce the dimensionality** of the features, as well as fit one of the **linear models** to predict the average price change over the next `n` bars (i.e., regression task!). Based on each prediction, you can then decide whether a position is worth opening or closing out.

Summary

Backtesting requires an overhaul of traditional cross-validation schemes centered around ML, and vectorbt offers the needed toolset. The heart of the new functionality is the juggernaut class **Splitter**; it not only provides time-series-safe splitting schemes but also enables us to thoroughly analyze the generated splits and apply them to data of arbitrary complexity efficiently. For instance, we can enjoy the offered flexibility to either split data into slices and perform CV on them manually or construct a pipeline and let the splitter do the slicing and execution parts for us. There's even a decorator for parameterizing and cross-validating any Python function - `@cv_split`. And for any ML-related tasks, the class **SplitterCV** offers a splitter-enhanced interface well understood by scikit-learn and many other packages, such as by the scikit-learn compatible neural network library **skorch** wrapping PyTorch. As a result, validation of rule-based and ML-based models has become as easy as ever 🦋

 [Python code](#)

 [Notebook](#)

✓ Cross-validation

Splitting

? Question

Learn more in [Cross-validation tutorial](#).

To pick a fixed number of windows and optimize the window length such that they collectively cover the maximum amount of the index while keeping the train or test set non-overlapping, use `Splitter.from_n_rolling` with `length="optimize"`. Under the hood, it minimizes any empty space using SciPy.

Pick longest 20 windows for WFA such that test ranges don't overlap

```
splitter = vbt.Splitter.from_n_rolling(
    data.index,
    n=20,
    length="optimize",
    split=0.7, ❶
    optimize_anchor_set=1, ❷
    set_labels=["train", "test"]
)
```

- ❶ 70% for training, 30% for testing
- ❷ Make the test set non-overlapping. Change to 0 for the train set.

———— + ————

When using `Splitter.from_rolling` and the last window doesn't fit, it will be removed, leaving a gap on the right-hand side. To remove the oldest window instead, use `backwards="sorted"`.

Roll a window that fills more recent data and with no gaps between test sets

```
length = 1000
ratio = 0.95
train_length = round(length * ratio)
test_length = length - train_length

splitter = vbt.Splitter.from_rolling(
    data.index,
    length=length,
    split=train_length,
    offset_anchor_set=None,
    offset=-test_length,
    backwards="sorted"
)
```

———— + ————

To create a gap between the train set and the test set, use `RelRange` with `is_gap=True`.

Roll an expanding window with a variable train set, a gap of 10 rows, and a test set of 20 rows

```
splitter = vbt.Splitter.from_expanding(
    data.index,
    min_length=130,
    offset=10, ❶
    split=(1.0, vbt.RelRange(length=10, is_gap=True), 20),
    split_range_kwargs=dict(backwards=True) ❷
)
```

- ❶ Shift each split by the same number of rows as in the gap
- ❷ Split each range by first calculating the test set and only then the train set, otherwise `1.0` (100%) will be calculated first and will take the entire split

———— + ————

To roll a time-periodic window, use `Splitter.from_ranges` with `every` and `lookback_period` arguments as date offsets.

Reserve 3 years for training and 1 year for testing

```
splitter = vbt.Splitter.from_ranges(
    data.index,
    every="Y",
    lookback_period="4Y",
    split=(
        vbt.RepEval("index.year != index.year[-1]"), ❶
        vbt.RepEval("index.year == index.year[-1]") ❷
    )
)
```

- ❶ Training set should include all years in the split apart from the last one

- ② Test set should include the last year in the split only

Taking

To split an object along the index (time) axis, we need to create a [Splitter](#) instance and then "take" chunks from that object.

How to split an object in two lines

```
splitter = vbt.Splitter.from_n_rolling(data.index, n=10)
data_chunks = splitter.take(data) ①

# -----

splitter = vbt.Splitter.from_ranges(df.index, every="W")
new_df = splitter.take(df, into="reset_stacked") ②
```

- ① VBT object
② Regular array

———— + ————

Also, most VBT objects have a `split` method that can combine these both operations into one. The method will determine the correct splitting operation automatically based on the supplied arguments.

How to split an object in one line

```
data_chunks = data.split(n=10) ①

# -----

new_df = df.vbt.split(every="W") ②
```

- ① VBT object. Method `from_n_rolling` is guessed based on `n=10`.
② Regular array. Method `from_ranges` is guessed based on `every="W"`. Option `into="reset_stacked"` is enabled automatically.

Testing

To cross-validate a function that takes only one parameter combination at a time on a grid of parameter combinations, use [@vbt.cv_split](#). It's a combination of [@vbt.parameterized](#) (which takes a grid of parameter combinations and runs a function on each combination while merging the results) and [@vbt.split](#) (which runs a function on each split and set combination).

Cross-validate a function to maximize the Sharpe ratio

```
def selection(grid_results): ①
    return vbt.LabelSel([grid_results.idxmax()]) ②

@vbt.cv_split(
    splitter="from_n_rolling", ③
    splitter_kwargs=dict(n=10, split=0.5, set_labels=["train", "test"]), ④
    takeable_args=["data"], ⑤
    execute_kwargs=dict(), ⑥
    parameterized_kwargs=dict(merge_func="concat"), ⑦
    merge_func="concat", ⑧
    selection=vbt.RepFunc(selection), ⑨
    return_grid=False ⑩
)
def my_pipeline(data, param1_value, param2_value): ⑪
    ...
    return pf.sharpe_ratio

cv_sharpe_ratios = my_pipeline( ⑫
    data,
    vbt.Param(param1_values),
    vbt.Param(param2_values)
)

# -----

@vbt.cv_split(..., takeable_args=None) ⑬
def my_pipeline(range_, data, param1_value, param2_value):
    data_range = data.iloc[range_]
    ...
    return pf.sharpe_ratio

cv_sharpe_ratios = my_pipeline(
    vbt.Rep("range_"),
    data,
    vbt.Param([1, 2, 3]),
    vbt.Param([1, 2, 3]),
    _index=data.index ⑭
)
```

- ① Function that returns the index of the best parameter combination
② Find the parameter combination of the highest Sharpe ratio. Wrap with `LabelSel` to tell vectorbtpro that the returned value is a label and not a position in case it's an integer. Also, wrap the value with a list to show the parameter combination in the final index.

- ③ Name of the splitting method, such as `Splitter.from_n_rolling`
- ④ Keyword arguments passed to the splitting method
- ⑤ Name of the arrays that should be split. You can also pass `vbt.Takeable(data)` directly to the function instead.
- ⑥ Keyword arguments passed to `execute` to control the execution of split and set combinations
- ⑦ Keyword arguments passed to the `@vbt.parameterized`. Here we want to concatenate the results of all the parameter combinations into a single Pandas Series.
- ⑧ Function (name) to merge all the split and set combinations. Here we want to concatenate all the Pandas Series into a single Pandas Series.
- ⑨ Either a specific index or a template to pick the best parameter combination
- ⑩ Whether to return the entire grid of parameter combinations and not only the best ones
- ⑪ Similarly to `@vbt.parameterized`, VBT will pass only one parameter combination at a time as single values. Any takeable argument (here `data`) will contain only values that correspond to the current split and set combination.
- ⑫ The returned Pandas Series will contain the cross-validation results by split and set combination
- ⑬ Same as above but select the date range manually in the function
- ⑭ Any argument that is meant to be passed to the decorator can be also passed directly to the function by prepending the underscore

———— + ————

To skip a parameter combination, return `NoResult`. This may be helpful to exclude a parameter combination that raises an error. `NoResult` can be also returned by the selection function to skip the entire split and set combination. Once excluded, the combination won't be visible in the final index.

Skip split and set combinations where there are no satisfactory parameters

```

1
def selection(grid_results):
    sharpe_ratio = grid_results.xs("Sharpe Ratio", level=-1).astype(float)
    return vbt.LabelSel([sharpe_ratio.idxmax()])

@vbt.cv_split(...)
def my_pipeline(...):
    ...
    stats_sr = pf.stats(agg_func=None)
    if stats_sr["Min Value"] > 0 and stats_sr["Total Trades"] >= 20: ②
        return stats_sr
    return vbt.NoResult

# -----
3
def selection(grid_results):
    sharpe_ratio = grid_results.xs("Sharpe Ratio", level=-1).astype(float)
    min_value = grid_results.xs("Min Value", level=-1).astype(float)
    total_trades = grid_results.xs("Total Trades", level=-1).astype(int)
    sharpe_ratio = sharpe_ratio[(min_value > 0) & (total_trades >= 20)]
    if len(sharpe_ratio) == 0:
        return vbt.NoResult
    return vbt.LabelSel([sharpe_ratio.idxmax()])

@vbt.cv_split(...)
def my_pipeline(...):
    ...
    return pf.stats(agg_func=None)

```

- ① Filter parameter combinations on the fly and then select the best ones from those that left (if any)
- ② Keep the parameter combination only if the minimum portfolio value is greater than 0 (i.e., the position hasn't been liquidated) and the number of trades is 20 or greater
- ③ Same as above but return all parameter combinations and then filter them in the selection function

———— + ————

To warm up one or more indicators, instruct VBT to pass a date range instead of selecting it from data, and prepend a buffer to this date range. Then, manually select this extended date range from the data and run your indicators on the selected date range. Finally, remove the buffer from the indicator(s).

Warm up a SMA crossover

```

@vbt.cv_split(..., index_from="data")
def buffered_sma_pipeline(data, range_, fast_period, slow_period, ...):
    buffer_len = max(fast_period, slow_period) ①
    buffered_range = slice(range_.start - buffer_len, range_.stop) ②
    data_buffered = data.iloc[buffered_range] ③

    fast_sma_buffered = data_buffered.run("sma", fast_period, hide_params=True)
    slow_sma_buffered = data_buffered.run("sma", slow_period, hide_params=True)
    entries_buffered = fast_sma_buffered.real_crossed_above(slow_sma_buffered)
    exits_buffered = fast_sma_buffered.real_crossed_below(slow_sma_buffered)

    data = data_buffered.iloc[buffer_len:] ④
    entries = entries_buffered.iloc[buffer_len:]
    exits = exits_buffered.iloc[buffer_len:]
    ...

```

```
buffered_sma_pipeline(  
    data, ⑤  
    vbt.Rep("range_"), ⑥  
    vbt.Param(fast_periods, condition="x < slow_period"),  
    vbt.Param(slow_periods),  
    ...  
)
```

- ① Determine the length of the buffer
- ② Extend the date range with the buffer
- ③ Select the data that corresponds to the extended date range
- ④ After running all the indicators, remove the buffer
- ⑤ Pass data as it is (without selection)
- ⑥ Instruct VBT to pass the date range as a slice

Tutorials Cross-validation

✓ Cross-validation

Once we develop a rule-based or ML-based strategy, it's time to backtest it. The first time around we obtain a low Sharpe ratio we're unhappy with, we decide to tweak our strategy. Eventually, after multiple iterations of tweaking parameters, we end up with a "flawless" combination of parameters and a strategy with an exceptional Sharpe ratio. However, in live trading the performance took a different turn: we essentially tanked and lost money. What went wrong?

Markets inherently have noise - small and frequent idiosyncrasies in the price data. When modelling a strategy, we want to avoid optimizing for one specific period because there is a chance the model adapts so closely to historical data that it becomes ineffective in predicting the future. It'd be like tuning a car specifically for one racetrack, while expecting it to perform well everywhere. Especially with vectorbt, which enables us to search extensive databases of historical market data for patterns, it is often possible to develop elaborate rules that appear to predict price development with close accuracy (see [p-hacking](#)) but make random guesses when applied to data outside the sample the model was constructed from.

Overfitting (aka [curve fitting](#)) usually occurs for one or more of the following reasons: mistaking noise for signal, and overly tweaking too many parameters. To curb overfitting, we should use [cross-validation](#) (CV), which involves partitioning a sample of data into complementary subsets, performing the analysis on one subset of data called the training or *in-sample* (IS) set, and validating the analysis on the other subset of data called the validation or *out-of-sample* (OOS) set. This procedure is repeated until we have multiple OOS periods and can draw statistics from these results combined. The ultimate questions we need to ask ourselves: is our choice of parameters robust in the IS periods? Is our performance robust on the OOS periods? Because if not, we're shooting in the dark, and as a quant investor we should not leave room for second-guessing when real money is at stake.

Consider a simple strategy around a moving average crossover.

First, we'll pull some data:

```
>>> from vectorbtpro import *

>>> data = vbt.BinanceData.pull("BTCUSD", end="2022-11-01 UTC")
>>> data.index
DatetimeIndex(['2017-08-17 00:00:00+00:00', '2017-08-18 00:00:00+00:00',
              '2017-08-19 00:00:00+00:00', '2017-08-20 00:00:00+00:00',
              ...,
              '2022-10-28 00:00:00+00:00', '2022-10-29 00:00:00+00:00',
              '2022-10-30 00:00:00+00:00', '2022-10-31 00:00:00+00:00'],
              dtype='datetime64[ns, UTC]', name='Open time', length=1902, freq='D')
```

Let's construct a parameterized mini-pipeline that takes data and the parameters, and returns the Sharpe ratio that should reflect the performance of our strategy on that test period:

```
>>> @vbt.parameterized(merge_func="concat") ①
... def sma_crossover_perf(data, fast_window, slow_window):
...     fast_sma = data.run("sma", fast_window, short_name="fast_sma") ②
...     slow_sma = data.run("sma", slow_window, short_name="slow_sma")
...     entries = fast_sma.real_crossed_above(slow_sma)
...     exits = fast_sma.real_crossed_below(slow_sma)
...     pf = vbt.Portfolio.from_signals(
...         data, entries, exits, direction="both") ③
...     return pf.sharpe_ratio ④
```

- ① The decorator `@parameterized` enhances `sma_crossover_perf` to take arguments wrapped with [Param](#), build the grid of parameter combinations, run `sma_crossover_perf` on each parameter combination, and merge the results using concatenation
- ② Use `Data.run` to run an indicator on a data instance
- ③ Enable shorting by marking exit signals as short entry signals
- ④ Our function returns a single value, which will become a Series once all parameter combinations are processed

Let's test a grid of `fast_window` and `slow_window` combinations on one year of that data:

```
>>> perf = sma_crossover_perf( ①
...     data["2020": "2020"], ②
...     vbt.Param(np.arange(5, 50), condition="x < slow_window"), ③
...     vbt.Param(np.arange(5, 50)), ④
...     _execute_kwargs=dict( ⑤
...         clear_cache=50, ⑥
...         collect_garbage=50
...     )
... )
>>> perf
fast_window  slow_window
5            6            0.625318
            7            0.333243
            8            1.171861
            9            1.062940
           10            0.635302
...
46           48            0.534582
           49            0.573196
47           48            0.445239
           49            0.357548
48           49           -0.826995
Length: 990, dtype: float64
```

- 1 Even while being decorated with `parameterized`, the function has the same arguments as `sma_crossover_perf`
- 2 `Data` instances can be sliced like regular Pandas objects. Remember that the right bound is inclusive.
- 3 There is no sense in having a window for a fast SMA longer than a window for a slow SMA, thus let's define a condition that cuts the number of parameter combinations in half
- 4 We don't need to define any condition for the second parameter since it's already captured by the first parameter
- 5 Any arguments that we want to pass to the `@parameterized` decorator instead must have the prefix `_`
- 6 By default, the `parameterized` decorator is calling the `SerialEngine` to execute each parameter combination. This engine has arguments to clear the cache and collect any memory garbage. Here, we're doing it once every 50 iterations to keep memory in check.

Combination 990/990

It took 30 seconds to test 990 parameter combinations, or 30 milliseconds per run. Below we're sorting the Sharpe ratios in descending order to unveil the best parameter combinations:

```
>>> perf.sort_values(ascending=False)
fast_window  slow_window
15           20           3.669815
14           19           3.484855
15           18           3.480444
14           21           3.467951
13           19           3.457093
...
36           41           0.116606
37           37           0.075805
42           43           0.004402
10           12          -0.465247
48           49          -0.826995
Length: 990, dtype: float64
```

Looks like `fast_window=15` and `slow_window=20` can make us millionaires! But before we bet our entire life savings on that configuration, let's test it on the next year:

```
>>> best_fast_window, best_slow_window = perf.idxmax() 1
>>> sma_crossover_perf(
...     data["2021":"2021"],
...     best_fast_window, 2
...     best_slow_window
... )
-1.1940481501019478
```

- 1 Get the parameter values with the highest Sharpe ratio
- 2 No need to wrap the parameters with `Param` if they are single values or if we don't want to build the grid out of them. They are simply forwarded down to `sma_crossover_perf`.

The result is discouraging, but maybe we still performed well compared to a baseline? Let's compute the Sharpe ratio of the buy-and-hold strategy applied to that year:

```
>>> data["2021":"2021"].run("from_holding").sharpe_ratio 1
0.9641311236043749
```

- 1 `Data.run` not only accepts indicator names but also portfolio method names. First run will take a while because it has to be compiled first.

Seems like our strategy failed miserably 📉

But this was just one optimization test, what if this period was an outlier and our strategy does perform well *on average*? Let's try answering this question by conducting the test above on each consecutive 180 days in the data:

```
>>> start_index = data.index[0] 1
>>> period = pd.Timedelta(days=180) 2
>>> all_is_bounds = {} 3
>>> all_is_bl_perf = {}
>>> all_is_perf = {}
>>> all_oos_bounds = {} 4
>>> all_oos_bl_perf = {}
>>> all_oos_perf = {}
>>> split_idx = 0
>>> period_idx = 0

>>> with vbt.ProgressBar() as pbar: 5
...     while start_index + 2 * period <= data.index[-1]: 6
...         pbar.set_prefix(str(start_index))
...
...         is_start_index = start_index
...         is_end_index = start_index + period - pd.Timedelta(nanoseconds=1) 7
...         is_data = data[is_start_index : is_end_index]
...         is_bl_perf = is_data.run("from_holding").sharpe_ratio
...         is_perf = sma_crossover_perf(
...             is_data,
...             vbt.Param(np.arange(5, 50), condition="x < slow_window"),
...             vbt.Param(np.arange(5, 50)),
...             _execute_kwargs=dict(
...                 clear_cache=50,
...                 collect_garbage=50
...             )
...         )
```



```

...     )
...
...     oos_start_index = start_index + period ❸
...     oos_end_index = start_index + 2 * period - pd.Timedelta(nanoseconds=1)
...     oos_data = data[oos_start_index : oos_end_index]
...     oos_bl_perf = oos_data.run("from_holding").sharpe_ratio
...     best_fw, best_sw = is_perf.idxmax()
...     oos_perf = sma_crossover_perf(oos_data, best_fw, best_sw)
...     oos_perf_index = is_perf.index[is_perf.index == (best_fw, best_sw)]
...     oos_perf = pd.Series([oos_perf], index=oos_perf_index) ❹
...
...     all_is_bounds[period_idx] = (is_start_index, is_end_index)
...     all_oos_bounds[period_idx + 1] = (oos_start_index, oos_end_index)
...     all_is_bl_perf[(split_idx, period_idx)] = is_bl_perf
...     all_oos_bl_perf[(split_idx, period_idx + 1)] = oos_bl_perf
...     all_is_perf[(split_idx, period_idx)] = is_perf
...     all_oos_perf[(split_idx, period_idx + 1)] = oos_perf
...     start_index = start_index + period ❺
...     split_idx += 1
...     period_idx += 1
...     pbar.update() ❻

```

- ❶ Get the first date in the index. Has the type `pandas.Timestamp`.
- ❷ Define the period as `pandas.Timedelta` to be able to combine it with `pandas.Timestamp`
- ❸ Range start (inclusive) and end (exclusive) index, baseline (i.e., buy-and-hold) performance, and optimization performance of each IS period
- ❹ Range start and end index, baseline performance, and validation performance of each OOS period
- ❺ Make sure to install `tqdm` to display the progress bar
- ❻ Why not a for-loop? Because we don't know the number of splits in advance. We process one split, increment the starting date, and repeat, which translates perfectly into a while-loop. We do it as long as both periods are fully contained in our index.
- ❼ The right bound of the IS period should be **exclusive** since we don't want it to overlap with the left bound of the OOS period, but since `pandas.DataFrame.loc` includes the right bound, we simply subtract one nanosecond from it (remember this trick!)
- ❽ The IS period covers the days from [0, 180) while the OOS period covers the days from [180, 360)
- ❾ The function call above returns a single value, but let's make it a Series with a proper index to know which parameter combination it corresponds to
- ❿ The next IS period starts where the current IS period ends
- ⓫ Tell the progress bar that we processed this split

Period 9/9

```

>>> is_period_ranges = pd.DataFrame.from_dict( ❶
...     all_is_bounds,
...     orient="index",
...     columns=["start", "end"]
... )
>>> is_period_ranges.index.name = "period"
>>> oos_period_ranges = pd.DataFrame.from_dict(
...     all_oos_bounds,
...     orient="index",
...     columns=["start", "end"]
... )
>>> oos_period_ranges.index.name = "period"
>>> period_ranges = pd.concat((is_period_ranges, oos_period_ranges)) ❷
>>> period_ranges = period_ranges.drop_duplicates()
>>> period_ranges

```

	start	end
period		
0	2017-08-17 00:00:00+00:00	2018-02-12 23:59:59.999999999+00:00
1	2018-02-13 00:00:00+00:00	2018-08-11 23:59:59.999999999+00:00
2	2018-08-12 00:00:00+00:00	2019-02-07 23:59:59.999999999+00:00
3	2019-02-08 00:00:00+00:00	2019-08-06 23:59:59.999999999+00:00
4	2019-08-07 00:00:00+00:00	2020-02-02 23:59:59.999999999+00:00
5	2020-02-03 00:00:00+00:00	2020-07-31 23:59:59.999999999+00:00
6	2020-08-01 00:00:00+00:00	2021-01-27 23:59:59.999999999+00:00
7	2021-01-28 00:00:00+00:00	2021-07-26 23:59:59.999999999+00:00
8	2021-07-27 00:00:00+00:00	2022-01-22 23:59:59.999999999+00:00
9	2022-01-23 00:00:00+00:00	2022-07-21 23:59:59.999999999+00:00

```

>>> is_bl_perf = pd.Series(all_is_bl_perf) ❸
>>> is_bl_perf.index.names = ["split", "period"]
>>> oos_bl_perf = pd.Series(all_oos_bl_perf)
>>> oos_bl_perf.index.names = ["split", "period"]
>>> bl_perf = pd.concat(( ❹
...     is_bl_perf.vbt.select_levels("period"),
...     oos_bl_perf.vbt.select_levels("period")
... ))
>>> bl_perf = bl_perf.drop_duplicates()
>>> bl_perf

```

period	
0	1.846205
1	-0.430642
2	-1.741407
3	3.408079
4	-0.556471
5	0.954291
6	3.241618
7	0.686198
8	-0.038013
9	-0.917722

```

dtype: float64

>>> is_perf = pd.concat(all_is_perf, names=["split", "period"]) ⑤
>>> is_perf
split  period  fast_window  slow_window
0      0      5            6            1.766853
...      ...      ...      ...
8      8      46            48            0.043127
...      ...      ...      ...
48     48     49            49            0.159483
Length: 8910, dtype: float64

>>> oos_perf = pd.concat(all_oos_perf, names=["split", "period"])
>>> oos_perf
split  period  fast_window  slow_window
0      1      19            34            0.534007
1      2      6            7            -1.098628
2      3      18           20            1.687363
3      4      14           18            0.035346
4      5      18           21            1.877054
5      6      20           27            2.567751
6      7      11           18           -2.061754
7      8      29           30            0.965434
8      9      25           28            1.253361
dtype: float64

>>> is_best_mask = is_perf.index.vbt.drop_levels("period").isin( ⑥
...     oos_perf.index.vbt.drop_levels("period"))
>>> is_best_perf = is_perf[is_best_mask]
>>> is_best_perf
split  period  fast_window  slow_window
0      0      19            34            4.380746
1      1      6            7            2.538909
2      2      18           20            4.351354
3      3      14           18            3.605775
4      4      18           21            3.227437
5      5      20           27            3.362096
6      6      11           18            4.644594
7      7      29           30            3.379370
8      8      25           28            2.143645
dtype: float64

```

- ① If each value of a dict consists of multiple values, use `pandas.DataFrame.from_dict` to concatenate them into one DataFrame
- ② We know the bounds of each IS and OOS set, but since OOS periods are effectively next IS periods, let's create just one Series keyed by period
- ③ If each value of a dict is a single value, wrap the dict directly with Series
- ④ The same as with bounds, but we need to remove the `split` level beforehand
- ⑤ If each value of a dict is a Series, use `pandas.concat` to concatenate them into one Series
- ⑥ Get the performance of the best parameter combination in each IS set using the index of the corresponding OOS set, but since both Series are connected using `split`, `fast_window`, and `slow_window`, remove `period` prior to the operation

We've gathered information on 9 splits and 10 periods, it's time to evaluate the results! The index of each Series makes it almost too easy to connect information and analyze the entire thing as a whole: we can use the `split` level to connect elements that are part of the same split, the `period` level to connect elements that are part of the same time period, and `fast_window` and `slow_window` to connect elements by parameter combination. For starters, let's compare their distributions:

```

>>> pd.concat((
...     is_perf.describe(),
...     is_best_perf.describe(),
...     is_bl_perf.describe(),
...     oos_perf.describe(),
...     oos_bl_perf.describe()
... ), axis=1, keys=[
...     "IS",
...     "IS (Best)",
...     "IS (Baseline)",
...     "OOS (Test)",
...     "OOS (Baseline)"
... ])

```

	IS	IS (Best)	IS (Baseline)	OOS (Test)	OOS (Baseline)
count	8882.000000	9.000000	9.000000	9.000000	9.000000
mean	0.994383	3.514881	0.818873	0.639993	0.511770
std	1.746003	0.843435	1.746682	1.480066	1.786012
min	-3.600854	2.143645	-1.741407	-2.061754	-1.741407
25%	-0.272061	3.227437	-0.430642	0.035346	-0.556471
50%	1.173828	3.379370	0.686198	0.965434	-0.038013
75%	2.112042	4.351354	1.846205	1.687363	0.954291
max	4.644594	4.644594	3.408079	2.567751	3.408079

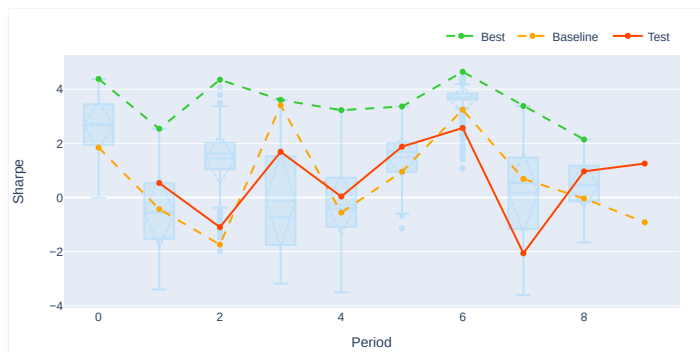
Even though the OOS results are far away from the best IS results, our strategy actually performs better (on average) than the baseline! More than 50% of periods have a Sharpe ratio of 0.96 or better, while for the baseline it's only -0.03. Another way of analyzing such information is by plotting it. Since all of those Series can be connected by period, we will use the `period` level as X-axis and the performance (Sharpe in our case) as Y-axis. Most Series can be plotted as lines, but since the IS sets capture multiple parameter combinations each, we should plot their distributions as boxes instead:

```

>>> fig = is_perf.vbt.boxplot( ❶
...     by_level="period", ❷
...     trace_kwargs=dict( ❸
...         line=dict(color="lightskyblue"),
...         opacity=0.4,
...         showlegend=False
...     ),
...     xaxis_title="Period", ❹
...     yaxis_title="Sharpe",
... )
>>> is_best_perf.vbt.select_levels("period").vbt.plot( ❺
...     trace_kwargs=dict(
...         name="Best",
...         line=dict(color="limegreen", dash="dash")
...     ),
...     fig=fig ❻
... )
>>> bl_perf.vbt.plot(
...     trace_kwargs=dict(
...         name="Baseline",
...         line=dict(color="orange", dash="dash")
...     ),
...     fig=fig
... )
>>> oos_perf.vbt.select_levels("period").vbt.plot(
...     trace_kwargs=dict(
...         name="Test",
...         line=dict(color="orangered")
...     ),
...     fig=fig
... )
>>> fig.show()

```

- ❶ Use [GenericAccessor.boxplot](#)
- ❷ This Series has many levels, we need to decide by which one(s) to aggregate
- ❸ This argument controls the appearance of the trace (see [plotly.graph_objects.Box](#))
- ❹ The first plotting method will create a Plotly figure and set it up, thus pass layout settings such as axis labels here
- ❺ Use [GenericAccessor.plot](#)
- ❻ Don't forget to pass the figure to plot the new trace on, otherwise it will create a new one



Here's how to interpret the plot above.

The green line follows the performance of the best parameter combination in each IS set; the fact that it touches the top-most point in each box proves that our best-parameter selection algorithm is correct. The dashed orange line follows the performance of the "buy-and-hold" strategy during each period, which acts as our baseline. The red line follows the test performance; it starts at the second range and corresponds to the parameter combination that yielded the best result during the previous period (i.e., the previous green dot).

The semi-opaque blue boxes represent the distribution of Sharpe ratios during IS (training) periods, that is, each box describes 990 parameter combinations that were tested during each period of optimization. There's no box on the far right because the last period is a OOS (test) period. For example, the period 6 (which is the seventh period because the counting starts from 0) incorporates all the Sharpe ratios ranging from 1.07 to 4.64, which most likely means that the price had an upward trend during that time. Here's the proof:

```

>>> is_perf_split6 = is_perf.xs(6, level="split") ❶
>>> is_perf_split6.describe()
count      990.000000
mean         3.638821
std          0.441206
min          1.073553
25%          3.615566
50%          3.696611
75%          3.844124
max          4.644594
dtype: float64

>>> first_left_bound = period_ranges.loc[6, "start"]
>>> first_right_bound = period_ranges.loc[6, "end"]
>>> data[first_left_bound : first_right_bound].plot().show() ❷

```

- ❶ Select the values with the `split` level being 6 and collect statistics from them

- ② Display the candlestick chart during that period for visual confirmation



No matter which parameter combination we choose during that period of time, the Sharpe ratio will stay relatively high and will likely delude us and make our strategy appear to be performing well. To make sure that this isn't the case, we need to analyze the test performance in relation to other points, which is the main reason why we drew the lines over the [box plot](#). For instance, we can see that during the period ⑥ both the baseline and the test performance are located below the first quartile (or 25th percentile) - they are worse than at least 75% of the parameter combinations tested in that time range:

```
>>> oos_perf.xs(6, level="period")
split fast_window slow_window
5      20         27         2.567751
dtype: float64

>>> is_perf_split6.quantile(0.25) ①
3.615566166097048
```

- ① 25% of parameter combinations have a Sharpe ratio below that value, including our parameter combination used for testing

The picture gives us mixed feelings: on the one hand, the picked parameter combination does better than most parameter combinations tested during 5 different time periods; on the other hand, it even fails to beat the lowest-performing 25% of parameter combinations during other 3 time periods. In defence of our strategy, the number of splits is relatively low: most statisticians agree that the minimum sample size to get any kind of meaningful result is 100, hence the analysis above gives us just a tiny glimpse into the true performance of a SMA crossover.

So, how can we simplify all of that?

[Python code](#)

[Notebook](#)

Tutorials Cross-validation

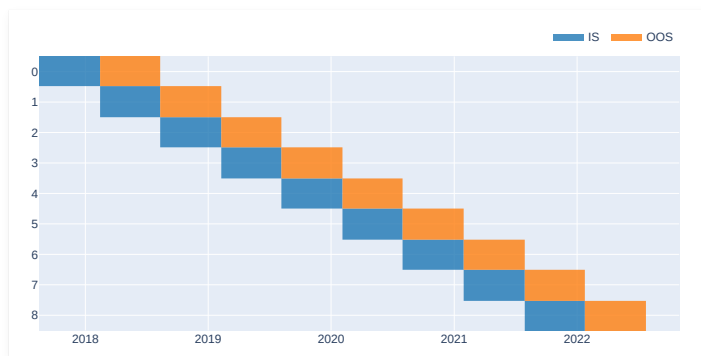
Splitter

The manual approach that we did previously can be decomposed into three distinctive steps: splitting the whole period into sub-periods, applying a UDF on each sub-period and merging its outputs, and analyzing the merged outputs using available data science tools. The first and the second step can be well automated; for example, [scikit-learn](#) has a range of [classes for cross-validation](#), each taking an array and performing some job on chunks of that array. The issue with that (otherwise excellent) Python package is that it lacks robust CV schemes for time series data, charting and analysis tools for split distributions, as well as an easy-to-extend interface for custom use cases. It also focuses on machine learning (ML) models that are trained on one data and validated on another by doing predictions (that's why it's called *scikit-learn*, after all), while rule-based algorithms that aren't predicting but producing a range of scores (one per test rather than data point) aren't receiving enough love.

That's why *vectorbt* needs to go its own way and implement a functionality cut to the needs of quantitative analysts rather than ML enthusiasts. At the heart of this functionality is the class [Splitter](#), whose main responsibility is to produce arbitrary splits and perform operations on those splits. The workings of this class are dead-simple: the user calls one of the class methods with the prefix `from_` (sounds familiar, right?) to generate splits; in return, a splitter instance is returned with splits and their labels being saved in a memory-efficient array format. This instance can be used to analyze the split distribution, to chunk array-like objects, and to run UDFs. Get yourself ready, because this class alone has twice as many lines of code as the entire [backtesting.py](#) library 🤖

Let's create a splitter for the schema of our first example:

```
>>> splitter = vbt.Splitter.from_rolling(
...     data.index,
...     length=360,
...     split=0.5,
...     set_labels=["IS", "OOS"]
... )
>>> splitter.plot().show()
```



That's it! We've got a splitter that can manipulate the periods (blue and orange boxes on the plot) and the data under those periods to our liking - no more while-loops. But before we dive into the dozens of implemented generation and analysis techniques, let's take a look under the hood and make ourselves comfortable with some basic concepts first.

Schema

The smallest unit of a splitter is a *range*, which is a period of time that can be mapped onto data. On the plot above, we can count a total of 18 ranges - 9 blue ones and 9 orange ones. Multiple ranges next to each other and representing a single test are called a *split*; there are 9 splits present in the chart, such that we expect one pipeline to be tested on 9 different data ranges. Different range types within each split are called *sets*. Usually, there is either one set, two sets - "training" and "test" (commonly used in backtesting), or three sets - "training", "validation", and "test" (commonly used in ML). The number of sets is fixed throughout all splits.

This schema fits perfectly into the philosophy of *vectorbt* because we can represent things in an array format where rows are splits, columns are sets, and elements are ranges:

```
>>> splitter.splits
set      IS      OOS
split
0      slice(0, 180, None)  slice(180, 360, None)
1      slice(180, 360, None)  slice(360, 540, None)
2      slice(360, 540, None)  slice(540, 720, None)
3      slice(540, 720, None)  slice(720, 900, None)
4      slice(720, 900, None)  slice(900, 1080, None)
5      slice(900, 1080, None)  slice(1080, 1260, None)
6      slice(1080, 1260, None)  slice(1260, 1440, None)
7      slice(1260, 1440, None)  slice(1440, 1620, None)
8      slice(1440, 1620, None)  slice(1620, 1800, None)
```

Notice how the index are split labels and columns are set labels: in contrast to other classes in *vectorbt*, the wrapper of this class doesn't represent time and assets, but splits and sets. Time is being tracked separately as [Splitter.index](#) while assets aren't being tracked at all since they have no implications on splitting.

```
>>> splitter.index
DatetimeIndex(['2017-08-17 00:00:00+00:00', '2017-08-18 00:00:00+00:00',
```

```

        '2017-08-19 00:00:00+00:00', '2017-08-20 00:00:00+00:00',
        ...
        '2022-10-28 00:00:00+00:00', '2022-10-29 00:00:00+00:00',
        '2022-10-30 00:00:00+00:00', '2022-10-31 00:00:00+00:00'],
        dtype='datetime64[ns, UTC]', name='Open time', length=1982, freq='D')

>>> splitter.wrapper.index
RangeIndex(start=0, stop=9, step=1, name='split')

>>> splitter.wrapper.columns
Index(['IS', 'OOS'], dtype='object', name='set')

```

Such a design has one nice property: we can apply indexing directly on a splitter instance to select specific splits and sets. Let's select the OOS set:

```

>>> oos_splitter = splitter["OOS"]
>>> oos_splitter.splits
split
0      slice(180, 360, None)
1      slice(360, 540, None)
2      slice(540, 720, None)
3      slice(720, 900, None)
4      slice(900, 1080, None)
5      slice(1080, 1260, None)
6      slice(1260, 1440, None)
7      slice(1440, 1620, None)
8      slice(1620, 1800, None)
Name: OOS, dtype: object

```

This operation has created a completely new splitter for OOS ranges 🤔

Why bother? Because we can select one set and apply a UDF on it, and then select the next set and apply a completely different UDF on it. Sounds like a prerequisite for CV, right?

Range format

So, how do ranges look like? In the first example of this tutorial, we used a start and end date to slice the data using `loc`. But as we also learned that the end date in a `loc` operation should be inclusive, which makes it annoyingly difficult to make sure that neighboring ranges do not overlap. Also, dates cannot be used to slice NumPy arrays, unless they are translated into positions beforehand. That's why the splitter does integer-location based indexing and accepts the following range formats that can be used to slice both Pandas (using `pandas.DataFrame.iloc`) and NumPy arrays:

- A list or array of integers, e.g. `[4, 3, 0]`
- A slice object with ints, e.g. `slice(1, 7)`
- A boolean array
- A callable function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above)

For example, the slice `slice(1, 7)` covers the indices `[0, 1, 2, 3, 4, 5, 6]`:

```

>>> index = vbt.date_range("2020", periods=14)
>>> index[slice(1, 7)] ❶
DatetimeIndex(['2020-01-02', '2020-01-03', '2020-01-04', '2020-01-05',
              '2020-01-06', '2020-01-07'],
              dtype='datetime64[ns]', freq='D')

>>> index[1], index[6]
(Timestamp('2020-01-02 00:00:00', freq='D'),
 Timestamp('2020-01-07 00:00:00', freq='D'))

```

❶ The same as `index[1:7]`

Having the index and integer-location based ranges kept separately makes designing non-overlapping, bug-free ranges super-easy.

Relative

The range format introduced above is called "fixed" because ranges do not depend on each other. But there's another range format called "relative" that makes one range depend on the previous range. For example, instead of defining a range between fixed index positions `110` and `150`, we can issue an instruction to create a range that starts `10` points after the end point of the previous range and has the length of `40`. Such an instruction can be created using `RelRange`:

```

>>> rel_range = vbt.RelRange(offset=10, length=40)
>>> rel_range
RelRange(
  offset=10,
  offset_anchor='prev_end',
  offset_space='free',
  length=40,
  length_space='free',
  out_of_bounds='warn',
  is_gap=False
)

```

This instruction will be evaluated later in time by calling `RelRange.to_slice`:

```

>>> rel_range.to_slice(total_len=len(splitter.index), prev_end=100) ❶
slice(110, 150, None)

```

① `prev_end` is the end point of the previous range

Relative ranges are usually converted into fixed ones prior to constructing a splitter instance, but splitter instances can also hold relative ranges in case this is wanted by the user.

Array format

But how do we store such range formats efficiently? The most flexible range formats are apparently indices and masks because they allow gaps and can enable a more traditional **k-fold cross-validation**, but they may also produce a huge memory footprint even for simple use cases. For example, consider a range stretching over 1 year of 1-minute data; it would take roughly 4MB of RAM as an integer array and 0.5MB as a mask:

```
>>> index = vbt.date_range("2020", "2021", freq="1min")
>>> range_ = np.arange(len(index))
>>> range_.nbytes / 1024 / 1024
4.02099609375

>>> range_ = np.full(len(index), True)
>>> range_.nbytes / 1024 / 1024
0.50262451171875
```

This means that only 100 splits and 2 sets would consume 800MB and 100MB of RAM respectively, and this is only to keep the splitter metadata in memory! Moreover, most "ranges" don't need to be that complex: they have predefined start and end points (that should occupy at most 18 bytes of memory), while being capable of pulling the same exact period of data as their integer and boolean array counterparts. That's why the **Splitter** class tries to convert any array into a slice, by the way.

To make sure that the user can make use of lightweight ranges, complex arrays, and relative ranges using the same API, the array that holds ranges has an object data type:

```
>>> splitter.splits_arr.dtype ①
dtype('O')
```

① **Splitter.splits_arr** is the NumPy (raw) version of **Splitter.splits**

Info

If an element of an array is a complex object, it doesn't mean that the array holds this entire object - it only holds the **reference** to that object:

```
>>> id(slice(0, 100, None))
140627839366784
```

The object data type is completely legit; it only becomes a burden once attempted to be passed to Numba, but the splitting functionality is entirely written in Python because the number of ranges (splits x sets) is usually kept relatively low such that the main bottleneck lies in running UDFs and not in iterating over ranges (and no worries, UDFs can still be run in Numba 😊). The other drawback is that the array cannot be numerically processed with NumPy or Pandas anymore, but that's why we have **Splitter** that can extract the meaning out of such an array!

This array format has even more benefits: we can use different range formats across different splits, we can store index arrays of different lengths, and since the `splits` array only stores references, we don't need to duplicate an array if two range values are pointing to the same array object. For example, let's construct a splitter where (differently-sized) ranges are stored as integer-location based arrays:

```
>>> range_00 = np.arange(0, 5)
>>> range_01 = np.arange(5, 15)
>>> range_10 = np.arange(15, 30)
>>> range_11 = np.arange(30, 50)

>>> ind_splitter = vbt.Splitter.from_splits(
...     data.index,
...     [[range_00, range_01], [range_10, range_11]],
...     fix_ranges=False
... )
>>> ind_splitter.splits
set
split
0      FixRange(range_=array([0, 1, 2, 3, 4]))
1      FixRange(range_=array([15, 16, 17, 18, 19, 20,...

set
split
0      FixRange(range_=array([ 5,  6,  7,  8,  9, 10,...
1      FixRange(range_=array([30, 31, 32, 33, 34, 35,...

>>> ind_splitter.splits.loc[0, "set_1"] ①
FixRange(range_=array([20, 21, 22, 23, 24, 25, 26, 27, 28, 29]))

>>> ind_splitter.splits.loc[0, "set_1"].range_ ②
array([20, 21, 22, 23, 24, 25, 26, 27, 28, 29])
```

① Get the fixed range for `range_01`

② Get the actual array

Hint

Why is the value of **FixRange** called `range_` and not `range`? Because `range` is a reserved keyword in Python.

As we see, each element of the splits array is... a NumPy array wrapped with **FixRange**, which is perfectly fine. Why are arrays wrapped and slices not? Because sub-arrays would expand the entire array to three dimensions, which are disliked by Pandas.

Preparation

In a nutshell, a splitter instance keeps track of three objects:

1. `wrapper` - Wrapper with split and set labels (**ArrayWrapper**)
2. `index` - Data index (mostly **pandas.DatetimeIndex**)
3. `splits` - Two-dimensional NumPy array with range objects

Although we can prepare these objects manually, there are convenient methods that automate this workflow for us. The base class method that most other class methods are based upon is **Splitter.from_splits**, which takes a sequence of splits, optionally does some pre-processing on each split, and converts that sequence into a suitable array format. It also prepares the labels and the wrapper. But let's focus on preparing the splits first.

Splits

Splitting is a process of dividing a bigger range into chunks of smaller ranges, which is implemented by the method **Splitter.split_range**. What it takes is a fixed range `range_` and a split specification `new_split`, and returns a tuple of new fixed ranges. Keep in mind that the returned ranges are always fixed, hence this method is also used to convert relative ranges to fixed. But the main use case of this method is to check whether the provided specification makes any sense and doesn't violate any bounds. Let's generate two ranges: one that takes 75% of space and another that takes the remaining 25%.

```
>>> vbt.Splitter.split_range(
...     slice(None), ①
...     (vbt.RelRange(length=0.75), vbt.RelRange()), ②
...     index=data.index
... )
(slice(0, 1426, None), slice(1426, 1902, None))
```

- ① Use the entire range to split. This effectively means `index[None:None]` or just `index[:]`. See [this explanation](#).
- ② If the length is not provided, the range will use 100% of the remaining space

Hint

This method is a hybrid method: it can be called both as a class method and instance method. If the latter, we don't need to provide the index since it's already stored:

```
>>> splitter.split_range(
...     slice(None),
...     (vbt.RelRange(length=0.75), vbt.RelRange())
... )
(slice(0, 1426, None), slice(1426, 1902, None))
```

These two slices can then be used to slice the data:

```
>>> data[slice(0, 1426, None)] ①
<vectorbtpro.data.custom.binance.BinanceData at 0x7fe12a7df310>
```

- ① Same as `data.iloc[:1426]`

The two relative ranges can be substituted by just one number, which translates into the length reserved for the first range, while the second range gets the remaining space:

```
>>> vbt.Splitter.split_range(
...     slice(None),
...     0.75,
...     index=data.index
... )
(slice(0, 1426, None), slice(1426, 1902, None))
```

Very often in CV, we want to fix the length of the OOS period and put everything else to the IS period. This can be specified by a negative number, which effectively reverses the processing order. For example, let's set the length of the OOS period to 25%:

```
>>> vbt.Splitter.split_range(
...     slice(None),
...     -0.25,
...     index=data.index
... )
(slice(0, 1427, None), slice(1427, 1902, None))
```

Hint

Why are both results not identical? Because of rounding:

```
>>> int(0.75 * len(data.index))
1426

>>> len(data.index) - int(0.25 * len(data.index))
1427
```


Or manually:

```
>>> vbt.Splitter.split_range(
...     slice(None),
...     (vbt.RelRange(), vbt.RelRange(length=0.25)),
...     backwards=True,
...     index=data.index
... )
(slice(0, 1427, None), slice(1427, 1902, None))
```

Relative ranges with just the length defined can be substituted by numbers for more convenience. For example, make the OOS period 30 data points long:

```
>>> vbt.Splitter.split_range(
...     slice(None),
...     (1.0, 30),
...     backwards=True,
...     index=data.index
... )
(slice(0, 1872, None), slice(1872, 1902, None))
```

Hint

How does the method decide whether the length is relative or absolute? If the number is between 0 and 1, the length is relative to the surrounding space, otherwise it reflects the number of data points.

When using relative lengths, we can also specify the space the length should be relative to using `RelRange.length_space`. By default, a length is relative to the remaining space (i.e., from the right-most bound of the previous range to the right-most bound of the whole period), but we can force it to be relative to the entire space instead. For example, let's define three ranges with 40%, 40%, and 20% respectively:

```
>>> vbt.Splitter.split_range(
...     slice(None),
...     (
...         vbt.RelRange(length=0.4, length_space="all"),
...         vbt.RelRange(length=0.4, length_space="all"),
...         vbt.RelRange()
...     ),
...     index=data.index
... )
(slice(0, 760, None), slice(760, 1520, None), slice(1520, 1902, None))
```

To introduce a gap between two ranges, we can use an offset. Similarly to lengths, offsets can be relative or absolute too. Also, offsets have an anchor, which defaults to the right-most bound of the previous range (if any, otherwise 0). Let's require both ranges to have a gap of 1 point:

```
>>> vbt.Splitter.split_range(
...     slice(None),
...     (vbt.RelRange(length=0.75), vbt.RelRange(offset=1)),
...     index=data.index
... )
(slice(0, 1426, None), slice(1427, 1902, None))
```

The exact same result can be achieved by placing a relative range of 1 data point between both ranges and enabling `RelRange.is_gap`:

```
>>> vbt.Splitter.split_range(
...     slice(None),
...     (
...         vbt.RelRange(length=0.75),
...         vbt.RelRange(length=1, is_gap=True),
...         vbt.RelRange()
...     ),
...     index=data.index
... )
(slice(0, 1426, None), slice(1427, 1902, None))
```

This method's power is not only in converting relative ranges into fixed ones, but also in trying to optimize the target ranges for best memory efficiency. Let's make the first range an array without gaps and the second range an array with gaps, and see what happens:

```
>>> vbt.Splitter.split_range(
...     slice(None),
...     (np.array([3, 4, 5]), np.array([6, 8, 10])),
...     index=data.index
... )
(slice(3, 6, None), array([ 6,  8, 10]))
```

We can see that the method was successful in optimizing the first array into a slice, but not the second. If such a conversion is not desired, we can disable it using the argument `range_format`:

```
>>> vbt.Splitter.split_range(
...     slice(None),
...     (np.array([3, 4, 5]), np.array([6, 8, 10])),
...     range_format="indices",
...     index=data.index
... )
(array([3, 4, 5]), array([ 6,  8, 10]))
```

Since `Splitter` uses integer-location and mask based indexing under the hood, we cannot use dates and times to slice arrays. Gladly, `Splitter.split_range` can take any slices and arrays as `pd.Timestamp`, `np.datetime64`, `datetime.datetime`, and even datetime-looking strings, and convert them into integers for us! It even

takes care of the timezone.

```
>>> vbt.Splitter.split_range(
...     slice(None),
...     (slice("2020", "2021"), slice("2021", "2022")), ❶
...     index=data.index
... )
(slice(867, 1233, None), slice(1233, 1598, None))

>>> data.index[867:1233]
DatetimeIndex(['2020-01-01 00:00:00+00:00', ..., '2020-12-31 00:00:00+00:00'],
              dtype='datetime64[ns, UTC]', name='Open time', length=366, freq='D')

>>> data.index[1233:1598]
DatetimeIndex(['2021-01-01 00:00:00+00:00', ..., '2021-12-31 00:00:00+00:00'],
              dtype='datetime64[ns, UTC]', name='Open time', length=365, freq='D')
```

❶ Stop date must be exclusive

The same goes for relative ranges where the arguments `offset` and `length` can be provided as `pd.Timedelta`, `np.timedelta64`, `datetime.timedelta`, and `timedelta-looking strings`:

```
>>> vbt.Splitter.split_range(
...     slice(None),
...     (
...         vbt.RelRange(length="180 days"),
...         vbt.RelRange(offset="1 day", length="90 days")
...     ),
...     index=data.index
... )
(slice(0, 180, None), slice(181, 271, None))
```

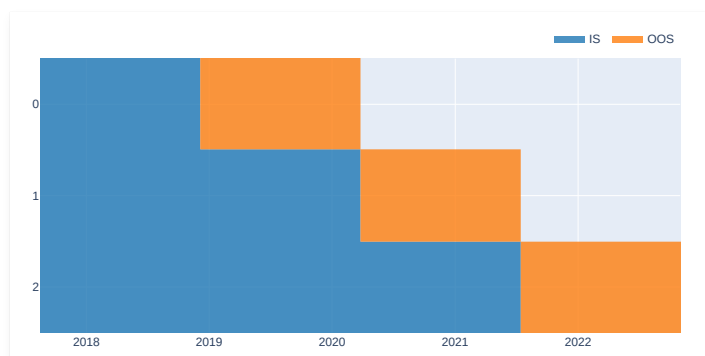
Method

Back to [Splitter.from_splits](#). We've learned that each split is being prepared by converting a split specification into a sequence of ranges, one per set. By passing multiple of such specifications, we thus get a two-dimensional array available under [Splitter.splits](#). Let's manually generate expanding splits with a OOS set having a fixed length of 25%:

```
>>> manual_splitter = vbt.Splitter.from_splits(
...     data.index,
...     [
...         (vbt.RelRange(), vbt.RelRange(offset=0.5, length=0.25, length_space="all")), ❶
...         (vbt.RelRange(), vbt.RelRange(offset=0.25, length=0.25, length_space="all")),
...         (vbt.RelRange(), vbt.RelRange(offset=0, length=0.25, length_space="all")),
...     ],
...     split_range_kwargs=dict(backwards=True), ❷
...     set_labels=["IS", "OOS"]
... )
>>> manual_splitter.splits
set          IS          OOS
split
0      slice(0, 476, None)  slice(476, 951, None)
1      slice(0, 952, None)  slice(952, 1427, None)
2      slice(0, 1427, None) slice(1427, 1902, None)

>>> manual_splitter.plot().show()
```

- ❶ Offset the range by 25% and reserve for it 25% of the entire period. Since we're moving backwards (see below), the operation starts at the end of the period and moves left.
- ❷ Reverse the range processing order as well inverse the period anchors



Generation

We know how to build a splitter manually, but most CV schemes involve generation through iteration, just like we did with the while-loop in our first example. Moreover, the start point of a split usually depends on the preceding split, which would require us to explicitly call [Splitter.split_range](#) on each split to get its boundaries. To reduce the amount of a boilerplate code required to enable such workflow, [Splitter](#) implements a collection of class methods, such as [Splitter.from_rolling](#), that can produce a logically-coherent schema from a simple user query.

Most of these methods first divide the entire period into windows (either in advance or iteratively), and then split each sub-period using the argument `split`, which is simply being passed to `Splitter.split_range` as `new_split`. This way, the split specification becomes relative the sub-period and not the entire period as we did above.

Info

Internally, `slice(None)` (that we used every time previously) is being replaced by the window slice such that `0.5` would split only the window in half, not the entire period.

Rolling

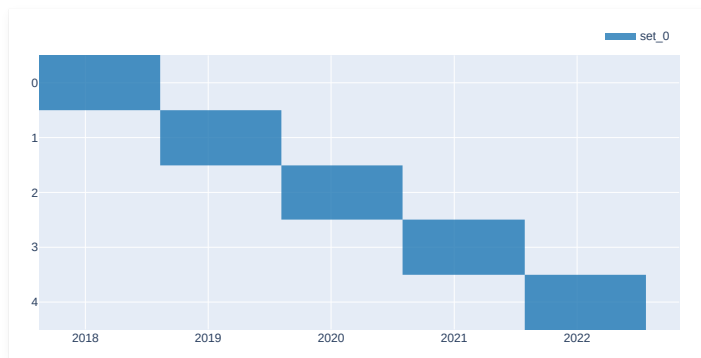
The most important method for CV is `Splitter.from_rolling`, which deploys a simple while-loop that appends splits until any split exceeds the right bound of the index. If the last split has the length less than the one requested, it gets discarded, such that there is usually some unused space at the end of the backtesting period.

But the most interesting question is: where do we place the next split? By default, if there's only one set, the next split is placed right after the previous one. If there are multiple sets though, the next split is placed right after the first (IS) range in the previous split, such that IS ranges never overlap across splits. And of course, we can control the offset behavior using `offset_anchor_set` (which range in the previous split acts as an anchor?), `offset_anchor` (left or right bound of that range acts as an anchor?), `offset` (positive/negative distance from anchor), and `offset_space` (see [RelRange](#)).

One set

```
>>> vbt.Splitter.from_rolling(
...     data.index,
...     length=360,
...     ).plot().show()
```

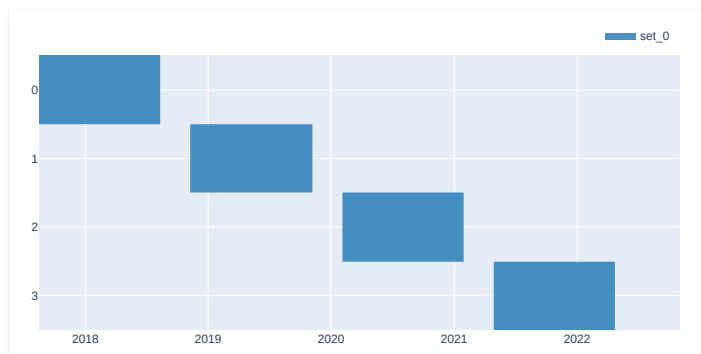
- 1 Length can be a number of bars (int), a percentage (float such as `0.1` for 10% of the entire period) and a duration (string such as `"360 days"` or any `timedelta`-like object)



One set with a gap

```
>>> vbt.Splitter.from_rolling(
...     data.index,
...     length=360,
...     offset=90,
...     ).plot().show()
```

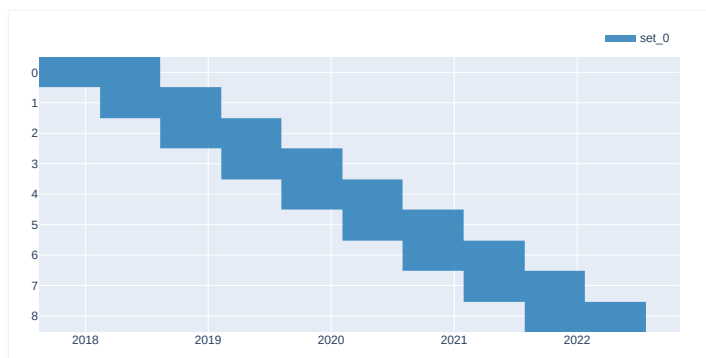
1. Similarly to a length, an offset can also be a percentage (but relative to `offset_space`) and a duration



One set with overlaps

```
>>> vbt.Splitter.from_rolling(
...     data.index,
...     length=360,
...     offset=-0.5,
...     ).plot().show()
```

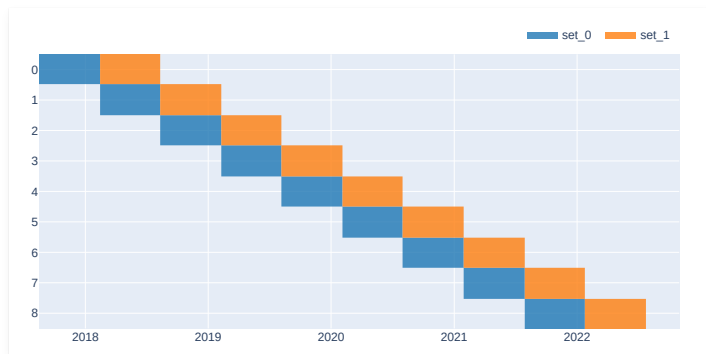
1. In contrast to a length, an offset can also be negative (also duration, for example `"-180 days"`). Since `offset_space` defaults to the length of the previous split or anchor range, this will move the next window to the left by 50% of the previous window's length.



Two sets

```
>>> vbt.Splitter.from_rolling(
...     data.index,
...     length=360,
...     split=0.5 ①
... ).plot().show()
```

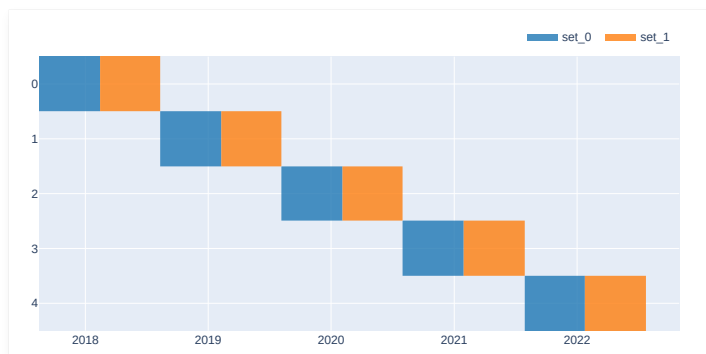
1. Split instruction passed to `Splitter.split_range` to split each window. Here, allocate 50% for the first range and the remaining space for the second.



Two sets without overlaps

```
>>> vbt.Splitter.from_rolling(
...     data.index,
...     length=360,
...     split=0.5,
...     offset_anchor_set=None ①
... ).plot().show()
```

1. Use the entire previous split (`set_0 + set_1`) as an anchor. Otherwise, it would use only the first range in that split (`set_0`).

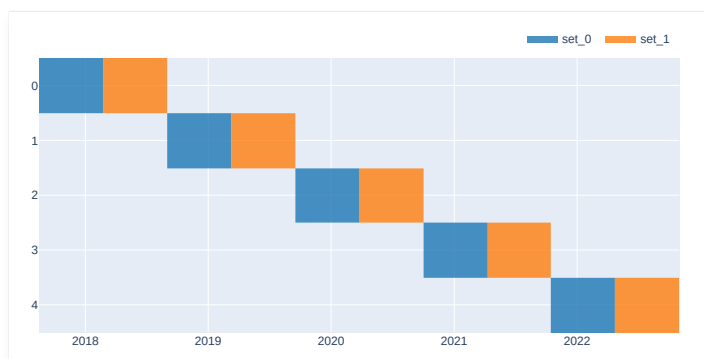


Another popular approach is by dividing the entire period into `n` equally-spaced, potentially-overlapping windows, which is implemented by `Splitter.from_n_rolling`. If the length of the window is `None` (i.e., not provided), it simply calls `Splitter.from_rolling` with the length set to `len(index) // n`. Note that in contrast to the previous method, this one doesn't allow us to control the offset.

Without length

```
>>> vbt.Splitter.from_n_rolling(
...     data.index,
...     n=5,
```

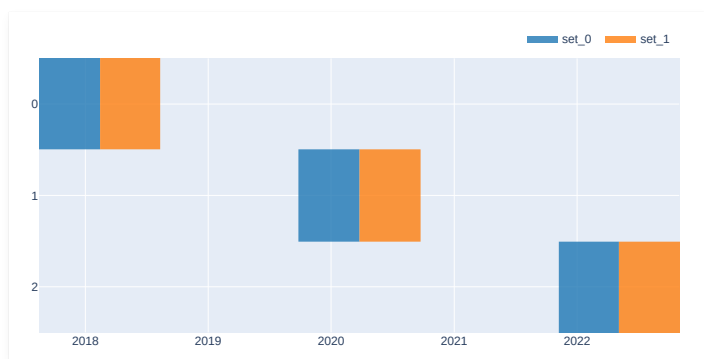
```
... split=0.5
... ).plot().show()
```



With length and without overlaps

```
>>> vbt.Splitter.from_n_rolling(
...     data.index,
...     n=3,
...     length=360,
...     split=0.5
... ).plot().show()
```

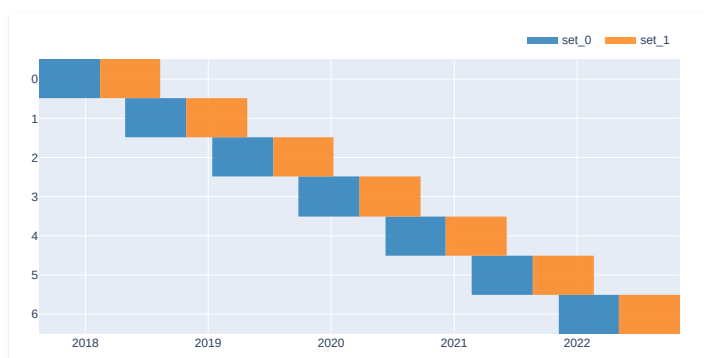
1. Splits are not overlapping when `n` is small enough



With length and with overlaps

```
>>> vbt.Splitter.from_n_rolling(
...     data.index,
...     n=7,
...     length=360,
...     split=0.5
... ).plot().show()
```

1. Splits are overlapping when `n` is big enough

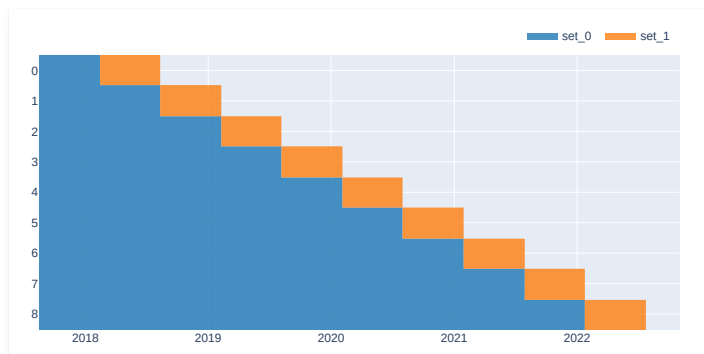


The windows that we've generated above have all the same length, which makes it easier to conduct fair experiments in backtesting. But sometimes, especially when training ML models, we need every training period to incorporate all the previous history. Such windows are called expanding and can be generated automatically using [Splitter.from_expanding](#), which works similarly to its rolling counterpart except that the offset controls the number of windows, the offset anchor is always the end of the previous split (window), and there's an argument `min_length` for a minimum window length. There's also a method [Splitter.from_n_expanding](#) that allows us to generate a predefined number of expanding windows.

Using offset

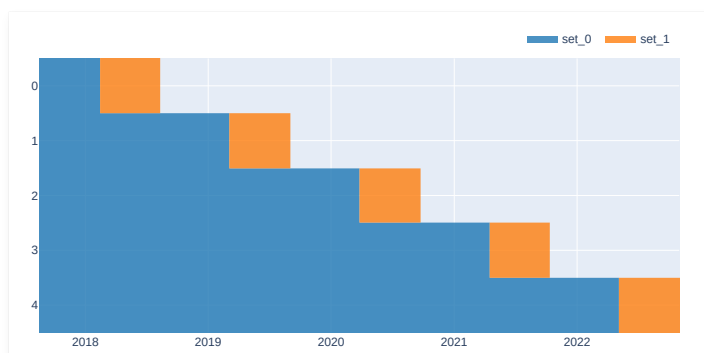
```
>>> vbt.Splitter.from_expanding(
...     data.index,
...     min_length=360, ①
...     offset=180, ②
...     split=-180 ③
... ).plot().show()
```

- ① Same format as `length` in the rolling version
- ② Same format as `offset` in the rolling version but must be always positive
- ③ Minus means set resolution in reversed order: 180 bars for the second (OOS) range and everything else for the first (IS) set



Using number of windows

```
>>> vbt.Splitter.from_n_expanding(
...     data.index,
...     n=5,
...     min_length=360,
...     split=-180
... ).plot().show()
```



Anchored

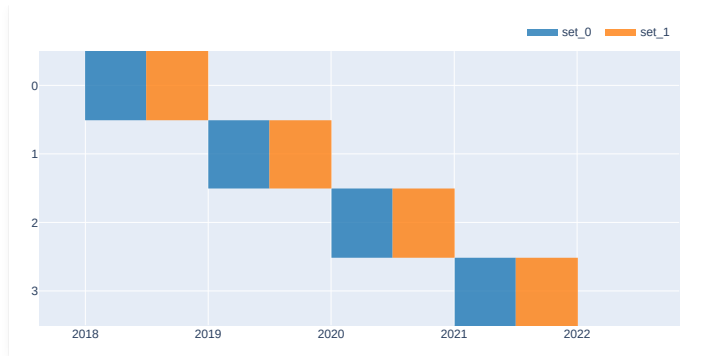
Let's consider a scenario where we want to generate a set of one-year long splits. Using any of the approaches above, we would get splits that last for one year but most likely start in somewhere in the middle of the year. But what if our requirement was for each split to start exactly at the beginning of the year? Such time anchors are only possible by grouping or resampling. There are two class methods in **Splitter** that enable this behavior: **Splitter.from_ranges** and **Splitter.from_grouper**.

The first method uses **get_index_ranges** to translate a user query into a set of start and end indices. It allows us to provide custom start and end dates, to resample using a lookahead period, to select a time range within each day, and more, - just like resampling but on steroids 🍌

Yearly

```
>>> vbt.Splitter.from_ranges(
...     data.index,
...     every="Y", ①
...     split=0.5
... ).plot().show()
```

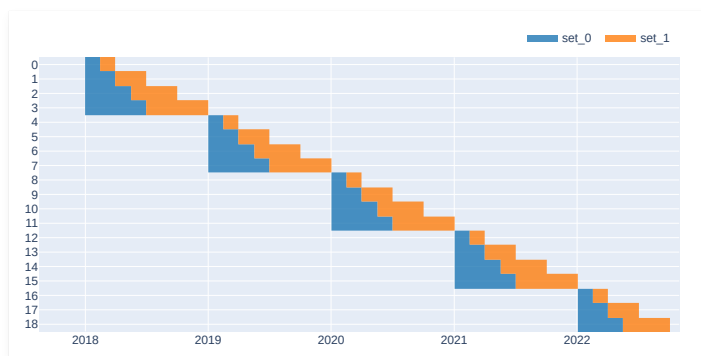
- ① Every year start, see [DateOffset objects](#)



Quarterly from year start

```
>>> vbt.Splitter.from_ranges(
...     data.index,
...     every="Q", ①
...     lookback_period="Y", ②
...     split=0.5
... ).plot().show()
```

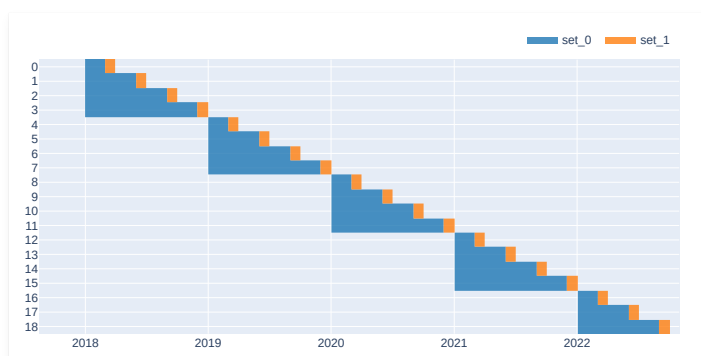
1. Every quarter start
2. Look back all the way to the year start



Last month for OOS

```
>>> vbt.Splitter.from_ranges(
...     data.index,
...     every="Q",
...     lookback_period="Y",
...     split=(
...         ①
...         vbt.RepEval("index.month != index.month[-1]"), ②
...         vbt.RepEval("index.month == index.month[-1]")
...     )
... ).plot().show()
```

1. Same as above but reserve the last calendar month for the OOS set in each split
2. Template should return a range (here a mask). Variable `index` in each template will be substituted by the index of the respective split.



Expanding with last quarter for OOS

```
>>> def qyear(index): ①
...     return index.to_period("Q")

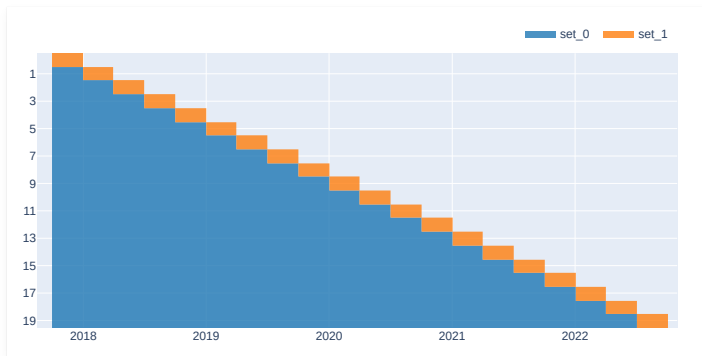
>>> vbt.Splitter.from_ranges(
...     data.index,
```

```

...     start=0,
...     fixed_start=True, ②
...     every="Q",
...     closed_end=True,
...     split=(
...         lambda index: qyear(index) != qyear(index)[-1], ③
...         lambda index: qyear(index) == qyear(index)[-1] ④
...     )
... ).plot().show()

```

1. Function to classify each date by quarter
2. Make each range start from the very first bar
3. Assign each quarter apart from the last one to the IS set
4. Assign the last quarter to the IS set



The second method takes a grouping or resampling instruction and converts each group into a split. It's based on the method [BaseDXAccessor.get_grouper](#), and accepts a variety of formats from both vectorbt and Pandas, even [pandas.Grouper](#) and [pandas.Resampler](#). The only issue that we may encounter are incomplete splits, which can be filtered out using a template provided as `split_check_template` and forwarded down to [Splitter.from_splits](#).

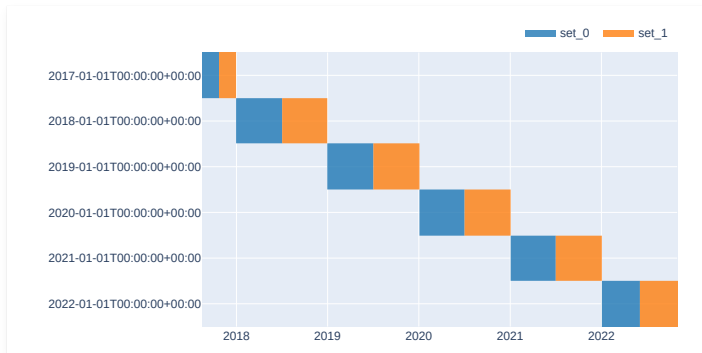
Resampling annually

```

>>> vbt.Splitter.from_grouper(
...     data.index,
...     by="Y", ①
...     split=0.5
... ).plot().show()

```

- ① Will be recognized as a resampling instruction



Removing incomplete years

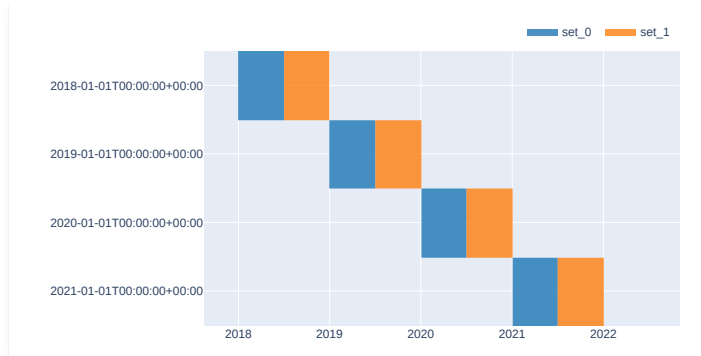
```

>>> def is_split_complete(index, split): ①
...     first_range = split[0]
...     first_index = index[first_range][0]
...     last_range = split[-1]
...     last_index = index[last_range][-1]
...     return first_index.is_year_start and last_index.is_year_end

>>> vbt.Splitter.from_grouper(
...     data.index,
...     by="Y",
...     split=0.5,
...     split_check_template=vbt.RepFunc(is_split_complete) ②
... ).plot().show()

```

1. Check whether the first day is the year's start and the last day is the year's end. This function will become a template, such that vectorbt will substitute `index` for the `full` index and `split` for the current split. Should return False if the split should be discarded.
2. Wrap with `RepFunc` to make a template out of `is_split_complete`

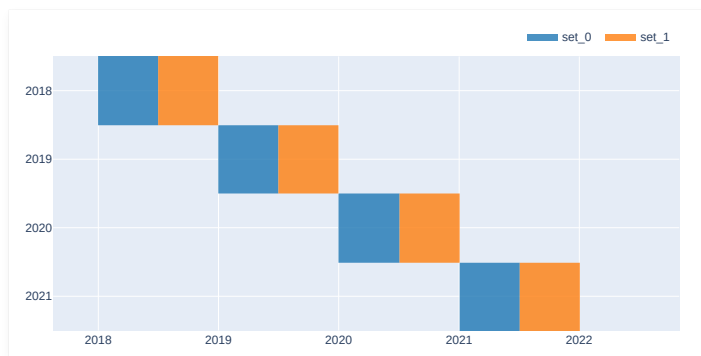


Formatting labels

```
>>> def format_split_labels(index, splits_arr): ❶
...     years = map(lambda x: index[x[0]][0].year, splits_arr) ❷
...     return pd.Index(years, name="split_year")

>>> vbt.Splitter.from_grouper(
...     data.index,
...     by="Y",
...     split=0.5,
...     split_check_template=vbt.RepFunc(is_split_complete),
...     split_labels=vbt.RepFunc(format_split_labels) ❸
... ).plot().show()
```

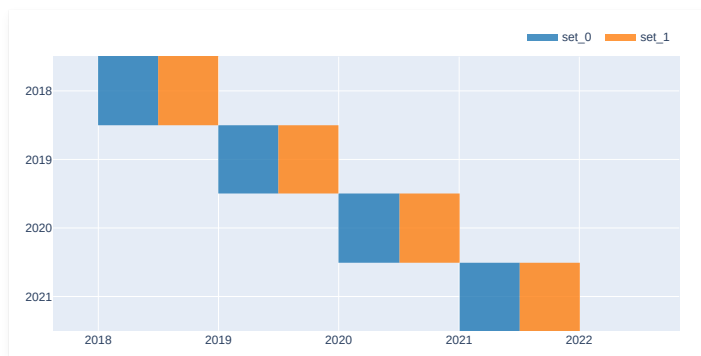
1. Extract the year from each split. This function will become a template, such that vectorbt will substitute `index` for the full index and `splits_arr` for the splits array. Should return an index-like object.
2. For each split, get the year of the first timestamp in the first range
3. Split labels can be an index-like object or a template



Using grouping

```
>>> vbt.Splitter.from_grouper(
...     data.index,
...     by=data.index.year, ❶
...     split=0.5,
...     split_check_template=vbt.RepFunc(is_split_complete) ❷
... ).plot().show()
```

1. Regular Pandas Index with years as values. Each unique value will become a split label.
2. Still need to check for completeness of splits, but at least labels are already formatted



Random

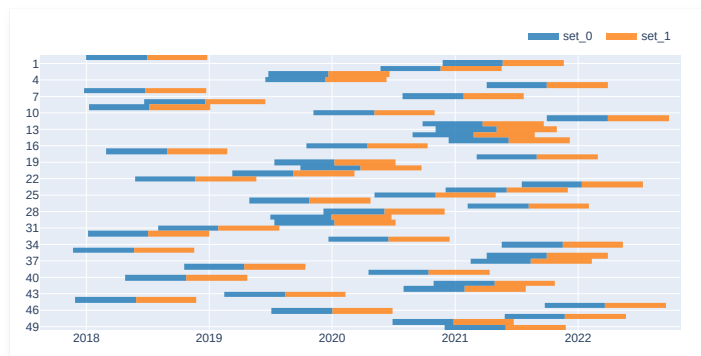
So far, we generated windows based on some pre-defined schema. But there is also a special place for randomness in CV, especially when it comes to **bootstrapping** and **block bootstrap** in particular. The method `Splitter.from_n_random` draws a predefined number of windows of a (optionally) variable length. At the heart of this method are two callbacks: `length_choice_func` and `start_choice_func`, selecting the next window's length and start point respectively. By default, they are set to `numpy.random.Generator.choice`, which generates a random sample with replacement (i.e., the same window can occur more than once). Another two callbacks, `length_p_func` and `start_p_func`, can control the probabilities of picking each entry (e.g., to select more windows to the right of the period).

Fixed length

```
>>> vbt.Splitter.from_n_random(
...     data.index,
...     n=50,
...     min_length=360, ①
...     seed=42, ②
...     split=0.5
... ).plot().show()
```

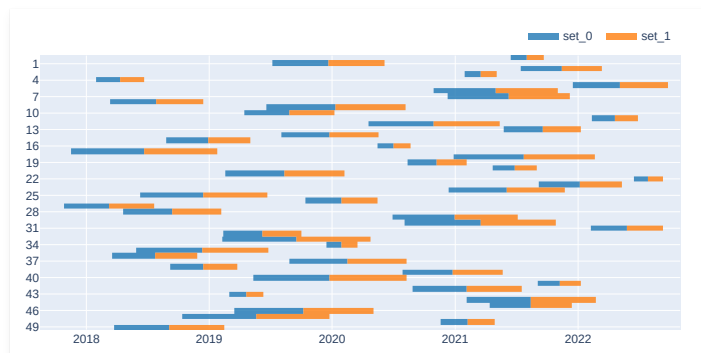
① Without `max_length`, this becomes the fixed length

② Set a seed to make the results the same with each run



Variable length

```
>>> vbt.Splitter.from_n_random(
...     data.index,
...     n=50,
...     min_length=30,
...     max_length=300,
...     seed=42,
...     split=0.5
... ).plot().show()
```

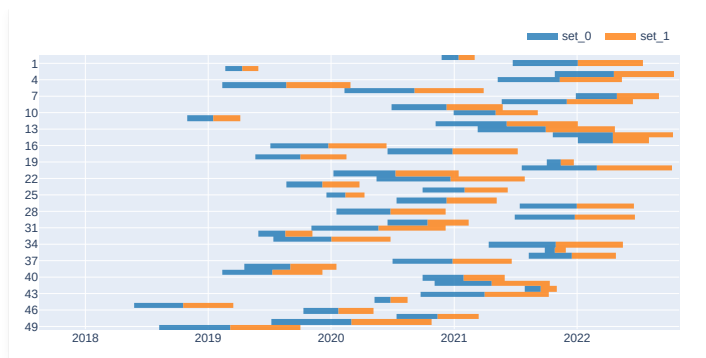


With probabilities

```
>>> def start_p_func(i, indices): ①
...     return indices / indices.sum()

>>> vbt.Splitter.from_n_random(
...     data.index,
...     n=50,
...     min_length=30,
...     max_length=300,
...     seed=42,
...     start_p_func=start_p_func,
...     split=0.5
... ).plot().show()
```

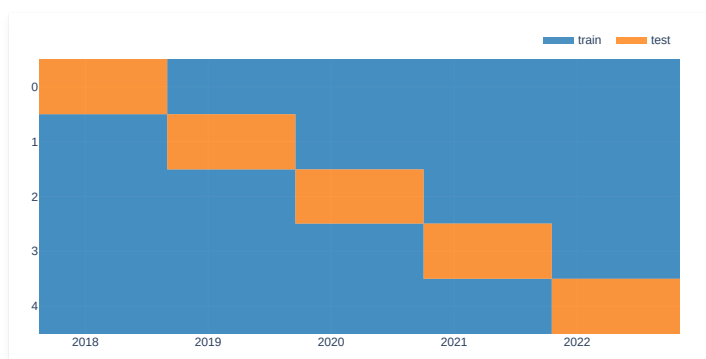
1. Takes the iteration number and a set of potential start points



Scikit-learn

For k-fold and many other standard CV schemes where scikit-learn has an upper hand, there is a method `Splitter.from_sklearn` that can parse just about every cross-validator that subclasses the scikit-learn's `BaseCrossValidator` class.

```
>>> from sklearn.model_selection import KFold
>>> vbt.Splitter.from_sklearn(
...     data.index,
...     KFold(n_splits=5)
... ).plot().show()
```



Warning

There is a temporal dependency between observations: it makes no sense to use the values from the future to forecast values in the past, thus make sure that the test period always succeeds the training period.

Dynamic

The final and the most flexible generation method involves calling a UDF that takes a context including all the splits generated previously, and returns a new split to be appended. This all happens in an infinite while-loop; to break out of the loop, the UDF must return `None`. Similarly to many other methods in vectorbt that take functions as arguments, this one also makes use of templates to substitute them for information from the context. The context itself includes the appended and resolved splits (`splits`) and the bounds of each range in each split (but only when `fix_ranges=True`), which makes the generation process a child's play.

Roll one-year window each month

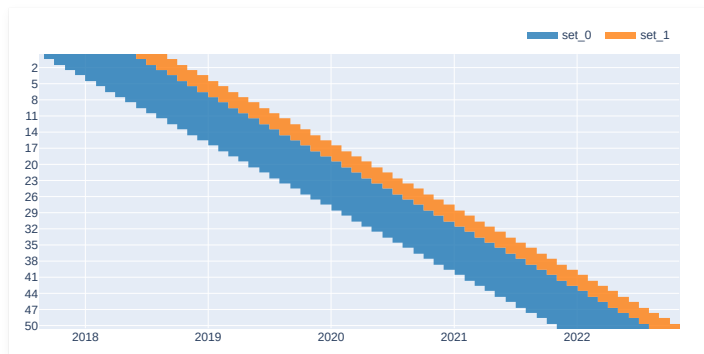
```
>>> def split_func(index, prev_start):
...     if prev_start is None: ①
...         prev_start = index[0]
...     new_start = prev_start + pd.offsets.MonthBegin(1) ②
...     new_end = new_start + pd.DateOffset(years=1) ③
...     if new_end > index[-1] + index.freq: ④
...         return None
...     return [
...         slice(new_start, new_start + pd.offsets.MonthBegin(9)), ⑤
...         slice(new_start + pd.offsets.MonthBegin(9), new_end)
...     ]

>>> vbt.Splitter.from_split_func(
...     data.index,
...     split_func=split_func,
...     split_args=(vbt.Rep("index"), vbt.Rep("prev_start")), ⑥
...     range_bounds_kwargs=dict(index_bounds=True) ⑦
... ).plot().show()
```

① If this is the first split, `prev_start` (i.e., the start index of the previous split) will be `None`

② The start date of this window is the beginning of the next month (inclusive)

- ③ The end date of this window is the same date but in the next year (exclusive)
- ④ If the split is incomplete (i.e., the end date comes after the next possible end date), abort!
- ⑤ Allocate 9 months for the IS period and 3 months for the OOS period
- ⑥ Tell vectorbt to replace these strings by the respective context
- ⑦ Tell vectorbt to return the bounds of each range as timestamps, not number of bars, otherwise `prev_start` will be an integer



Train one business week, test next

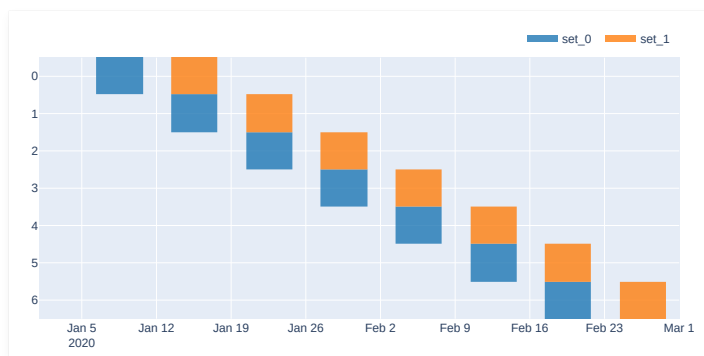
```
>>> def get_next_monday(from_date):
...     if from_date.weekday == 0 and from_date.ceil("H").hour <= 9: ①
...         return from_date.floor("D")
...     return from_date.floor("D") + pd.offsets.Week(n=0, weekday=0) ②

>>> def get_next_business_range(from_date):
...     monday_0000 = get_next_monday(from_date)
...     monday_0900 = monday_0000 + pd.DateOffset(hours=9) ③
...     friday_1700 = monday_0900 + pd.DateOffset(days=4, hours=8) ④
...     return slice(monday_0900, friday_1700)

>>> def split_func(index, bounds):
...     if len(bounds) == 0:
...         from_date = index[0] ⑤
...     else:
...         from_date = bounds[-1][1][0] ⑥
...     train_range = get_next_business_range(from_date)
...     test_range = get_next_business_range(train_range.stop)
...     if test_range.stop > index[-1] + index.freq:
...         return None
...     return train_range, test_range

>>> vbt.Splitter.from_split_func(
...     vbt.date_range("2020-01", "2020-03", freq="15min"), ⑦
...     split_func=split_func,
...     split_args=(vbt.Rep("index"), vbt.Rep("bounds")),
...     range_bounds_kwargs=dict(index_bounds=True)
... ).plot().show()
```

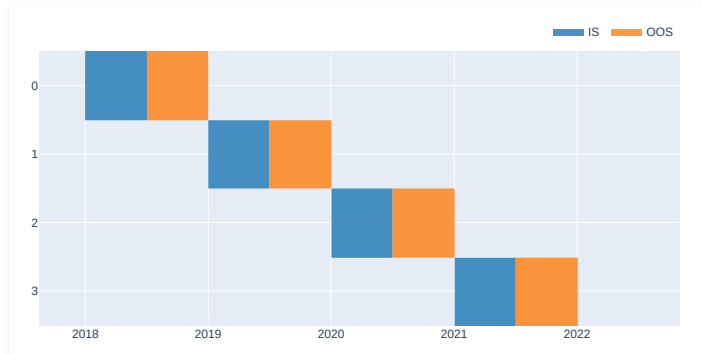
1. If `from_date` is a Monday 09:00 or earlier, normalize it to get the Monday 00:00
2. If not, normalize and find the next Monday 00:00
3. Add 9 hours to get the Monday 09:00
4. Add 4 days and 9 hours to get the Friday 17:00
5. Start from the first date if this is the first split
6. Use the start date of the previous test set as the start date of the new train set
7. Generate a 15-minute index for this example



Validation

Consider the following splitter that splits the entire period into years, one per split:

```
>>> splitter = vbt.Splitter.from_ranges(
...     data.index,
...     every="Y",
...     closed_end=True,
...     split=0.5,
...     set_labels=["IS", "OOS"]
... )
>>> splitter.plot().show()
```



By default, `closed_end` is `False` such that any neighboring ranges do not overlap, but we deliberately made a mistake here by setting `closed_end` to `True`, hence we've produced splits that overlap by exactly one bar. How do we detect such a mistake post-factum? The [Splitter](#) class has various tools exactly for that.

Bounds

The first tool involves computing the bounds of each range. The bounds consist of two numbers (`index_bounds=False`) or dates (`index_bounds=True`): the start (always inclusive) and the end (exclusive, but can be inclusive using `right_inclusive=True`). Depending on what the analysis goal is, they can be returned in two different formats. The first format is a three-dimensional NumPy array returned by [Splitter.get_bounds_arr](#) where the first axis are splits, the second axis are sets, and the third axis are bounds:

```
>>> bounds_arr = splitter.get_bounds_arr() ❶
>>> bounds_arr.shape
(4, 2, 2)

>>> bounds_arr
[[[ 137  320]
 [ 320  503]
 [ 502  685]
 [ 685  868]
 [ 867 1050]
 [1050 1234]
 [1233 1416]
 [1416 1599]]]
```

❶ Also, available via the property [Splitter.bounds_arr](#)

Another, probably a user-friendlier, format is a DataFrame returned by [Splitter.get_bounds](#) where rows represent ranges and columns represent bounds:

```
>>> bounds = splitter.get_bounds(index_bounds=True) ❶
>>> bounds.shape
(8, 2)

>>> bounds
bound      start      end
split set
0    IS  2018-01-01 00:00:00+00:00 2018-07-03 00:00:00+00:00
    OOS 2018-07-03 00:00:00+00:00 2019-01-02 00:00:00+00:00
1    IS  2019-01-01 00:00:00+00:00 2019-07-03 00:00:00+00:00
    OOS 2019-07-03 00:00:00+00:00 2020-01-02 00:00:00+00:00
2    IS  2020-01-01 00:00:00+00:00 2020-07-02 00:00:00+00:00
    OOS 2020-07-02 00:00:00+00:00 2021-01-02 00:00:00+00:00
3    IS  2021-01-01 00:00:00+00:00 2021-07-03 00:00:00+00:00
    OOS 2021-07-03 00:00:00+00:00 2022-01-02 00:00:00+00:00
```

❶ Also, available as properties [Splitter.bounds](#) and [Splitter.index_bounds](#)

We can then detect some IS ranges starting before the preceding OOS range ends:

```
>>> bounds.loc[(0, "OOS"), "end"]
Timestamp('2019-01-02 00:00:00+0000', tz='UTC')

>>> bounds.loc[(1, "IS"), "start"]
Timestamp('2019-01-01 00:00:00+0000', tz='UTC')
```

Masks

Another tool revolves around range masks. Since the index is the same for each single range, we can translate a range into a mask of the same length as the index and then stack all masks into a single array. Another advantage of masks is that we can combine them, apply logical operators on them, reduce them, and test them for `True` values. The only downside is clearly their memory consumption: as we calculated previously, 100 splits and 2 sets of 1 year of 1-minute data would consume whooping 100MB of RAM. Similarly to bounds, we have two methods that return a three-dimensional NumPy array and a DataFrame respectively:

[Splitter.get_mask_arr](#) and [Splitter.get_mask](#). Let's get the mask in the DataFrame format:

```
>>> mask = splitter.get_mask() ❶
>>> mask.shape
(1902, 8)

>>> mask
split      0      1      2      3 \
set      IS  OOS  IS  OOS  IS  OOS  IS
Open time
2017-08-17 00:00:00+00:00 False False False False False False False
2017-08-18 00:00:00+00:00 False False False False False False False
2017-08-19 00:00:00+00:00 False False False False False False False
...      ...      ...      ...      ...      ...      ...      ...
2022-10-29 00:00:00+00:00 False False False False False False False
2022-10-30 00:00:00+00:00 False False False False False False False
2022-10-31 00:00:00+00:00 False False False False False False False

split      OOS
set
Open time
2017-08-17 00:00:00+00:00 False
2017-08-18 00:00:00+00:00 False
2017-08-19 00:00:00+00:00 False
...      ...
2022-10-29 00:00:00+00:00 False
2022-10-30 00:00:00+00:00 False
2022-10-31 00:00:00+00:00 False

[1902 rows x 8 columns]
```

❶ Also, available via the property [Splitter.mask](#)

As opposed to bounds, the DataFrame lists split and set labels (i.e., range labels) in columns rather than rows. To demonstrate the power of masks, let's answer the following question: what ranges cover the year 2021?

```
>>> mask["2021":"2021"].any()
split  set
0      IS  False
      OOS  False
1      IS  False
      OOS  False
2      IS  False
      OOS  True
3      IS  True
      OOS  True
dtype: bool
```

We can spot the mistake once again: there's an OOS range that clearly overflows into the next year. Here's another question: get the number of dates covered by each set in each year.

```
>>> mask.resample(vbt.offset("Y")).sum() # (1)!
split      0      1      2      3
set      IS  OOS  IS  OOS  IS  OOS  IS  OOS
Open time
2017-01-01 00:00:00+00:00  0  0  0  0  0  0  0  0
2018-01-01 00:00:00+00:00 183 182 0  0  0  0  0  0
2019-01-01 00:00:00+00:00  0  1 183 182 0  0  0  0
2020-01-01 00:00:00+00:00  0  0  0  1 183 183 0  0
2021-01-01 00:00:00+00:00  0  0  0  0  0  1 183 182
2022-01-01 00:00:00+00:00  0  0  0  0  0  0  0  1
```

To mitigate potential memory issues, there are special approaches that translate only a subset of ranges into a mask at a time. Those approaches are based on two iteration schemas: by split and by set, implemented by [Splitter.get_iter_split_masks](#) and [Splitter.get_iter_set_masks](#) respectively. Each method returns a [Python generator](#) that can be iterated over in a loop. Let's answer the question above in a memory-friendly manner (if really needed):

```
>>> results = []
>>> for mask in splitter.get_iter_split_masks():
...     results.append(mask.resample(vbt.offset("Y")).sum())
>>> pd.concat(results, axis=1, keys=splitter.split_labels)
split      0      1      2      3
set      IS  OOS  IS  OOS  IS  OOS  IS  OOS
Open time
2017-01-01 00:00:00+00:00  0  0  0  0  0  0  0  0
2018-01-01 00:00:00+00:00 183 182 0  0  0  0  0  0
2019-01-01 00:00:00+00:00  0  1 183 182 0  0  0  0
2020-01-01 00:00:00+00:00  0  0  0  1 183 183 0  0
2021-01-01 00:00:00+00:00  0  0  0  0  0  1 183 182
2022-01-01 00:00:00+00:00  0  0  0  0  0  0  0  1
```

Coverage

Bounds and masks are convenient range formats that enable us to analyze ranges from various perspectives. To not delegate too much work to the user, there are additional methods that automate this analysis. Since we're mostly interested in knowing whether and how much do splits, sets, and ranges overlap, there are 4 methods that provide us with quick insights into that matter: [Splitter.get_split_coverage](#) for the coverage by split, [Splitter.get_set_coverage](#) for the coverage by set, [Splitter.get_range_coverage](#) for the coverage by range, and [Splitter.get_coverage](#) for the coverage by any of the above. For example, the split-relative coverage will return the % of bars in the index covered by each split, while the total coverage will return the % of bars in the index covered by any range.

Warning

Most of the methods below, apart from the plotting method, require the entire mask array to be in memory.

```
>>> splitter.get_split_coverage() ❶
split
0    0.192429
1    0.192429
2    0.192955
3    0.192429
Name: split_coverage, dtype: float64

>>> splitter.get_set_coverage() ❷
set
IS    0.384858
OOS   0.385384
Name: set_coverage, dtype: float64

>>> splitter.get_range_coverage() ❸
split  set
0      IS    0.096215
      OOS    0.096215
1      IS    0.096215
      OOS    0.096215
2      IS    0.096215
      OOS    0.096740
3      IS    0.096215
      OOS    0.096215
Name: range_coverage, dtype: float64

>>> splitter.get_coverage() ❹
0.768664563617245
```

- ❶ Also, available via the property [Splitter.split_coverage](#)
- ❷ Also, available via the property [Splitter.set_coverage](#)
- ❸ Also, available via the property [Splitter.range_coverage](#)
- ❹ Also, available via the property [Splitter.coverage](#)

Note

The default arguments will always return a metric relative to the length of the index.

As we can see above, the first split covers 19.24% of the entire period, while both ranges in that split take 9.62%, or exactly 50% of the split. Both sets cover roughly the same period of time - 38.53% of the entire period. The last metric tells us that 23.14% of the entire period aren't covered by the splitter, which makes sense because the years 2017 and 2022 are incomplete such that no split was produced for either of them. Finally, why do all ranges cover the same period of time except the OOS set in the split 2? It's because that year was a leap year such that the last months had one day more than the same months in other years:

```
>>> splitter.index_bounds.loc[(2, "OOS"), "start"].is_leap_year
True
```

So far, we analyzed coverage in relation to the full index. But is a special argument `relative` that allows us to analyze splits and sets relatively to the total coverage, as well as ranges relatively to the split coverage. For example, let's get the fraction of IS and OOS sets in their respective splits:

```
>>> splitter.get_range_coverage(relative=True)
split  set
0      IS    0.500000
      OOS    0.500000
1      IS    0.500000
      OOS    0.500000
2      IS    0.498638
      OOS    0.501362
3      IS    0.500000
      OOS    0.500000
Name: range_coverage, dtype: float64
```

Most periods except the leap year have a perfect 50/50 split, as we wanted. We can expand our analysis to answer whether sets in general follow the same 50/50 split:

```
>>> splitter.get_set_coverage(relative=True)
set
IS    0.500684
OOS   0.501368
Name: set_coverage, dtype: float64
```

Both numbers do not sum to one, which (again) hints at overlapping ranges. By using the argument `overlapping`, we can assess any overlaps of sets within each split, any overlaps of splits within each set, and any overlaps of ranges globally:

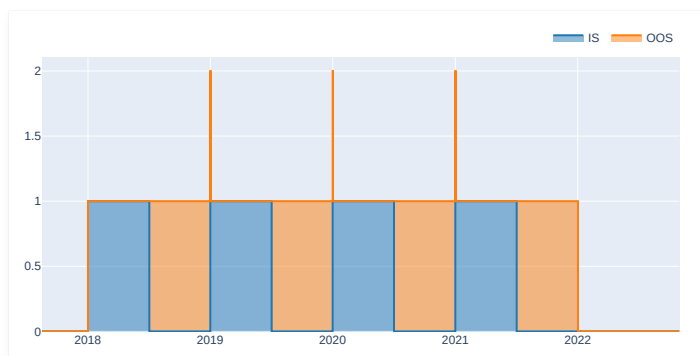
```
>>> splitter.get_split_coverage(overlapping=True)
split
0    0.0
1    0.0
2    0.0
3    0.0
Name: split_coverage, dtype: float64

>>> splitter.get_set_coverage(overlapping=True)
set
IS    0.0
OOS   0.0
Name: set_coverage, dtype: float64

>>> splitter.get_coverage(overlapping=True)
0.002051983584131327
```

Ranges neither overlap within each split, nor within each set. But there are still some overlaps of ranges recorded globally, which means they belong to opposite splits and sets. To have a better look, let's visualize the coverage using [Splitter.plot_coverage](#):

```
>>> splitter.plot_coverage().show()
```



The Y-axis represents the total number of ranges that cover one particular date. We can see that there are three dates covered by two ranges simultaneously.

The last and the most powerful tool in overlap detection are overlap matrices, which compute overlaps either between splits (`by="split"`), sets (`by="set"`), or ranges (`by="range"`). If `normalize` is `True`, which is the default value, the intersection of two range masks will be normalized by their union. Even though the operation is Numba-compiled, it's still very expensive: 100 splits and 2 sets would require going through all $200 * 200 = 40000$ range pairs to build the range overlap matrix. Let's finally shed some light on the overlapping ranges:

```
>>> splitter.get_overlap_matrix(by="range", normalize=False) ①
split      0      1      2      3
set      IS OOS  IS OOS  IS OOS  IS OOS
split set
0      IS 183   0   0   0   0   0   0
   OOS   0 183   1   0   0   0   0
1      IS   0   1 183   0   0   0   0
   OOS   0   0   0 183   1   0   0
2      IS   0   0   0   1 183   0   0
   OOS   0   0   0   0   0 184   1
3      IS   0   0   0   0   0   1 183   0
   OOS   0   0   0   0   0   0 183
```

① The default version is also available via the property [Splitter.range_overlap_matrix](#)

Grouping

Each of the methods above (and many others including [Splitter.plot](#)) accept the arguments `split_group_by` and `set_group_by`, which allow grouping splits and sets respectively. Their format is identical to the format of the argument `group_by`, which appears just about everywhere in the vectorbt's codebase. For example, by passing `True`, we can put all ranges into the same bucket and merge them. We can also pass a list of the same length as the number of splits/sets such that the splits/sets under the same unique value will get merged. The actual merging part is being done by the method [Splitter.merge_split](#).

For example, let's get the bounds of each entire split:

```
>>> splitter.get_bounds(index_bounds=True, set_group_by=True)
bound      start      end
split set_group
0      group 2018-01-01 00:00:00+00:00 2019-01-02 00:00:00+00:00
1      group 2019-01-01 00:00:00+00:00 2020-01-02 00:00:00+00:00
2      group 2020-01-01 00:00:00+00:00 2021-01-02 00:00:00+00:00
3      group 2021-01-01 00:00:00+00:00 2022-01-02 00:00:00+00:00
```

This makes certain kinds of analysis much easier 🚀

Manipulation

We'll end this page with an overview of the methods that can be used to change a splitter. Let's build a splitter that has just one set representing the current year:

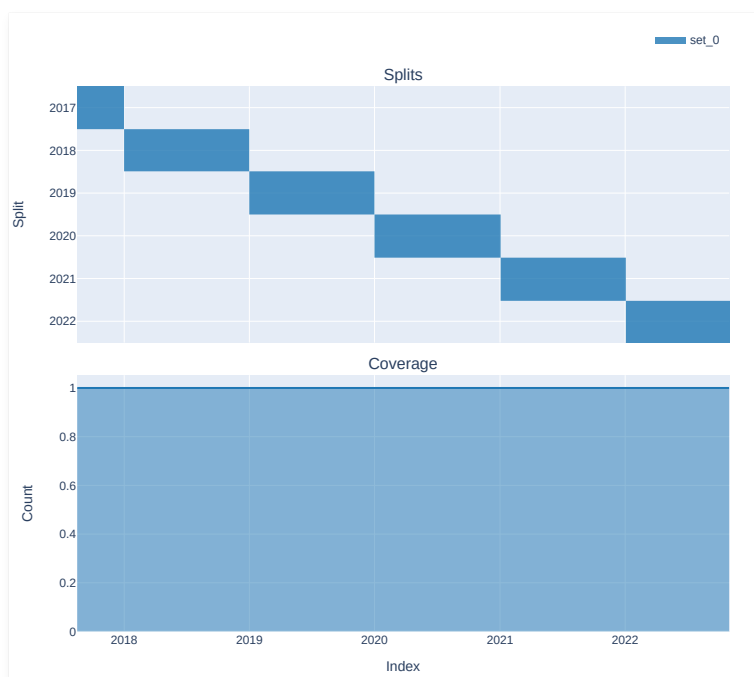

```
>>> splitter = vbt.Splitter.from_grouper(
...     data.index,
...     by=data.index.year.rename("split_year") ❶
... )
```

❶ Without renaming, the name will be derived from the index and become "Open time"

Since the class **Splitter** subclasses the class **Analyzable**, we can get a quick and nice overview of the most important metrics and plots:

```
>>> splitter.stats()
Index Start      2017-08-17 00:00:00+00:00
Index End        2022-10-31 00:00:00+00:00
Index Length      1902
Splits            6
Sets              1
Coverage [%]      100.0
Overlap Coverage [%] 0.0
Name: agg_stats, dtype: object

>>> splitter.plots().show()
```



Info

Since we have only one split, most metrics were hidden from the statistics.

As we already know, we can select specific splits and sets using regular Pandas indexing (which is another great property of **Analyzable**). Since we're not interested in incomplete years, let's remove the first and the last split:

```
>>> splitter = splitter.iloc[1:-1]
>>> splitter.stats()
Index Start      2017-08-17 00:00:00+00:00
Index End        2022-10-31 00:00:00+00:00
Index Length      1902
Splits            4 << changed
Sets              1
Coverage [%]      76.81388 << changed
Overlap Coverage [%] 0.0
Name: agg_stats, dtype: object
```

Now, let's split the only set into three: a train set covering the first two quarters, a validation set covering the third one, and a test set covering the last one. This is possible thanks to the method **Splitter.split_set**, which takes the split specification and the labels of the new split as `new_split` and `new_set_labels` respectively. We'll use a function template to divide the set:

```

>>> def new_split(index): ❶
...     return [
...         np.isin(index.quarter, [1, 2]), ❷
...         index.quarter == 3,
...         index.quarter == 4
...     ]

>>> splitter = splitter.split_set(
...     vbt.RepFunc(new_split), ❸
...     new_set_labels=["train", "valid", "test"] ❹
... )

```

- ❶ `index` will be recognized as an argument and substituted for the index covered by the original set. Why not the full index? Because we're splitting only a subset of it.
- ❷ `index.quarter` returns a NumPy array with quarters, one for each date
- ❸ We can use a template here because it's kindly accepted by [Splitter.split_range](#), which is used to split each range of the original set
- ❹ These labels should include only the labels for the new set

Info

Each operation on a splitter returns a new splitter: no information is changed in place to not mess up with caching and to keep the splitter (as any other vectorbt object) side effect free.

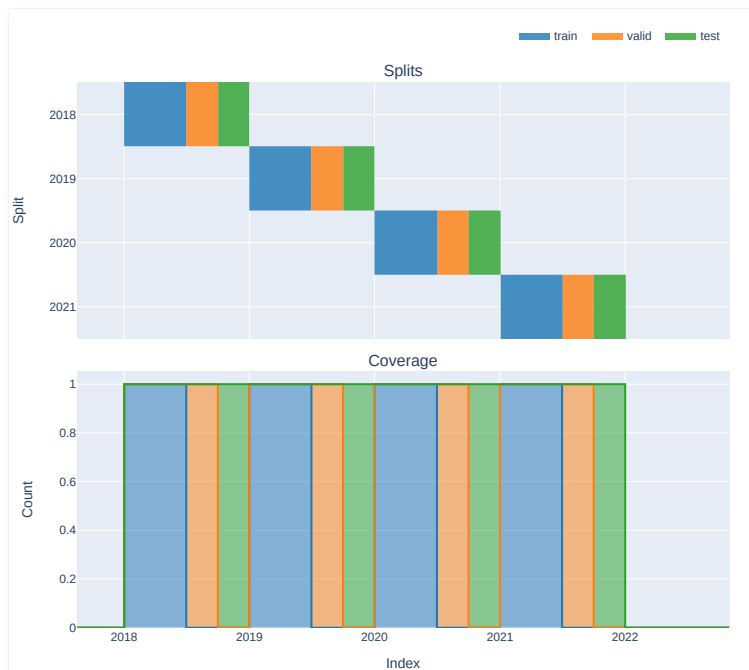
Let's take a look at the new splitter:

```

>>> splitter.stats()
Index Start          2017-08-17 00:00:00+00:00
Index End            2022-10-31 00:00:00+00:00
Index Length         1902
Splits                4
Sets                  3
Coverage [%]          76.81388
Coverage [%]: train    38.117771
Coverage [%]: valid    19.348055
Coverage [%]: test     19.348055
Mean Rel Coverage [%]: train 49.623475
Mean Rel Coverage [%]: valid 25.188263
Mean Rel Coverage [%]: test 25.188263
Overlap Coverage [%]   0.0
Overlap Coverage [%]: train 0.0
Overlap Coverage [%]: valid 0.0
Overlap Coverage [%]: test 0.0
Name: agg_stats, dtype: object

>>> splitter.plots().show()

```



As you might have guessed, there's also a method that can merge multiple sets - [Splitter.merge_sets](#). Your homework is to merge the "valid" and "test" sets into "test" 😊

We did our job perfectly, let's move on to applications! 🚀

[Python code](#)
[Notebook](#)

Tutorials Cross-validation

✓ Applications

We've got a splitter instance, what's next?

Remember that CV involves running a backtesting job on each range. Thanks to the ability of vectorbt to process two-dimensional data, we have two major ways of accomplishing that: either 1) split all arrays into chunks, merge them into two-dimensional arrays, and run one monolithic backtesting job, or 2) run a backtesting job on each chunk separately. We can also go a hybrid path: for example, build a two-dimensional array out of each set, and backtest it in isolation from other sets. Which approach to use depends on your requirements for RAM consumption and performance: two-dimensional arrays are faster to process, but there is a threshold of the number of stacked columns after which performance starts to downgrade (mostly because your system starts using swap memory).

Taking

Taking is a process of extracting one or more "slices" from an array-like object, which is implemented by the method `Splitter.take`. This method takes an object, iterates over each range in question, and takes the range from the object using `Splitter.take_range` - a dead-simple method that does `arr.iloc[range_]` on Pandas-like arrays and `arr[range_]` on NumPy arrays. Since many vectorbt classes inherit the indexing schema of Pandas, this method can even take a slice from vectorbt objects such as `Portfolio`! Once all slices have been collected, it can either stack them by rows (`stack_axis=0`) or columns (`stack_axis=1`, which is default), stack only splits and sets, or don't stack anything at all. When some slices shouldn't be stacked, they will be returned as a single Pandas Series that holds slices as values (may sound weird to have an array as a value of another array, but it's much easier for indexing than having a list).

Without stacking

Let's split the close price using default arguments:

```
>>> close_slices = splitter.take(data.close)
>>> close_slices
split_year  set
2018      train  Open time
2018-01-01 00:00:00+00:00  13380.0...
          test   Open time
2018-07-01 00:00:00+00:00  6356.81...
2019      train  Open time
2019-01-01 00:00:00+00:00  3797.1...
          test   Open time
2019-07-01 00:00:00+00:00  10624.9...
2020      train  Open time
2020-01-01 00:00:00+00:00  7200.85...
          test   Open time
2020-07-01 00:00:00+00:00  9232.0...
2021      train  Open time
2021-01-01 00:00:00+00:00  29331.6...
          test   Open time
2021-07-01 00:00:00+00:00  33504.6...
dtype: object
```

If you're wondering what this format is: a regular Pandas Series with the split and set labels as index and the close price slices as values - basically `pd.Series` within `pd.Series` 😊 Remember that array values can be any complex Python objects. For example, let's get the close price corresponding to the test set in 2020:

```
>>> close_slices[2020, "test"]
Open time
2020-07-01 00:00:00+00:00  9232.00
2020-07-02 00:00:00+00:00  9086.54
2020-07-03 00:00:00+00:00  9058.26
...
2020-12-29 00:00:00+00:00  27385.00
2020-12-30 00:00:00+00:00  28875.54
2020-12-31 00:00:00+00:00  28923.63
Freq: D, Name: Close, Length: 184, dtype: float64
```

Bingo!

And here's how simple is to apply a UDF on each range:

```
>>> def get_total_return(sr):
...     return sr.vbt.to_returns().vbt.returns.total()

>>> close_slices.apply(get_total_return) ❶
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test   2.132976
2021      train   0.194783
          test   0.379417
dtype: float64
```

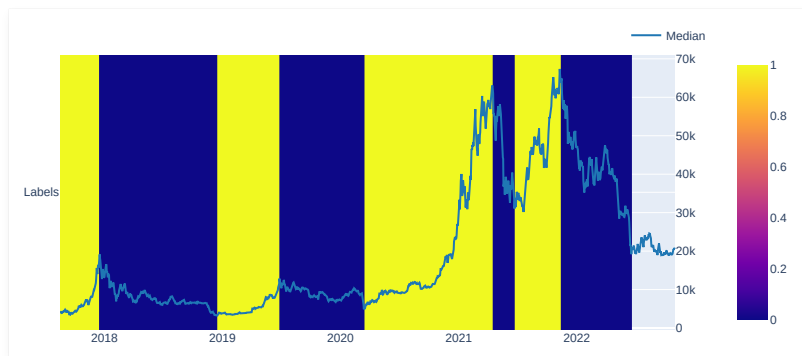
❶ Using `pandas.Series.apply`

Complex objects

One of the many unique features of vectorbt is the standardized behavior of its classes. Since this package is highly specialized on processing Pandas and NumPy arrays, most classes act as a proxy between the user and a set of such arrays; basically, each class enhances the feature set of Pandas and NumPy, and allows building connections between multiple arrays based on one common metadata stored in a [wrapper](#). Since such a wrapper can be sliced just like a regular Pandas array, we can slice most vectorbt objects that hold this wrapper as well, including [Splitter](#). And because we can slice them using the same indexing API as offered by Pandas arrays (thanks to [indexing](#)), we can pass most vectorbt objects directly to [Splitter.take](#)!

For example, let's analyze the performance of a portfolio during different market regimes. First, we'll use the forward-looking label generator [TRENDLB](#) to annotate each data point with either 1 (uptrend), 0 (downtrend), or NaN (cannot be classified). Given the volatility of our data, we'll register an uptrend once the price jumps by 100% from its previous low point, and a downtrend once the price falls by 50% from its previous high point:

```
>>> trendlb = data.run("trendlb", 1.0, 0.5)
>>> trendlb.plot().show()
```

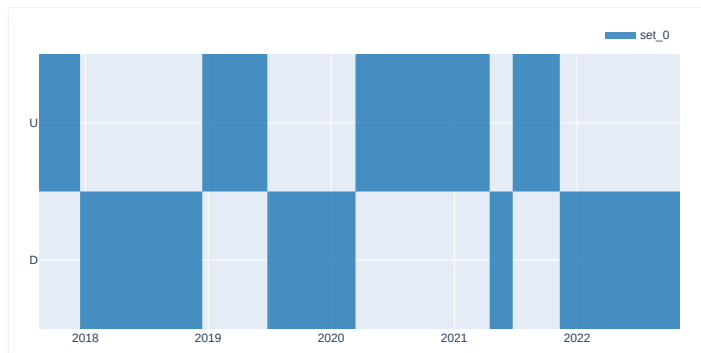


Hint

If you're not sure which pair of thresholds to use, look at the plot that the labeler produces and find the thresholds that work best for you.

Then, we'll build a splitter that converts the labels into splits:

```
>>> grouper = pd.Index(trendlb.labels.map({1: "U", 0: "D"}), name="trend")
>>> trend_splitter = vbt.Splitter.from_grouper(data.index, grouper)
>>> trend_splitter.plot().show()
```



In the next step, we'll run the full grid of parameter combinations on the entire period through column stacking, and extract the returns accessor of the type [ReturnsAccessor](#), which will enable us in analyzing the returns. To make a comparison, we'll also do the same on our baseline model.

```
>>> hold_pf = vbt.Portfolio.from_holding(data)
>>> hold_returns_acc = hold_pf.returns_acc

>>> fast_sma, slow_sma = vbt.talib("SMA").run_combs(
...     data.close, np.arange(5, 50), short_names=["fast_sma", "slow_sma"])
>>> entries = fast_sma.real_crossed_above(slow_sma)
>>> exits = fast_sma.real_crossed_below(slow_sma)
>>> strat_pf = vbt.Portfolio.from_signals(
...     data, entries, exits, direction="both")
>>> strat_returns_acc = strat_pf.returns_acc
```

Now, take both slices from the accessor (remember that most vectorbt objects are indexable, including accessors) and plot the Sharpe heatmap for each market regime:

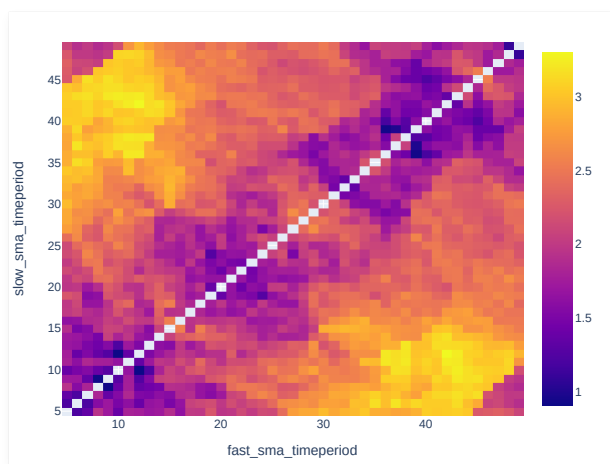
```
>>> hold_returns_acc_slices = trend_splitter.take(hold_returns_acc)
>>> strat_returns_acc_slices = trend_splitter.take(strat_returns_acc)
```

Uptrends

```
>>> hold_returns_acc_slices["U"].sharpe_ratio()
```

3.4490778178230763

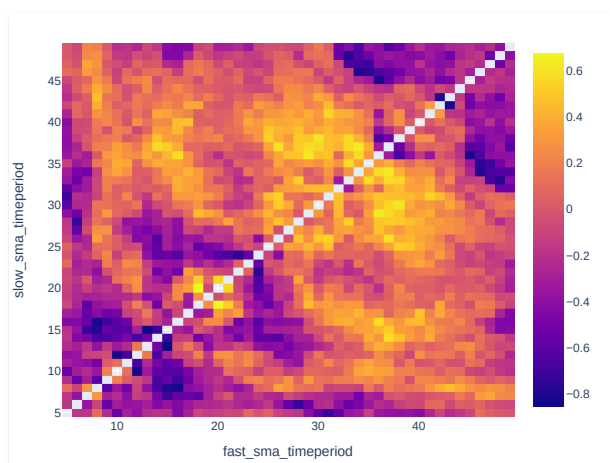
```
>>> strat_returns_acc_slices["U"].sharpe_ratio().vbt.heatmap(
...     x_level="fast_sma_timeperiod",
...     y_level="slow_sma_timeperiod",
...     symmetric=True
... ).show()
```



Downtrends

```
>>> hold_returns_acc_slices["D"].sharpe_ratio()
-1.329832516209626

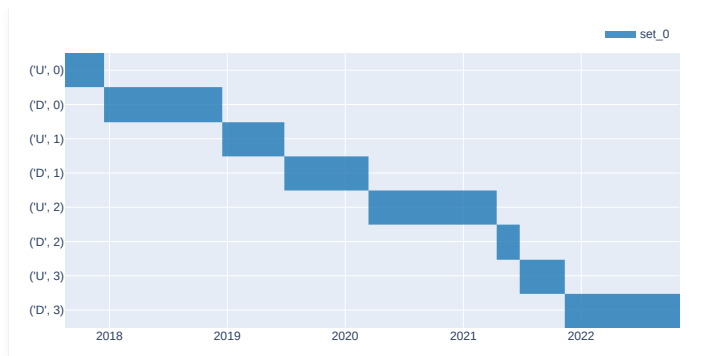
>>> strat_returns_acc_slices["D"].sharpe_ratio().vbt.heatmap(
...     x_level="fast_sma_timeperiod",
...     y_level="slow_sma_timeperiod",
...     symmetric=True
... ).show()
```



We can see that it takes a lot of effort (and some may say luck) to pick the right parameter combination and consistently beat the baseline. Since both pictures are completely different, we cannot rely on a single parameter combination; rather, we have to recognize the regime we're in and act accordingly, which is a massive challenge on its own. The above analysis should be used with caution though: there may be a position overflow from one market regime to another skewing the results since both periods are part of a single backtest. But at least we gained another bit of valuable information 🧐

But what if we try to take slices from the portfolio? The operation would fail because each split contains gaps, and portfolio cannot be indexed with gaps, only using non-interrupting ranges. Thus, we would need to break up each split into multiple smaller splits, also called "split parts". Luckily for us, there are two functionalities that make this possible: `Splitter.split_range` accepts an option "by_gap" as `new_split` to split a range by gap, while `Splitter.break_up_splits` can apply this operation on each split. This way, we can flatten the splits such that only one trend period is processed at once. We'll also sort the splits by their start index such that they come in the same temporal order as the labels:

```
>>> trend_splitter = trend_splitter.break_up_splits("by_gap", sort=True)
>>> trend_splitter.plot().show()
```



Another trick: instead of calling `Splitter.take`, we can call `split` on the object directly!

```
>>> strat_pf_slices = pf.split(trend_splitter)
>>> strat_pf_slices
trend split_part
U 0 Portfolio(\n wrapper=ArrayWrapper(\n ...
D 0 Portfolio(\n wrapper=ArrayWrapper(\n ...
U 1 Portfolio(\n wrapper=ArrayWrapper(\n ...
D 1 Portfolio(\n wrapper=ArrayWrapper(\n ...
U 2 Portfolio(\n wrapper=ArrayWrapper(\n ...
D 2 Portfolio(\n wrapper=ArrayWrapper(\n ...
U 3 Portfolio(\n wrapper=ArrayWrapper(\n ...
D 3 Portfolio(\n wrapper=ArrayWrapper(\n ...
dtype: object
```

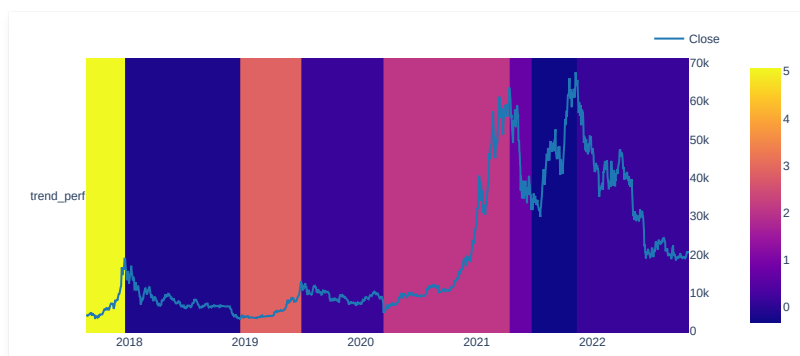
Let's analyze the median, that is, there are 50% of the parameter combinations with the same value or better, and, conversely, 50% with the same value or worse.

```
>>> trend_range_perf = strat_pf_slices.apply(lambda pf: pf.sharpe_ratio)
>>> median_trend_perf = trend_range_perf.median(axis=1)
>>> median_trend_perf
trend split_part
U 0 4.829429
D 0 -0.058159
U 1 2.874709
D 1 -0.510703
U 2 2.220633
D 2 0.917946
U 3 -0.736989
D 3 0.225841
dtype: float64
```

And now visually: replace the labels in `trend1b` by the freshly-computed values.

```
>>> trend_perf_ts = data.symbol_wrapper.fill().rename("trend_perf")
>>> for label, sr in trend_splitter.boundaries.iterrows():
...     trend_perf_ts.iloc[sr["start"]:sr["end"]] = median_trend_perf[label]
>>> data.close.vbt.overlay_with_heatmap(trend_perf_ts).show()
```

1 Using `GenericAccessor.overlay_with_heatmap`



Can you spot something strange? Right, at least 50% of the parameter combinations during the last uptrend have a negative Sharpe 😬 At least my explanation is that moving averages are lagging indicators and by the time they reacted to the previous sharp decline, we already arrived at the next top; basically, the rebound took them off-guard such that any short positions from the previous decline couldn't close out on time.

Column stacking

This was a glimpse into the second approach that we mentioned at the beginning of this page, which involves applying a UDF on each range separately. But how about stacking all the slices into a single one and applying our `get_total_return` just once for a nice performance boost? Turns out, we can stack the slices manually using `pandas.concat`:

```
>>> close_stacked = pd.concat(
```

```

...     close_slices.values.tolist(),
...     axis=1, ①
...     keys=close_slices.index ②
... )
>>> close_stacked
split_year
set          2018      2019      2020      2021
Open time
2018-01-01 00:00:00+00:00 13380.00 NaN      NaN      NaN      NaN      NaN
2018-01-02 00:00:00+00:00 14675.11 NaN      NaN      NaN      NaN      NaN
2018-01-03 00:00:00+00:00 14919.51 NaN      NaN      NaN      NaN      NaN
...
2021-12-29 00:00:00+00:00      NaN      NaN      NaN      NaN      NaN 46464.66
2021-12-30 00:00:00+00:00      NaN      NaN      NaN      NaN      NaN 47120.87
2021-12-31 00:00:00+00:00      NaN      NaN      NaN      NaN      NaN 46216.93

[1461 rows x 8 columns]

```

① Along columns

② Slice labels become column labels

As we can see, even though the operation produced a lot of NaNs, we now have a format that is perfectly acceptable by vectorbt. Let's apply our UDF to this array:

```

>>> get_total_return(close_stacked)
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test    2.132976
2021      train   0.194783
          test    0.379417
Name: total_return, dtype: float64

```

Pure magic ✨

But the stacking approach above works only if we're splitting Pandas objects, but what about NumPy arrays and complex vectorbt objects? We can instruct `Splitter.take` to do the stacking job for us. The method will use `column_stack_merge` for stacking any array-like objects in the most efficient way. For example, let's replicate the above format using a single line of code:

```

>>> close_stacked = splitter.take(data.close, into="stacked")
>>> close_stacked.shape
(1461, 8)

```

To get rid of many NaNs in the stacked data, we can reset the index of each slice prior to stacking by adding a prefix "reset_" to any "stacked" option provided as `into`:

```

>>> close_stacked = splitter.take(data.close, into="reset_stacked")
>>> close_stacked
split_year      2018      2019      2020      \
set      train  test  train  test  train  test
0      13380.00  6356.81  3797.14  10624.93  7200.85  9232.00
1      14675.11  6615.29  3858.56  10842.85  6965.71  9086.54
2      14919.51  6513.86  3766.78  11940.00  7344.96  9058.26
..      ...      ...      ...      ...      ...      ...
181      NaN    3695.32      NaN    7388.24  9138.55  27385.00
182      NaN    3801.91      NaN    7246.00      NaN  28875.54
183      NaN    3702.90      NaN    7195.23      NaN  28923.63

split_year      2021
set      train  test
0      29331.69  33504.69
1      32178.33  33786.55
2      33000.05  34669.13
..      ...      ...
181      NaN    46464.66
182      NaN    47120.87
183      NaN    46216.93

[184 rows x 8 columns]

```

We can also instruct the method to align each slice by the end point rather than the start point, which will push NaNs to the beginning of the array:

```

>>> close_stacked = splitter.take(data.close, into="from_end_stacked")
>>> close_stacked
split_year      2018      2019      2020      \
set      train  test  train  test  train  test
0      NaN    6356.81      NaN    10624.93      NaN  9232.00
1      NaN    6615.29      NaN    10842.85      NaN  9086.54
2      NaN    6513.86      NaN    11940.00  7200.85  9058.26
..      ...      ...      ...      ...      ...      ...
181    5853.98  3695.32  12400.63  7388.24  9116.35  27385.00
182    6197.92  3801.91  11903.13  7246.00  9192.56  28875.54
183    6390.07  3702.90  10854.10  7195.23  9138.55  28923.63

split_year      2021
set      train  test
0      NaN    33504.69
1      NaN    33786.55

```

```

2          NaN  34669.13
..          ...      ...
181       34494.89  46464.66
182       35911.73  47120.87
183       35045.00  46216.93

```

[184 rows x 8 columns]

Hint

If all slices have the same length, both alignments will produce the same array.

As we can see, there are two potential issues associated with this operation: the final array will have no datetime index, and the slices may have different lengths such that there will still be some NaNs present in the array. But also, we are rarely interested in stacking training and test sets together since they are bound to different pipelines, thus let's only stack the splits that belong to the same set, which will also produce slices of roughly the same length, by using "reset_stacked_splits" as `into`:

```

>>> close_stacked = splitter.take(data.close, into="reset_stacked_by_set")
>>> close_stacked
set
train  split_year    2018    2019    2020    2...
test   split_year    2018    2019    2020    2...
dtype: object

```

We've got some weird-looking format once again, but it's nothing more than a Series with the set labels as index and the stacked close price slices as values:

```

>>> close_stacked["train"]
split_year    2018    2019    2020    2021
0      13380.00  3797.14  7200.85  29331.69
1      14675.11  3858.56  6965.71  32178.33
2      14919.51  3766.78  7344.96  33000.05
..          ...      ...      ...      ...
179     6197.92  11903.13  9116.35  35911.73
180     6390.07  10854.10  9192.56  35045.00
181          NaN      NaN   9138.55      NaN

```

[182 rows x 4 columns]

By resetting the index, we save a tremendous amount of RAM: our arrays hold only $182 * 8 = 1456$ instead of $1461 * 8 = 11688$ (i.e., 88% less) values in memory. But how do we access the index associated with each column? We can slice it as a regular array!

```

>>> index_slices = splitter.take(data.index)
>>> index_slices
split_year  set
2018      train  DatetimeIndex(['2018-01-01 00:00:00+00:00', '2...
           test   DatetimeIndex(['2018-07-01 00:00:00+00:00', '2...
2019      train  DatetimeIndex(['2019-01-01 00:00:00+00:00', '2...
           test   DatetimeIndex(['2019-07-01 00:00:00+00:00', '2...
2020      train  DatetimeIndex(['2020-01-01 00:00:00+00:00', '2...
           test   DatetimeIndex(['2020-07-01 00:00:00+00:00', '2...
2021      train  DatetimeIndex(['2021-01-01 00:00:00+00:00', '2...
           test   DatetimeIndex(['2021-07-01 00:00:00+00:00', '2...
dtype: object

```

Another way of getting the index information as by attaching the bounds to the range labels using `attach_bounds`, which can be `True` and "index" to attach the bounds in the integer and datetime format respectively. Let's use the latter but also enable `right_inclusive` to make the right bound inclusive if we want to use it in indexing with `loc`:

```

>>> close_stacked_wb = splitter.take(
...     data.close,
...     into="reset_stacked_by_set",
...     attach_bounds="index",
...     right_inclusive=True
... )
>>> close_stacked_wb["train"]
split_year    2018    2019 \
start  2018-01-01 00:00:00+00:00  2019-01-01 00:00:00+00:00
end    2018-06-30 00:00:00+00:00  2019-06-30 00:00:00+00:00
0      13380.00      3797.14
1      14675.11      3858.56
2      14919.51      3766.78
..          ...      ...
179     6197.92     11903.13
180     6390.07     10854.10
181          NaN          NaN

split_year    2020    2021
start  2020-01-01 00:00:00+00:00  2021-01-01 00:00:00+00:00
end    2020-06-30 00:00:00+00:00  2021-06-30 00:00:00+00:00
0      7200.85      29331.69
1      6965.71      32178.33
2      7344.96      33000.05
..          ...      ...
179     9116.35      35911.73
180     9192.56      35045.00
181     9138.55          NaN

```

[182 rows x 4 columns]

Though, to keep the index clean, we will go with the arrays without bounds first. We've established two arrays: one for training and another one for testing purposes. Let's modify our pipeline `sma_crossover_perf` from the first page to be run on a single set: replace the argument `data` with `close`, and add another argument for the frequency required by Sharpe since our index isn't datetime-like anymore.

```
>>> @vbt.parameterized(merge_func="concat")
... def set_sma_crossover_perf(close, fast_window, slow_window, freq):
...     fast_sma = vbt.talib("sma").run(
...         close, fast_window, short_name="fast_sma", hide_params=True) ❶
...     slow_sma = vbt.talib("sma").run(
...         close, slow_window, short_name="slow_sma", hide_params=True)
...     entries = fast_sma.real_crossed_above(slow_sma)
...     exits = fast_sma.real_crossed_below(slow_sma)
...     pf = vbt.Portfolio.from_signals(
...         close, entries, exits, freq=freq, direction="both")
...     return pf.sharpe_ratio
```

❶ Do not add parameter to the index as it will be added by the `parameterized` decorator

Apply it on the training set to get the performance per parameter combination and split:

```
>>> train_perf = set_sma_crossover_perf(
...     close_stacked["train"],
...     vbt.Param(np.arange(5, 50), condition="x < slow_window"),
...     vbt.Param(np.arange(5, 50)),
...     data.index.freq,
...     _execute_kwargs=dict(
...         clear_cache=50,
...         collect_garbage=50
...     )
... )
```

Combination 990/990

```
>>> train_perf
fast_window  slow_window  split_year
5            6            2018         1.158471
                    2019         1.901410
                    2020        -0.426441
                    2021         0.052654
                    2018         2.231909
                    ....
47           49            2021        -0.446099
48           49            2018        -1.169584
                    2019         3.727154
                    2020        -1.913321
                    2021        -0.400270
Name: sharpe_ratio, Length: 3960, dtype: float64
```

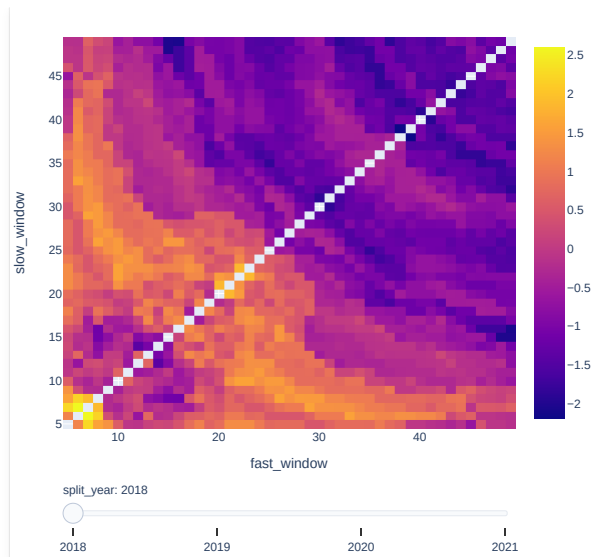
A total of 990 parameter combinations were run in 20 seconds, or 20ms per run 🚀

Let's generate the performance heatmap for each split:

```
>>> train_perf.vbt.heatmap(
...     x_level="fast_window",
...     y_level="slow_window",
...     slider_level="split_year",
...     symmetric=True
... ).show()
```

Dashboard

Run the notebook to view the dashboard!



By looking at the plot above, we can identify a number of yellow points that may be good candidates for our strategy. But, in contrast to the approach from the first page where we selected the best parameter combination, let's conduct a neighborhood analysis by finding a Sharpe ratio that's surrounded by high Sharpe ratios, so that we can diminish the effect of outliers and bring in more robustness into our optimizer.

This query can be well quantified by transforming the `train_perf` Series into a DataFrame using the accessor method `BaseAccessor.unstack_to_df` and running the accessor method `GenericAccessor.proximity_apply`, which rolls a two-dimensional window over the entire matrix and reduces this window using a UDF. This mini-pipeline must be applied separately to each split. We will use a window of 2, that is, the two rows and columns that surround the central point, which yields a total of $(n * 2 + 1) ** 2 = 25$ values. As a UDF we'll take the median, that is, each value will mean that 50% or more of the surrounding values are better than it. Our UDF will also filter out all the windows that have less than 20 valid values.

```
>>> @njit
... def prox_median_nb(arr): ❶
...     if (~np.isnan(arr)).sum() < 20:
...         return np.nan
...     return np.nanmedian(arr) ❷

>>> prox_perf_list = []
>>> for split_label, perf_sr in train_perf.groupby("split_year"): ❸
...     perf_df = perf_sr.vbt.unstack_to_df(0, [1, 2]) ❹
...     prox_perf_df = perf_df.vbt.proximity_apply(2, prox_median_nb) ❺
...     prox_perf_sr = prox_perf_df.stack([0, 1]) ❻
...     prox_perf_list.append(prox_perf_sr.reindex(perf_sr.index)) ❼

>>> train_prox_perf = pd.concat(prox_perf_list) ❽
>>> train_prox_perf
fast_window  slow_window  split_year
5            6           2018      NaN
              6           2019      NaN
              6           2020      NaN
              6           2021      NaN
              7           2018      NaN
              ...
47           49           2021      NaN
48           49           2018      NaN
              49           2019      NaN
              49           2020      NaN
              49           2021      NaN
Length: 3960, dtype: float64
```

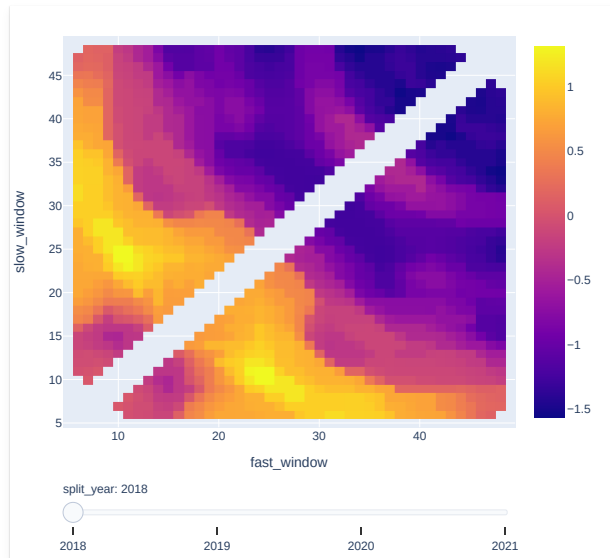
- ❶ This function will take the median out of the proximity windows. Argument `arr` is a two-dimensional window flattened to one dimension.
- ❷ Since our window still may contain NaN values, use the NaN-resistant median
- ❸ Run the following per split
- ❹ Make a DataFrame with one level as rows, and two other levels as columns. We could have also used level names.
- ❺ Run our smoothing window over the entire DataFrame
- ❻ Make a Series by stacking all columns. Again, we could have also used level names.
- ❼ Reindex the Series to match the original index and append to list
- ❽ Stack all Series in the list into one big Series

At first, it may appear that all values are NaN, but let's prove otherwise:

```
>>> train_prox_perf.vbt.heatmap(
...     x_level="fast_window",
...     y_level="slow_window",
...     slider_level="split_year",
...     symmetric=True
... ).show()
```

Dashboard

Run the notebook to view the dashboard!



Fans of [computer vision](#) will recognize what we did above: we used a 5x5 neighboring window acting as a [filter](#) that replaces each pixel with the median pixel value of it and a neighborhood window of adjacent pixels. The effect is a more smooth image with sharp features removed (in our case Sharpe outliers). For example, in the first split, the highest Sharpe fell from 2.5 to 1.3, which is still extraordinary since it means that there are points that have at least 50% of surrounding points with a Sharpe 1.3 or higher! We can now search in each split for the parameter combination that has the highest proximity performance:

```
>>> best_params = train_prox_perf.groupby("split_year").idxmax()
>>> best_params = train_prox_perf[best_params].index
>>> train_prox_perf[best_params]
fast_window  slow_window  split_year
10           24          2018        1.311910
9            39          2019        3.801643
14           19          2020        2.077684
31           41          2021        2.142695
dtype: float64
```

What we're waiting for? Let's test those combinations on our test set! But wait, how do we apply each parameter combination on each column in the test array? Isn't each parameter combination being applied on the entire input? That's right, but there's a trick: use templates to instruct the `parameterized` decorator to pass only one column at a time, depending on the parameter combination being processed.

```
>>> test_perf = set_sma_crossover_perf(
...     vbt.RepEval(
...         "test_close.iloc[:, [config_idx]]", ❶
...         context=dict(test_close=close_stacked["test"])
...     ),
...     vbt.Param(best_params.get_level_values("fast_window"), level=0), ❷
...     vbt.Param(best_params.get_level_values("slow_window"), level=0),
...     data.index.freq
... )
>>> test_perf
fast_window  slow_window  split_year
10           24          2018        -0.616204
9            39          2019         0.017269
14           19          2020         4.768589
31           41          2021        -0.363900
Name: sharpe_ratio, dtype: float64
```

- ❶ This template will be substituted during processing of each parameter combination (also called a parameter config). Here, we select the column from our test data that has the same index as the parameter config. We wrap the index with a list to run the pipeline on a DataFrame with meaningful columns.
- ❷ Extract the parameter values and assign both `fast_window` and `slow_window` the same level to avoid building a product between them

Let's compare these values against the baseline:

```
>>> def get_index_sharpe(index):
...     return data.loc[index].run("from_holding").sharpe_ratio
>>> index_slices.xs("test", level="set").apply(get_index_sharpe) ❶
```

```
split_year
2018    -1.327655
2019    -0.788038
2020     4.425057
2021     1.304871
dtype: float64
```

- ① Each value in `index_slices` is an index, so we simply apply `get_index_sharpe` on it!

Not bad! Our model beats the baseline for three years in a row.

Row stacking

The same way as we stacked ranges along columns, we can stack ranges along rows. The major difference between both approaches is that column stacking is meant for producing independent tests while row stacking puts ranges into the same test and thus introduces a temporal dependency between them.

What row stacking is perfect for is block resampling required for time-series bootstrapping. The bootstrap is a flexible and powerful statistical tool that can be used to quantify the uncertainty. Rather than repeatedly obtaining independent datasets (which are limited in finance), we instead obtain distinct datasets by repeatedly sampling observations from the original dataset. Each of these "bootstrap datasets" is created by sampling *with replacement*, and is the same size as our original dataset. As a result some observations may appear more than once and some not at all (about two-thirds of the original data points appear in each bootstrap sample). Since we're working on time series, we can't simply sample the observations with replacement; rather, we should create blocks of consecutive observations, and sample those. Then we paste together sampled blocks to obtain a bootstrap dataset (learn more [here](#)).

For our example, we'll use the **moving block bootstrap**, which involves rolling a fixed-size window with an offset of just one bar:

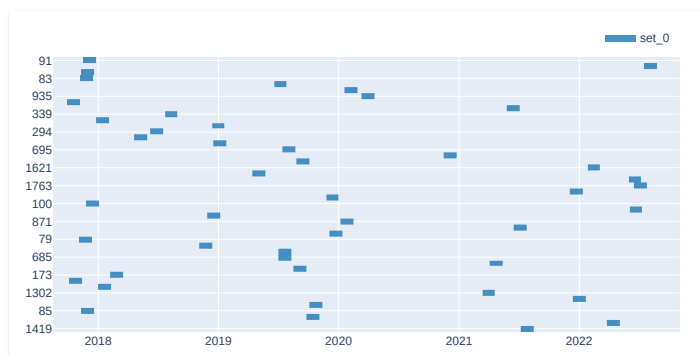
```
>>> block_size = int(3.15 * len(data.index) ** (1 / 3)) ①
>>> block_size
39

>>> block_splitter = vbt.Splitter.from_rolling(
...     data.index,
...     length=block_size,
...     offset=1,
...     offset_anchor="prev_start" ②
... )
>>> block_splitter.n_splits
1864
```

- ① Default block length used in the R package
- ② By default, the anchor is the right bound of the previous window

We've generated 1864 blocks. The next step is sampling. To generate a single sample, shuffle the blocks (i.e., splits) with replacement, which can be easily done using `Splitter.shuffle_splits`. We also need to limit the number of blocks such that they have roughly the same number of data points as our original data:

```
>>> size = int(block_splitter.n_splits / block_size)
>>> sample_splitter = block_splitter.shuffle_splits(size=size, replace=True)
>>> sample_splitter.plot().show()
```



Let's compute the returns, "take" the slices corresponding to the blocks in the splitter, and stack them along rows:

```
>>> returns = data.returns
>>> sample_rets = sample_splitter.take(
...     returns,
...     into="stacked", ①
...     stack_axis=0 ②
... )
>>> sample_rets
split  Open time
920    2020-02-23 00:00:00+00:00    0.029587
      2020-02-24 00:00:00+00:00   -0.028206
      2020-02-25 00:00:00+00:00   -0.035241
      2020-02-26 00:00:00+00:00   -0.056956
      2020-02-27 00:00:00+00:00    0.004321
      ...
1617   2022-02-23 00:00:00+00:00   -0.025642
      2022-02-24 00:00:00+00:00    0.028918
      2022-02-25 00:00:00+00:00    0.023272
      2022-02-26 00:00:00+00:00   -0.002612
```

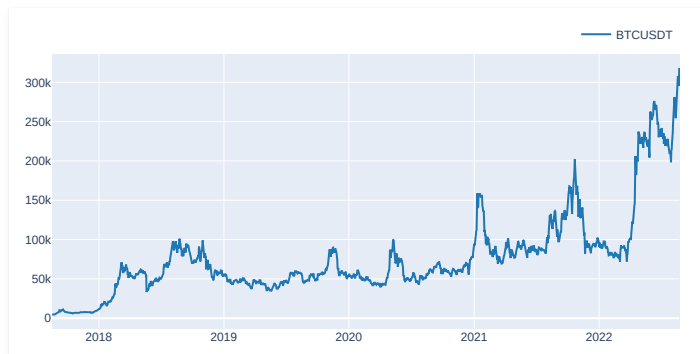
```
2022-02-27 00:00:00+00:00 -0.036242
Name: Close, Length: 1833, dtype: float64
```

- 1 Stack all slices into one Series
- 2 Along rows

We've created a "frankenstein" price series!

```
>>> sample_rets.index = data.index[:len(sample_rets)] 1
>>> sample_cumrets = data.close[0] * (sample_rets + 1).cumprod() 2
>>> sample_cumrets.vbt.plot().show()
```

- 1 Plotting requires a proper index
- 2 Compute the cumulative returns and project them from the start of our data



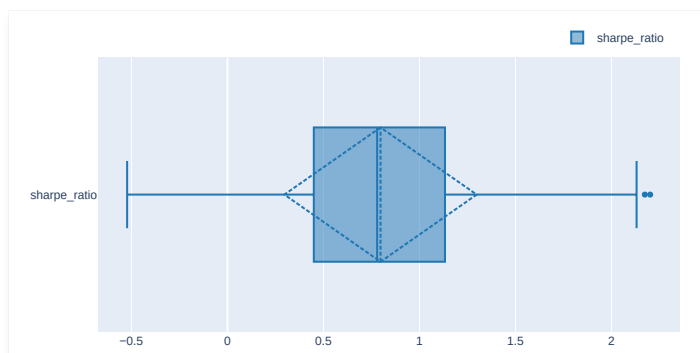
But one sample is not enough: we need to generate 100, 1000, or even 10000 samples for our estimates to be as accurate as possible.

```
>>> samples_rets_list = []
>>> for i in vbt.ProgressBar(range(1000)):
...     sample_spl = block_splitter.shuffle_splits(size=size, replace=True)
...     sample_rets = sample_spl.take(returns, into="stacked", stack_axis=0)
...     sample_rets.index = returns.index[:len(sample_rets)]
...     sample_rets.name = i
...     samples_rets_list.append(sample_rets)
>>> sample_rets_stacked = pd.concat(samples_rets_list, axis=1)
```

Sample 1000/1000

We can then analyze the distribution of a statistic of interest:

```
>>> sample_sharpe = sample_rets_stacked.vbt.returns.sharpe_ratio()
>>> sample_sharpe.vbt.boxplot(horizontal=True).show()
```



This histogram provides an estimate of the shape of the distribution of the sample Sharpe from which we can answer questions about how much the Sharpe varies across samples. Since this particular bootstrap distribution is symmetric, we can use percentile-based confidence intervals. For example, a 95% confidence interval translates to the 2.5th and 97.5th percentiles:

```
>>> sample_sharpe.quantile(0.025), sample_sharpe.quantile(0.975)
(-0.13636235254958026, 1.726050620753774)
```

The method here can be applied to almost any other statistic or estimator, including more complex backtesting metrics.

Applying

If the "taking" approach provides us with object slices that we can work upon, the "applying" approach using `Splitter.apply` runs a UDF on each range, and can not only do the "taking" part for us, but also easily merge the outputs of the UDF. First, it resolves the ranges it needs to iterate over; similarly to other methods, it takes

`split_group_by` and `set_group_by` to merge ranges, but also the arguments `split` and `set_` that can be used to select specific (merged) ranges. Then, while iterating over each range, it substitutes any templates and other instructions in the positional and keyword arguments meant to be passed to the UDF. For example, by wrapping any argument with the class `Takeable`, the method will select a slice from it and substitute the instruction with that slice. The arguments prepared in each iteration are saved in a list and passed to the executor - `execute`, which you're probably already familiar with. The executor lazily executes all the iterations and (optionally) merges the outputs. *Lazily* here means that none of the arrays will be sliced until the range is executed - good for memory ❤️

Let's run a simple example of calculating the total return of each close slice:

```
>>> splitter.apply(
...     get_total_return,
...     vbt.Takeable(data.close), ❶
...     merge_func="concat" ❷
... )
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test    2.132976
2021      train   0.194783
          test    0.379417
dtype: float64
```

❶ Any positional arguments passed after the function will be passed to the function. We could have also passed it as a keyword argument

```
sr=vbt.Takeable(data.close).
```

❷ See [resolve_merge_func](#)

That's how easy it is!

We could have also used templates:

```
>>> splitter.apply(
...     get_total_return,
...     vbt.RepFunc(lambda range_: data.close[range_]), ❶
...     merge_func="concat"
... )
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test    2.132976
2021      train   0.194783
          test    0.379417
dtype: float64
```

❶ `range_` will be substituted with the current range

Or, by manually selecting the range inside the function:

```
>>> def get_total_return(range_, data):
...     return data.returns[range_].vbt.returns.total()

>>> splitter.apply(
...     get_total_return,
...     vbt.Rep("range_"),
...     data,
...     merge_func="concat"
... )
split_year  set
2018      train  -0.534128
          test   -0.420523
2019      train   1.931243
          test   -0.337096
2020      train   0.270084
          test    2.165013
2021      train   0.211639
          test    0.318788
dtype: float64
```

Hint

Results are slightly different because slicing returns is more accurate than slicing the price.

Taking from complex vectorbt objects works too:

```
>>> def get_total_return(data):
...     return data.returns.vbt.returns.total()

>>> splitter.apply(
...     get_total_return,
...     vbt.Takeable(data), ❶
...     merge_func="concat"
... )
split_year  set
2018      train  -0.522416
```

```

2019    test    -0.417491
        train    1.858493
2020    test    -0.322797
        train    0.269093
2021    test    2.132976
        train    0.194783
        test    0.379417
dtype: float64

```

1 Splits the `Data` instance

Let's run the function above on the entire split:

```

>>> splitter.apply(
...     get_total_return,
...     vbt.Takeable(data),
...     set_group_by=True,
...     merge_func="concat"
... )
split_year
2018    -0.723251
2019     0.894908
2020     3.016697
2021     0.575665
dtype: float64

```

If we need to select specific ranges to run the pipeline upon, use `split` and `set_`, which can be an integer, a label, or a sequence of such:

```

>>> splitter.apply(
...     get_total_return,
...     vbt.Takeable(data),
...     split=[2020, 2021],
...     set_="train",
...     merge_func="concat"
... )
split_year
2020     0.270084
2021     0.211639
dtype: float64

```

Let's apply this approach to cross-validate our SMA crossover strategy:

```

>>> train_perf = splitter.apply(
...     sma_crossover_perf,
...     vbt.Takeable(data),
...     vbt.Param(np.arange(5, 50), condition="x < slow_window"), ①
...     vbt.Param(np.arange(5, 50)),
...     _execute_kwargs=dict( ②
...         clear_cache=50,
...         collect_garbage=50
...     ),
...     set_="train", ③
...     merge_func="concat"
... )

```

① Parameters are processed in `sma_crossover_perf`, not in `splitter.apply`

② This dictionary is also getting passed to `sma_crossover_perf`. We'll hide the progress bar for parameter combinations and display the one for splits below.

③ We could have also selected this set using `splitter.loc["train"]`

Split 4/4

```

>>> train_perf
split_year  fast_window  slow_window
2018        5           6           1.161661
              7           2.238117
              8           1.583246
              9           1.369400
             10          0.563251
              ...
2021       46          48          -0.238667
              49          -0.437822
              47          48          -0.799317
              49          -0.447324
              48          49          -0.401369
Length: 3960, dtype: float64

```

Let's find the best parameter combinations and pass them to the test set using templates:

```

>>> best_params = train_perf.groupby("split_year").idxmax()
>>> best_params = train_perf[best_params].index
>>> train_perf[best_params]
split_year  fast_window  slow_window
2018        6           7           2.612006
2019       35          36           4.246530
2020       16          17           2.605763
2021       32          45           3.073246
dtype: float64

>>> best_fast_windows = best_params.get_level_values("fast_window")

```

```

>>> best_slow_windows = best_params.get_level_values("slow_window")

>>> test_perf = splitter.apply(
...     sma_crossover_perf,
...     vbt.Takeable(data),
...     vbt.RepFunc(lambda split_idx: best_fast_windows[split_idx]), ❶
...     vbt.RepFunc(lambda split_idx: best_slow_windows[split_idx]),
...     set_="test",
...     merge_func="concat"
... )
>>> test_perf
split_year
2018    1.501380
2019    0.152723
2020    4.473387
2021   -1.147178
dtype: float64

```

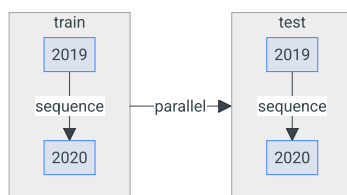
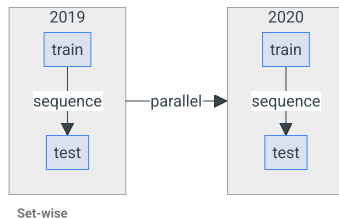
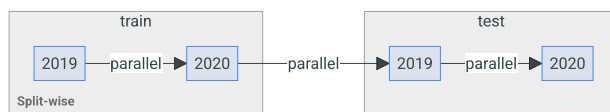
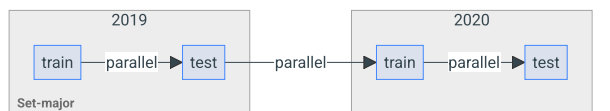
❶ `split_idx` will be substituted by the split number. Make sure that the values in `best_fast_windows` and `best_slow_windows` appear in the same order as the splits in the splitter. Or, use `split_label` if the order is different.

We've effectively outsourced the range iteration, taking, and execution steps.

Iteration schemes

The method `Splitter.apply` supports multiple iteration schemes. For instance, it can iterate over the selected ranges in the split-major or set-major order as a single sequence. It can also pack the splits/sets of the same set/split into the same bucket and run as a single iteration - split-wise and set-wise respectively. Why does it matter? Consider a scenario where a UDF needs to write the results to an object and then access these results in a specific order. For example, we may want a OOS range to read the results of the previous IS range, which is a requirement of parameterized CV; in such a case, the iteration can be in any order if the execution is sequential, in any major order if it (carefully) spans across multiple threads, and in the split-wise order if the execution spans across multiple processes since the sets need to share the same memory.

Split-major



Let's cross-validate our SMA crossover strategy using a single call! When an IS range is processed, find out the parameter combination with the highest Sharpe ratio and return it. When an OOS range is processed, access the parameter combination from the previous IS range and use it for testing. If you take a closer look at the available context at each iteration, there is no variable that holds the previous results: we need to store, write, and access the results manually. We'll iterate in a set-major order such that the splits in the training set are processed first.

```

>>> def cv_sma_crossover(
...     data,
...     fast_windows,
...     slow_windows,
...     split_idx, ❶
...     set_idx, ❷
...     train_perf_list ❸
... ):
...     if set_idx == 0: ❹
...         train_perf = sma_crossover_perf(
...             data,

```



```

...     vbt.Param(fast_windows, condition="x < slow_window"),
...     vbt.Param(slow_windows),
...     _execute_kwargs=dict(
...         clear_cache=50,
...         collect_garbage=50
...     )
... )
... train_perf_list.append(train_perf) ❸
... best_params = train_perf.idxmax()
... return train_perf[[best_params]] ❹
... else:
...     train_perf = train_perf_list[split_idx] ❺
...     best_params = train_perf.idxmax()
...     test_perf = sma_crossover_perf(
...         data,
...         vbt.Param([best_params[0]]), ❻
...         vbt.Param([best_params[1]]),
...     )
...     return test_perf ❼

```

```

>>> train_perf_list = []
>>> cv_perf = splitter.apply(
...     cv_sma_crossover,
...     vbt.Takeable(data),
...     np.arange(5, 50),
...     np.arange(5, 50),
...     vbt.Rep("split_idx"),
...     vbt.Rep("set_idx"),
...     train_perf_list,
...     iteration="set_major",
...     merge_func="concat"
... )

```

- ❶ Substituted by the current split number
- ❷ Substituted by the current set number
- ❸ List with the results from the training set
- ❹ If we're currently in the training set, run the optimization
- ❺ Append the results to the list
- ❻ Return the best result, which will be merged with the test results. Wrap with a list to return a Series rather than a single value (we want a nice index).
- ❼ Access the training results of the current split and test the best result
- ❽ Wrap with `Param` for `sma_crossover_perf` to return a Series rather than a single value (we want a nice index)
- ❼ Return the test result

Iteration 8/8

We've got 8 iterations, one per range: the first 4 iterations correspond to the training ranges, which took the most time to execute, while the last 4 iterations correspond to the test ranges, which executed almost instantly because of just one parameter combination. Let's concatenate the training results from `train_perf_dict` and see what's inside `cv_perf`:

```

>>> train_perf = pd.concat(train_perf_list, keys=splitter.split_labels)
>>> train_perf

```

split_year	fast_window	slow_window	
2018	5	6	1.161661
		7	2.238117
		8	1.583246
		9	1.369400
		10	0.563251
			...
2021	46	48	-0.238667
		49	-0.437822
		48	-0.799317
		49	-0.447324
		48	-0.401369

Length: 3960, dtype: float64

```

>>> cv_perf

```

set	split_year	fast_window	slow_window	
train	2018	6	7	2.612006
	2019	35	36	4.246530
	2020	16	17	2.605763
	2021	32	45	3.073246
test	2018	6	7	1.501380
	2019	35	36	0.152723
	2020	16	17	4.473387
	2021	32	45	-1.147178

dtype: float64

Same results, awesome!

Merging

Another great feature of this method is that it can merge arbitrary outputs: from single values and Series, to DataFrames and even tuples of such. There are two main merging options available: merging all outputs into a single object (`merge_all=True`) and merging by the main unit of iteration (`merge_all=False`). The first option does the following: it flattens all outputs into a single sequence, resolves the merging function using `resolve_merge_func`, and calls the merging function

on that sequence. If the merging function is not specified, it wraps the sequence into a Pandas Series (even if each output is a complex object). If each output is a tuple, returns multiple of such Series. Let's illustrate this by returning entries and exits, and stacking them along columns:

```
>>> def get_entries_and_exits(data, fast_window, slow_window):
...     fast_sma = data.run("sma", fast_window, short_name="fast_sma")
...     slow_sma = data.run("sma", slow_window, short_name="slow_sma")
...     entries = fast_sma.real_crossed_above(slow_sma)
...     exits = fast_sma.real_crossed_below(slow_sma)
...     return entries, exits

>>> entries, exits = splitter.apply(
...     get_entries_and_exits,
...     vbt.Takeable(data),
...     20,
...     30,
...     merge_func="column_stack"
... )

>>> entries
split_year
set
2018
2019
2020
2021
train test train test train test train test
Open time
2018-01-01 00:00:00+00:00 False NaN NaN NaN NaN NaN NaN NaN
2018-01-02 00:00:00+00:00 False NaN NaN NaN NaN NaN NaN NaN
2018-01-03 00:00:00+00:00 False NaN NaN NaN NaN NaN NaN NaN
...
2021-12-29 00:00:00+00:00 NaN NaN NaN NaN NaN NaN NaN False
2021-12-30 00:00:00+00:00 NaN NaN NaN NaN NaN NaN NaN False
2021-12-31 00:00:00+00:00 NaN NaN NaN NaN NaN NaN NaN False

[1461 rows x 8 columns]
```

We can then replace NaN with `False` and backtest them. If you don't want to have that many NaNs, use `"reset_column_stack"` as `merge_func`. We can also provide multiple merging functions (as a tuple) in case the outputs of our UDF have different formats.

Note

Even though you can return multiple different formats, the formats must remain the same across all ranges!

As said previously, we can also merge outputs by the main unit of iteration. Let's run the same UDF but only stack the masks that belong to the same split:

```
>>> entries, exits = splitter.apply(
...     get_entries_and_exits,
...     vbt.Takeable(data),
...     50,
...     200,
...     merge_all=False,
...     merge_func="row_stack"
... )

>>> entries.loc[2018]
set    Open time
train 2018-01-01 00:00:00+00:00 False
      2018-01-02 00:00:00+00:00 False
      2018-01-03 00:00:00+00:00 False
      2018-01-04 00:00:00+00:00 False
      2018-01-05 00:00:00+00:00 False
      ...
test  2018-12-27 00:00:00+00:00 False
      2018-12-28 00:00:00+00:00 False
      2018-12-29 00:00:00+00:00 False
      2018-12-30 00:00:00+00:00 False
      2018-12-31 00:00:00+00:00 True
Length: 365, dtype: bool
```

This way, each mask covers an entire year and can be backtested as a whole. The additional level `set` provides us with the information on which set each timestamp belongs to.

In a case where a range's start and end date cannot be inferred from the merged data alone, we can instruct the method to attach this information. Let's get the total number of signals:

```
>>> def get_signal_count(*args, **kwargs):
...     entries, exits = get_entries_and_exits(*args, **kwargs)
...     return entries.vbt.signals.total(), exits.vbt.signals.total()

>>> entry_count, exit_count = splitter.apply(
...     get_signal_count,
...     vbt.Takeable(data),
...     20,
...     30,
...     merge_func="concat",
...     attach_bounds="index" ①
... )

>>> entry_count
split_year  set    start    end
2018        train 2018-01-01 00:00:00+00:00 2018-07-01 00:00:00+00:00 2
          test  2018-07-01 00:00:00+00:00 2019-01-01 00:00:00+00:00 4
2019        train 2019-01-01 00:00:00+00:00 2019-07-01 00:00:00+00:00 2
          test  2019-07-01 00:00:00+00:00 2020-01-01 00:00:00+00:00 3
2020        train 2020-01-01 00:00:00+00:00 2020-07-01 00:00:00+00:00 1
          test  2020-07-01 00:00:00+00:00 2021-01-01 00:00:00+00:00 2
```

```

2021      train  2021-01-01 00:00:00+00:00  2021-07-01 00:00:00+00:00  4
      test  2021-07-01 00:00:00+00:00  2022-01-01 00:00:00+00:00  1
dtype: int64

```

1 Calculate the integer bounds, map them to `data.index`, and attach them to the output's index

Finally, to demonstrate the power of merging functions, let's create our own merging function that plots the returned signals depending on the set!

```

>>> def plot_entries_and_exits(results, data, keys):
...     set_labels = keys.get_level_values("set") 1
...     fig = data.plot(plot_volume=False) 2
...     train_seen = False
...     test_seen = False
...
...     for i in range(len(results)): 3
...         entries, exits = results[i] 4
...         set_label = set_labels[i]
...         if set_label == "train":
...             entries.vbt.signals.plot_as_entries( 5
...                 data.close,
...                 trace_kwargs=dict( 6
...                     marker=dict(color="limegreen"),
...                     name=f"Entries ({set_label})",
...                     legendgroup=f"Entries ({set_label})", 7
...                     showlegend=not train_seen 8
...                 ),
...                 fig=fig
...             ),
...             exits.vbt.signals.plot_as_exits(
...                 data.close,
...                 trace_kwargs=dict(
...                     marker=dict(color="orange"),
...                     name=f"Exits ({set_label})",
...                     legendgroup=f"Exits ({set_label})",
...                     showlegend=not train_seen
...                 ),
...                 fig=fig
...             )
...             train_seen = True
...         else:
...             entries.vbt.signals.plot_as_entries(
...                 data.close,
...                 trace_kwargs=dict(
...                     marker=dict(color="skyblue"),
...                     name=f"Entries ({set_label})",
...                     legendgroup=f"Entries ({set_label})",
...                     showlegend=not test_seen
...                 ),
...                 fig=fig
...             ),
...             exits.vbt.signals.plot_as_exits(
...                 data.close,
...                 trace_kwargs=dict(
...                     marker=dict(color="magenta"),
...                     name=f"Exits ({set_label})",
...                     legendgroup=f"Exits ({set_label})",
...                     showlegend=not test_seen
...                 ),
...                 fig=fig
...             )
...             test_seen = True
...     return fig 9

>>> splitter.apply(
...     get_entries_and_exits,
...     vbt.Takeable(data),
...     20,
...     30,
...     merge_func=plot_entries_and_exits,
...     merge_kwargs=dict(data=data, keys=vbt.Rep("keys")),
... ).show()

```

- 1 `keys` is a multi-index for `results` that contains split labels, set labels, and (optionally) bounds
- 2 Plot OHLC
- 3 Whenever `merge_all` is `True`, the results are flattened into a single list. To figure out which results belong to which set, we can read `keys`.
- 4 Each result is a tuple of entries and exits, exactly how returned by `get_entries_and_exits`
- 5 Use `SignalsAccessor.plot_as_entries` and `SignalsAccessor.plot_as_exits` to plot the signals as entry and exit markers respectively
- 6 Change the appearance of the markers depending on the signal type and set
- 7 By making the legend group the same for multiple traces, we can make them react to any user interactions in the same way
- 8 Show the legend only if there isn't any trace with the same name already plotted, otherwise there will be duplicates
- 9 Our merging function returns a Plotly figure



As we can see, `Splitter.apply` is a flexible method that can execute any UDF on each range in the splitter. Not only it can return arrays in an analysis-friendly format, but it can also post-process and merge the outputs using another UDF, which makes it ideal for quick CV.

Decorators

But even the method above isn't the end of automation that vectorbt offers: similarly to the decorator `@parameterized`, which can enhance any function with a parameter processing logic, there is a decorator `@split` that can enhance just about any function with a split processing logic. The workings of this decorator are dead-simple: wrap a function, resolve a splitting specification into a splitter, and forward all arguments down to `Splitter.apply`. This way, we're making CV pipeline-centric and not splitter-centric.

There are several ways how to make a function splittable:

```
>>> @vbt.split(splitter=splitter) ❶
... def get_split_total_return(data):
...     return data.returns.vbt.returns.total()

>>> get_split_total_return(vbt.Takeable(data))
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test    2.132976
2021      train   0.194783
          test    0.379417
dtype: float64

>>> def get_total_return(data):
...     return data.returns.vbt.returns.total()

>>> get_split_total_return = vbt.split( ❷
...     get_total_return,
...     splitter=splitter
... )
>>> get_split_total_return(vbt.Takeable(data))
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test    2.132976
2021      train   0.194783
          test    0.379417
dtype: float64
```

- ❶ Through decoration
- ❷ Through passing the function as the first argument

If we didn't pass any argument to the decorator, or if we want to override some argument, we can add a prefix `_` to an argument to pass it to `@split` rather than to the function itself:

```
>>> @vbt.split
... def get_split_total_return(data):
...     return data.returns.vbt.returns.total()

>>> get_split_total_return(vbt.Takeable(data), _splitter=splitter)
split_year  set
2018      train  -0.522416
          test   -0.417491
2019      train   1.858493
          test   -0.322797
2020      train   0.269093
          test    2.132976
2021      train   0.194783
          test    0.379417
dtype: float64
```

A potential inconvenience of the approach above is that for each data we pass, we need to separately construct a splitter using the index of the same data. To solve this, the decorator can also take an instruction on how to create a splitter from the passed data. The instruction consists of a method name (or the actual callable) provided as `splitter`, such as "from_rolling", and keyword arguments passed to this method as `splitter_kwargs`. Let's roll a window of 30 days of the year 2020:

```
>>> get_split_total_return(
...     vbt.Takeable(data.loc["2020":"2020"]),
...     _splitter="from_rolling",
...     _splitter_kwargs=dict(length="30d")
... )
split
0      0.321123
1     -0.088666
2     -0.250531
3      0.369418
4      0.093628
5     -0.059949
6      0.186424
7      0.020706
8     -0.069256
9      0.211424
10     0.372754
11     0.441005
dtype: float64
```

To avoid wrapping each object to take slices from with `Takeable`, we can also specify a list of such arguments as `takeable_args`:

```
>>> get_total_return_by_month = vbt.split(
...     get_total_return,
...     splitter="from_grouper",
...     splitter_kwargs=dict(by=vbt.RepEval("index.to_period('M')")),
...     takeable_args=["data"]
... )

>>> get_total_return_by_month(data.loc["2020":"2020"])
Open time
2020-01      0.298859
2020-02     -0.091746
2020-03     -0.248649
2020-04      0.297622
2020-05      0.070388
2020-06     -0.104131
2020-07      0.227844
2020-08     -0.012851
2020-09     -0.096073
2020-10      0.298694
2020-11      0.431230
2020-12      0.541364
Freq: M, dtype: float64
```

In the example above, a new splitter is built from every instance of data that we pass.

Furthermore, we can combine multiple decorators. For example, let's decorate the function `sma_crossover_perf` from the first page that we've already decorated with `@parameterized`, and make it split the entire period into 60% for the training set and 40% for the test set:

```
>>> cv_sma_crossover_perf = vbt.split(
...     sma_crossover_perf,
...     splitter="from_single", ①
...     splitter_kwargs=dict(split=0.6, set_labels=["train", "test"]),
...     takeable_args=["data"],
...     merge_func="concat", ②
... )
```

- ① Use `Splitter.from_single` to build a single split
- ② Any other argument passed to `@split` will be forwarded down to `Splitter.apply`

We'll run the full parameter grid on the training set only:

```
>>> train_perf = cv_sma_crossover_perf(
...     data.loc["2020":"2021"],
...     vbt.Param(np.arange(5, 50), condition="x < slow_window"),
...     vbt.Param(np.arange(5, 50)),
...     p_execute_kwargs=dict(
...         clear_cache=50,
...         collect_garbage=50
...     ),
...     _forward_kwargs_as={
...         "p_execute_kwargs": "_execute_kwargs" ①
...     },
...     _apply_kwargs=dict(set_="train") ②
... )
```

- ① Both `@split` and `@parameterized` take `execute_kwargs`. To differentiate between them, we'll pass `p_execute_kwargs`, let `@split` rename it to `_execute_kwargs` using `forward_kwargs_as`, and forward the argument down to `@parameterized`
- ② If we need to change any argument related to `Splitter.apply`, we can pass it directly to `@split`, or include it in `_apply_kwargs` when passed to an already decorated function

```
>>> train_perf
fast_window  slow_window
5            6            1.160003
            7            1.122994
            8            2.054193
            9            1.880043
           10            1.632951
            ...
46           48            1.362618
           49            1.415010
47           48            1.317795
           49            1.250835
48           49            0.160916
Length: 990, dtype: float64
```

We can then validate the optimization performance on the test set:

```
>>> test_perf = cv_sma_crossover_perf(
...     data.loc["2020":"2021"],
...     train_perf.idxmax()[0],
...     train_perf.idxmax()[1],
...     _apply_kwargs=dict(set_="test")
... )
```

Split 1/1

```
>>> test_perf
1.2596407796960982
```

Hint

If you want to have a proper Series instead of a single value returned, disable `squeeze_one_split` and `squeeze_one_set` in `Splitter.apply` using `_apply_kwargs`.

But even this decorator isn't the final form of automation: there's a special decorator `@cv_split` that combines `@split` and `@parameterized` to run the full parameter grid on the first set and the best parameter combination on the remaining sets. How is the best parameter combination defined? There's an argument `selection` that can be a template taking the previous results (available as `grid_results` in the context) and returning the integer position of the parameter combination that the user defines as the best. Moreover, the decorator can either return the best results only (`return_grid=False`), or additionally include the grid results from the training set (`return_grid=True`) or even the grid run on all sets (`return_grid="all"`).

Since we're waiting too much time for the entire grid of parameter combinations to complete, let's rewrite our pipeline with Numba to return the Sharpe ratio of a single parameter combination:

```
>>> @njit(nogil=True) ①
... def sma_crossover_perf_nb(close, fast_window, slow_window, ann_factor):
...     fast_sma = vbt.nb.ma_nb(close, fast_window) ②
...     slow_sma = vbt.nb.ma_nb(close, slow_window)
...     entries = vbt.nb.crossed_above_nb(fast_sma, slow_sma) ③
...     exits = vbt.nb.crossed_above_nb(slow_sma, fast_sma)
...     sim_out = vbt.pf_nb.from_signals_nb( ④
...         target_shape=close.shape,
...         group_lens=np.full(close.shape[1], 1),
...         close=close,
...         long_entries=entries,
...         short_entries=exits,
...         save_returns=True
...     )
...     return vbt.ret_nb.sharpe_ratio_nb( ⑤
...         sim_out.in_outputs.returns,
...         ann_factor
...     )
```

① Disable GIL for multithreading

② Use `ma_nb`

③ Use `crossed_above_nb`

④ Use `from_signals_nb`

⑤ Use `sharpe_ratio_nb`

Test the function on the full history:

```
>>> sma_crossover_perf_nb(vbt.to_2d_array(data.close), 20, 30, 365)
array([1.04969317])
```

Note

All Numba functions that we use are expecting a two-dimensional NumPy array as input.

Finally, let's define and run CV in a parallel manner:

```
>>> cv_sma_crossover_perf = vbt.cv_split(
...     sma_crossover_perf_nb,
...     splitter="from_rolling",
...     splitter_kwargs=dict(
```

```

...     length=360,
...     split=0.5,
...     set_labels=["train", "test"]
... ),
...     takeable_args=["close"],
...     merge_func="concat", ①
...     parameterized_kwargs=dict( ②
...         engine="dask",
...         chunk_len="auto",
...     )
... )

>>> grid_perf, best_perf = cv_sma_crossover_perf(
...     vbt.to_2d_array(data.close),
...     vbt.Param(np.arange(5, 50), condition="x < slow_window"),
...     vbt.Param(np.arange(5, 50)),
...     pd.Timedelta(days=365) // data.index.freq, ③
...     _merge_kwargs=dict(wrapper=data.symbol_wrapper), ④
...     _index=data.index, ⑤
...     _return_grid="all" ⑥
... )

```

- ① Keyword arguments passed to `@parameterized`
- ② Chunk length will be based on the number of CPUs
- ③ The merging function passed to `@split`, will be automatically passed to `@parameterized` as well (and yes, both decorators can have different merging functions)
- ④ Get the annualization factor by dividing the year frequency by the data frequency
- ⑤ Our Numba function returns a one-dimensional NumPy array (i.e., one value per column in the data), thus we need to pass the wrapper to assign each value in the returned array with the corresponding column name
- ⑥ `index` is an argument of `@split` and can be used to pass an index if no index can be extracted from the "takeable" arrays
- ⑦ Run the entire grid on each set. Will return both the grid results and the best results. Note that the test set will still use the previous grid, not its own.

Info

By default, the highest value is selected. To select the lowest value, set `selection="min"` in `@split`. Also, make sure to change the selection template if any other merging function is being used.

Split 9/9

That was fast! 🐌

Question

Why is the performance so different compared to the previous version, which, by the way, uses the same Numba functions under the hood? Remember that when running one function a thousand of times, even a 1-millisecond longer execution time translates into a 1-second longer **total** execution time.

Let's take a look at the CV results:

```

>>> grid_perf
split  set  fast_window  slow_window  symbol
0      train  5          6          BTCUSDT  1.771782
          7          BTCUSDT  2.206458
          8          BTCUSDT  2.705892
          9          BTCUSDT  1.430768
         10         BTCUSDT  0.851692
...
8      test  46         48          BTCUSDT  0.637332
          49         BTCUSDT -0.424650
          47         48          BTCUSDT -0.214946
          49         BTCUSDT  0.231712
          48         49          BTCUSDT -0.351245
Length: 17820, dtype: float64

>>> best_perf
split  set  fast_window  slow_window  symbol
0      train  19         34          BTCUSDT  4.392966
      test  19         34          BTCUSDT  0.535497
1      train  6          7          BTCUSDT  2.545991
      test  6          7          BTCUSDT -1.101692
2      train  18         20          BTCUSDT  4.363491
      test  18         20          BTCUSDT  1.692070
3      train  14         18          BTCUSDT  3.615833
      test  14         18          BTCUSDT  0.035444
4      train  18         21          BTCUSDT  3.236440
      test  18         21          BTCUSDT  1.882290
5      train  20         27          BTCUSDT  3.371474
      test  20         27          BTCUSDT  2.574914
6      train  11         18          BTCUSDT  4.657549
      test  11         18          BTCUSDT -2.067505
7      train  29         30          BTCUSDT  3.388797
      test  29         30          BTCUSDT  0.968127
8      train  25         28          BTCUSDT  2.149624
      test  25         28          BTCUSDT  1.256857
dtype: float64

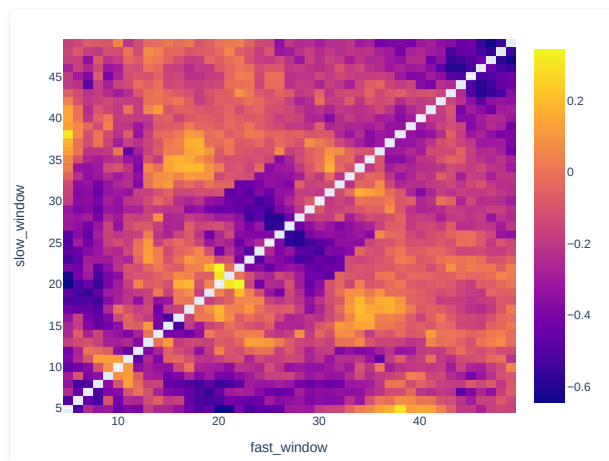
```

For instance, the test results negatively correlate with the training results, meaning that the parameter combinations with the highest Sharpe tend to underperform when they overperformed previously. This actually makes sense because BTC market regimes tend to switch frequently.

```
>>> best_train_perf = best_perf.xs("train", level="set")
>>> best_test_perf = best_perf.xs("test", level="set")
>>> best_train_perf.corr(best_test_perf)
-0.21641517083891232
```

To dig deeper, let's analyze the cross-set correlation of each parameter combination, that is, how the performance of a parameter combination in the training set correlates with the performance of the same parameter combination in the test set:

```
>>> param_cross_set_corr = grid_perf\
...     .unstack("set")\
...     .groupby(["fast_window", "slow_window"])\
...     .apply(lambda x: x["train"].corr(x["test"]))
>>> param_cross_set_corr.vbt.heatmap(symmetric=True).show()
```



Another perspective can be added by analyzing the test performance of the best parameter combinations in relation to the test performance of all parameter combinations. Let's get the relative number of parameter combinations that are better than the selected one in each split:

```
>>> grid_test_perf = grid_perf.xs("test", level="set")
>>> grid_df = grid_test_perf.rename("grid").reset_index() ❶
>>> del grid_df["fast_window"] ❷
>>> del grid_df["slow_window"]
>>> best_df = best_test_perf.rename("best").reset_index()
>>> del best_df["fast_window"]
>>> del best_df["slow_window"]
>>> merged_df = pd.merge(grid_df, best_df, on=["split", "symbol"]) ❸
>>> grid_better_mask = merged_df["grid"] > merged_df["best"] ❹
>>> grid_better_mask.index = grid_test_perf.index
>>> grid_better_cnt = grid_better_mask.groupby(["split", "symbol"]).mean() ❺
>>> grid_better_cnt
split  symbol
0      BTCUSD  0.242424
1      BTCUSD  0.988889
2      BTCUSD  0.214141
3      BTCUSD  0.404040
4      BTCUSD  0.359596
5      BTCUSD  0.963636
6      BTCUSD  0.908081
7      BTCUSD  0.342424
8      BTCUSD  0.250505
dtype: float64
```

- ❶ Make columns out of the index
- ❷ Remove the columns that we don't want to merge by
- ❸ Merge both DataFrames by split and symbol
- ❹ Get the mask of the grid results being better than the best results
- ❺ Use `sum()` to get the absolute count and `mean()` to get the relative count

The selected parameter combinations seem to beat the most other parameter combinations tested during the same time period, but some results are particularly disappointing: in the splits 1, 5, and 6, the selected parameter combination performed worse than other 90%.

Finally, let's compare the results against our buy-and-hold baseline. For this, we need to extract the price that belongs to each split, but how do we do that without a splitter? Believe it or not, `@split` has an argument to return the splitter without running the pipeline 😊

```
>>> cv_splitter = cv_sma_crossover_perf(
...     _index=data.index, ❶
...     _return_splitter=True
... )
>>> stacked_close = cv_splitter.take(
...     data.close,
```



```

...     into="reset_stacked",
...     set_="test" ❷
... )
>>> hold_pf = vbt.Portfolio.from_holding(stacked_close, freq="daily")
>>> hold_perf = hold_pf.sharpe_ratio
>>> hold_perf
split
0    -0.430642
1    -1.741407
2     3.408079
3    -0.556471
4     0.954291
5     3.241618
6     0.686198
7    -0.038013
8    -0.917722
Name: sharpe_ratio, dtype: float64

```

- ❶ We either need to pass the index or all arguments for parsing
- ❷ Select the test set only

As we can see, the "taking" and "applying" approaches can be safely combined since the underlying splitter is guaranteed to be built in the same way and thus produce the same results (unless the splitter method has some random component and hasn't been provided with a seed).

Modeling

The class [Splitter](#) can also be helpful in cross-validating ML models. In particular, you can casually step upon a class [SplitterCV](#) that acts as a regular cross-validator from scikit-learn by subclassing `BaseCrossValidator`. We'll demonstrate its usage on a simple classification problem of predicting the best entry and exit timings.

Before we start, we need to decide on features and labels that should act as predictor and response variables respectively. Features are usually multi-columnar time-series DataFrames where each row contains multiple data points (one per column) that should predict the same row in labels. Labels are usually a single-columnar time-series Series that should be predicted. Ask yourself the following questions to easily come up with a decision:

1. "How can the future performance be represented, preferably as a single number? Should it be the price at the next bar, the average price change over the next week, a vector of weights for rebalancing, a boolean containing a signal, or something else?"
2. "What kind of data that encompasses the past performance is likely to predict the future performance? Should it be indicators, news sentiment index, past backtesting results, or something else?"
3. "Which ML model can handle such a task?" (remember that most models are limited to just a couple of specific feature and label formats!)

For the sake of an example, we'll fit a [random forest classifier](#) on all [TA-Lib](#) indicators stacked along columns to predict the binary labels generated by the label generator [TRENDLB](#), where 1 means an uptrend and 0 means a downtrend. Sounds like fun 😊

First, run all the TA-Lib indicators on the entire data to get the feature set `X`:

```

>>> X = data.run("talib")
>>> X.shape
(1902, 174)

```

We've got 1902 rows (dates) and 174 columns (features).

Next, generate the labels `y` (we'll use the same configuration as previously):

```

>>> trendlb = data.run("trendlb", 1.0, 0.5, mode="binary")
>>> y = trendlb.labels
>>> y.shape
(1902,)

```

Both the features and the labels contain NaNs, which we need to carefully take care of. If we remove the rows with at least one NaN, we'll remove all the data. Instead, we'll first remove the columns that consist entirely of NaNs or a single unique value. Also, because `X` and `y` should have the same length, we need to do the row-filtering operation on both datasets simultaneously:

```

>>> X = X.replace([-np.inf, np.inf], np.nan)
>>> invalid_column_mask = X.isnull().all(axis=0) | (X.nunique() == 1)
>>> X = X.loc[:, ~invalid_column_mask]
>>> invalid_row_mask = X.isnull().any(axis=1) | y.isnull()
>>> X = X.loc[~invalid_row_mask]
>>> y = y.loc[~invalid_row_mask]
>>> X.shape, y.shape
((1773, 144), (1773,))

```

Warning

If you worked with ML before, you'll quickly feel the danger coming from the logical operation in the first cell: we're checking for a condition across the entire column, thus potentially catching the [look-ahead bias](#). Even though our operation isn't too dangerous because we remove only the columns that are likely to stay irrelevant in the future, other transformations such as data normalization should always be included in a [Pipeline](#) that's executed per split rather than once and globally.

We've successfully removed a total of 129 rows and 30 columns.

Next, we'll establish our classifier that will learn `X` to predict `y`:

```
>>> from sklearn.ensemble import RandomForestClassifier ❶
>>> clf = RandomForestClassifier(random_state=42)
```

❶ Use `RandomForestClassifier`

❷ Question

Why haven't we rescaled, normalized, or reduced the dimensionality of the features? Random forests are very robust modeling techniques and can handle high noise levels as well as a high number of features.

To cross-validate the classifier, let's create an `SplitterCV` instance that splits the entire period into expanding windows with non-overlapping test periods of 180 bars:

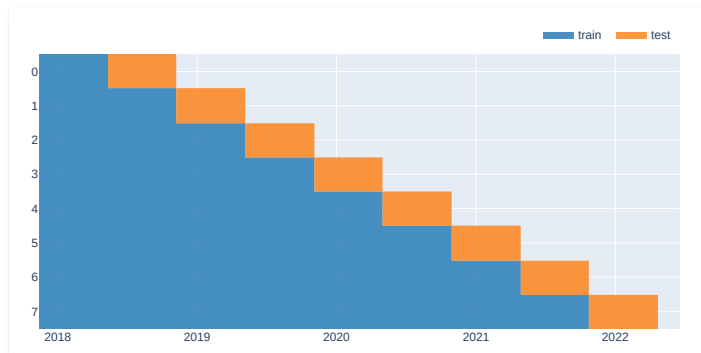
```
>>> cv = vbt.SplitterCV(
...     "from_expanding", ❶
...     min_length=360, ❷
...     offset=180,
...     split=-180,
...     set_labels=["train", "test"]
... )

>>> cv_splitter = cv.get_splitter(X) ❸
>>> cv_splitter.plot().show()
```

❶ Method of `Splitter`

❷ Other keyword arguments are forwarded down to this method

❸ Get the actual splitter for plotting



Finally, run the classifier on each training period and check the accuracy of its predictions on the respective test period. Even though the accuracy score is the most basic of all classification scores and has its own flaws, we'll keep things simplified for now:

```
>>> from sklearn.model_selection import cross_val_score ❶

>>> cross_val_score(clf, X, y, cv=cv, scoring="accuracy")
array([1.         , 0.20555556, 0.88333333, 0.72777778, 1.         ,
        0.92777778, 0.53333333, 0.30555556])
```

❶ Use `cross_val_score`

We can see that there are only two underperforming splits, and even two splits with 100% accuracy, how is this possible? Let's find out! What we need are raw predictions: we'll use the actual splitter to take the slices from `X` and `y`, generate the predictions on each test set using our classifier, and concatenate all the predictions into a single Series.

```
>>> X_slices = cv_splitter.take(X)
>>> y_slices = cv_splitter.take(y)

>>> test_labels = []
>>> test_preds = []
>>> for split in X_slices.index.unique(level="split"): ❶
...     X_train_slice = X_slices[(split, "train")] ❷
...     y_train_slice = y_slices[(split, "train")]
...     X_test_slice = X_slices[(split, "test")]
...     y_test_slice = y_slices[(split, "test")]
...     slice_clf = clf.fit(X_train_slice, y_train_slice) ❸
...     test_pred = slice_clf.predict(X_test_slice) ❹
...     test_pred = pd.Series(test_pred, index=y_test_slice.index)
...     test_labels.append(y_test_slice)
...     test_preds.append(test_pred)

>>> test_labels = pd.concat(test_labels).rename("labels") ❺
>>> test_preds = pd.concat(test_preds).rename("preds")
```

❶ Iterate over the split labels

❷ Get the arrays under the current split

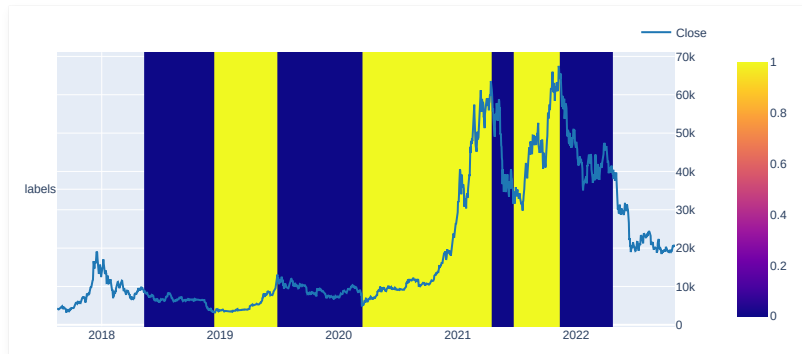
❸ Fit the classifier to the training data

- 4 Generate the predictions and wrap into Series
- 5 Concatenate all the prediction Series along rows

Let's compare the actual labels (left tab) to the predictions (right tab):

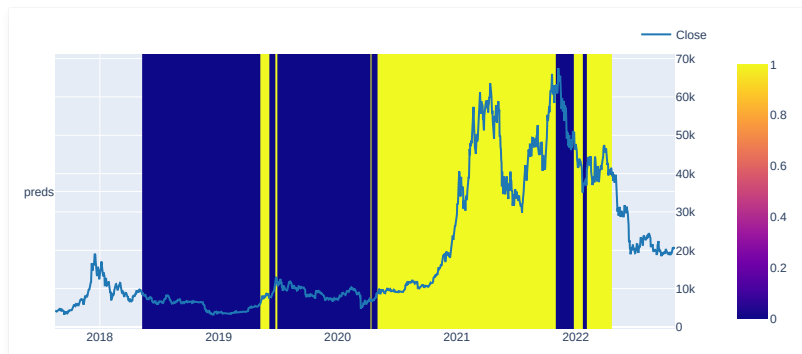
Labels

```
>>> data.close.vbt.overlay_with_heatmap(test_labels).show()
```

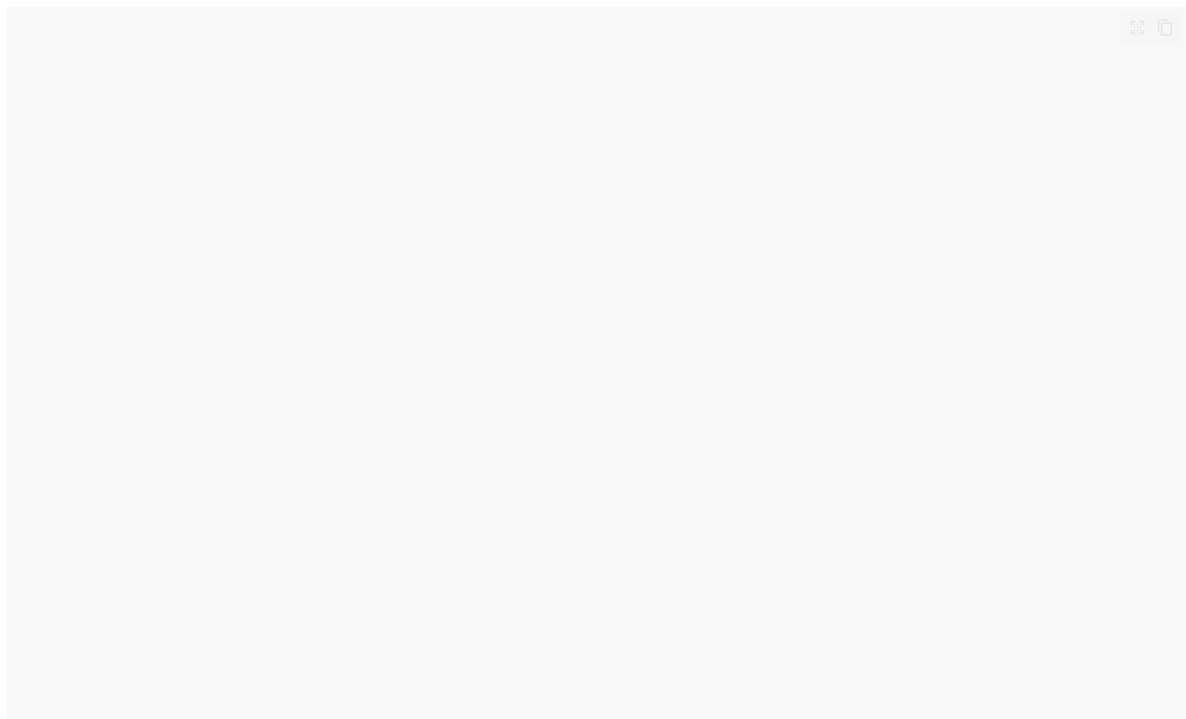


Predictions

```
>>> data.close.vbt.overlay_with_heatmap(test_preds).show()
```



The model seems to correctly classify many bigger uptrends and even issue an exit signal at the latest peak on time! Nevertheless, we shouldn't just rely on our visual intuition: let's backtest the predictions.



```

>>> pf = vbt.Portfolio.from_signals(
...     data.close[test_preds.index],
...     test_preds == 1,
...     test_preds == 0,
...     direction="both" ❶
... )
>>> pf.stats()
Start                2018-05-12 00:00:00+00:00
End                  2022-04-20 00:00:00+00:00
Period              1440 days 00:00:00
Start Value          100.0
Min Value            55.079685
Max Value            1238.365833
End Value            719.483655
Total Return [%]     619.483655
Benchmark Return [%] 388.524488
Total Time Exposure [%] 100.0
Max Gross Exposure [%] 532.441444
Max Drawdown [%]     71.915142
Max Drawdown Duration 244 days 00:00:00
Total Orders         20
Total Fees Paid       0.0
Total Trades         20
Win Rate [%]         52.631579
Best Trade [%]        511.777894
Worst Trade [%]       -27.856728
Avg Winning Trade [%] 66.38609
Avg Losing Trade [%]  -11.784147
Avg Winning Trade Duration 104 days 16:48:00
Avg Losing Trade Duration  35 days 05:20:00
Profit Factor         1.999834
Expectancy           32.802227
Sharpe Ratio          0.983656
Calmar Ratio          0.902507
Omega Ratio           1.202685
Sortino Ratio         1.627438
dtype: object

```

❶ It's important to have any false-positive exit signals be as painful as false-positive entry signals

We've got some pretty solid statistics 🌟

If you're willing to accept a challenge: build a pipeline to impute and (standard-)normalize the data, **reduce the dimensionality** of the features, as well as fit one of the **linear models** to predict the average price change over the next `n` bars (i.e., regression task!). Based on each prediction, you can then decide whether a position is worth opening or closing out.

Summary

Backtesting requires an overhaul of traditional cross-validation schemes centered around ML, and vectorbt offers the needed toolset. The heart of the new functionality is the juggernaut class **Splitter**; it not only provides time-series-safe splitting schemes but also enables us to thoroughly analyze the generated splits and apply them to data of arbitrary complexity efficiently. For instance, we can enjoy the offered flexibility to either split data into slices and perform CV on them manually or construct a pipeline and let the splitter do the slicing and execution parts for us. There's even a decorator for parameterizing and cross-validating any Python function - `@cv_split`. And for any ML-related tasks, the class **SplitterCV** offers a splitter-enhanced interface well understood by scikit-learn and many other packages, such as by the scikit-learn compatible neural network library **skorch** wrapping PyTorch. As a result, validation of rule-based and ML-based models has become as easy as ever 🦋

 [Python code](#)

 [Notebook](#)

API generic splitting

decorators

decorators module

Decorators for splitting.

cv_split function

```
cv_split(
    *args,
    parameterized_kwargs=None,
    selection='max',
    return_grid=False,
    skip_errored=False,
    raise_no_results=True,
    template_context=None,
    **split_kwargs
)
```

Decorator that combines [split](#) and [parameterized](#) for cross-validation.

Creates a new apply function that is going to be decorated with [split](#) and thus applied at each single range using [Splitter.apply](#). Inside this apply function, there is a test whether the current range belongs to the first (training) set. If yes, parameterizes the underlying function and runs it on the entire grid of parameters. The returned results are then stored in a global list. These results are then read by the other (testing) sets in the same split. If `selection` is a template, it can evaluate the grid results (available as `grid_results`) and return the best parameter combination. This parameter combination is then executed by each set (including training).

Argument `selection` also accepts "min" for `np.argmin` and "max" for `np.argmax`.

Keyword arguments `parameterized_kwargs` will be passed to [parameterized](#) and will have their templates substituted with a context that will also include the split-related context (including `split_idx`, `set_idx`, etc., see [Splitter.apply](#)).

If `return_grid` is True or 'first', returns both the grid and the selection. If `return_grid` is 'all', executes the grid on each set and returns along with the selection. Otherwise, returns only the selection.

If [NoResultsException](#) is raised or `skip_errored` is True and any exception is raised, will skip the current iteration and remove it from the final index.

Usage

- Permutate a series and pick the first value. Make the seed parameterizable. Cross-validate based on the highest picked value:

```
>>> from vectorbtpro import *

>>> @vbt.cv_split(
...     splitter="from_n_rolling",
...     splitter_kwargs=dict(n=3, split=0.5),
...     takeable_args=["sr"],
...     merge_func="concat",
... )
... def f(sr, seed):
...     np.random.seed(seed)
...     return np.random.permutation(sr)[0]

>>> index = pd.date_range("2020-01-01", "2020-02-01")
>>> np.random.seed(0)
>>> sr = pd.Series(np.random.permutation(np.arange(len(index))), index=index)
>>> f(sr, vbt.Param([41, 42, 43]))
split set seed
0 set_0 41 22
  set_1 41 28
1 set_0 43 8
  set_1 43 31
2 set_0 43 19
  set_1 43 0
dtype: int64
```

- Extend the example above to also return the grid results of each set:

```
>>> f(sr, vbt.Param([41, 42, 43]), _return_grid="all")
(split set seed
0 set_0 41 22
  42 22
  43 2
  set_1 41 28
  42 28
  43 20
1 set_0 41 5
  42 5
  43 8
  set_1 41 23
```

		42	23
		43	31
2	set_0	41	18
		42	18
		43	19
	set_1	41	27
		42	27
		43	0
dtype: int64,			
split	set	seed	
0	set_0	41	22
	set_1	41	28
1	set_0	43	8
	set_1	43	31
2	set_0	43	19
	set_1	43	0
dtype: int64)			

split function

```
split(
    *args,
    splitter=None,
    splitter_cls=None,
    splitter_kwargs=None,
    index=None,
    index_from=None,
    takeable_args=None,
    template_context=None,
    forward_kwargs_as=None,
    return_splitter=False,
    apply_kwargs=None,
    **var_kwargs
)
```

Decorator that splits the inputs of a function.

Does the following:

1. Resolves the splitter of the type [Splitter](#) using the argument `splitter`. It can be either an already provided splitter instance, the name of a factory method (such as "from_n_rolling"), or the factory method itself. If `splitter` is None, the right method will be guessed based on the supplied arguments using [Splitter.guess_method](#). To construct a splitter, it will pass `index` and `**splitter_kwargs`. Index is getting resolved either using an already provided `index`, by parsing the argument under a name/position provided in `index_from`, or by parsing the first argument from `takeable_args` (in this order).
2. Wraps arguments in `takeable_args` with [Takeable](#)
3. Runs [Splitter.apply](#) with arguments passed to the function as `args` and `kwargs`, but also `apply_kwargs` (the ones passed to the decorator)

Keyword arguments `splitter_kwargs` are passed to the factory method. Keyword arguments `apply_kwargs` are passed to [Splitter.apply](#). If variable keyword arguments are provided, they will be used as `splitter_kwargs` if `apply_kwargs` is already set, and vice versa. If `splitter_kwargs` and `apply_kwargs` aren't set, they will be used as `splitter_kwargs` if a splitter instance hasn't been built yet, otherwise as `apply_kwargs`. If both arguments are set, will raise an error.

Usage

- Split a Series and return its sum:

```
>>> from vectorbtpro import *

>>> @vbt.split(
...     splitter="from_n_rolling",
...     splitter_kwargs=dict(n=2),
...     takeable_args=["sr"]
... )
... def f(sr):
...     return sr.sum()

>>> index = pd.date_range("2020-01-01", "2020-01-06")
>>> sr = pd.Series(np.arange(len(index)), index=index)
>>> f(sr)
split
0      3
1     12
dtype: int64
```

- Perform a split manually:

```
>>> @vbt.split(
...     splitter="from_n_rolling",
...     splitter_kwargs=dict(n=2),
...     takeable_args=["index"]
... )
... def f(index, sr):
...     return sr[index].sum()

>>> f(index, sr)
split
0      3
1     12
```

```
dtype: int64
```

- Construct splitter and mark arguments as "takeable" manually:

```
>>> splitter = vbt.Splitter.from_n_rolling(index, n=2)
>>> @vbt.split(splitter=splitter)
... def f(sr):
...     return sr.sum()

>>> f(vbt.Takeable(sr))
split
0    3
1   12
dtype: int64
```

- Split multiple timeframes using a custom index:

```
>>> @vbt.split(
...     splitter="from_n_rolling",
...     splitter_kwargs=dict(n=2),
...     index=index,
...     takeable_args=["h12_sr", "d2_sr"]
... )
... def f(h12_sr, d2_sr):
...     return h12_sr.sum() + d2_sr.sum()

>>> h12_index = pd.date_range("2020-01-01", "2020-01-06", freq="12H")
>>> d2_index = pd.date_range("2020-01-01", "2020-01-06", freq="2D")
>>> h12_sr = pd.Series(np.arange(len(h12_index)), index=h12_index)
>>> d2_sr = pd.Series(np.arange(len(d2_index)), index=d2_index)
>>> f(h12_sr, d2_sr)
split
0    15
1    42
dtype: int64
```

API generic splitting

base splitting

base module

Base class for splitting.

FixRange class

```
FixRange(
    *args,
    **kwargs
)
```

Class that represents a fixed range.

Superclasses

- Base
- DefineMixin
- Hashable

Inherited members

- Base.chat
- Base.find_api
- Base.find_assets
- Base.find_docs
- Base.find_examples
- Base.find_messages
- DefineMixin.asdict
- DefineMixin.assert_field_not_missing
- DefineMixin.fields
- DefineMixin.fields_dict
- DefineMixin.get_field
- DefineMixin.hash
- DefineMixin.hash_key
- DefineMixin.is_field_missing
- DefineMixin.is_field_optional
- DefineMixin.is_field_required
- DefineMixin.merge_over
- DefineMixin.merge_with
- DefineMixin.replace
- DefineMixin.resolve
- DefineMixin.resolve_field
- Hashable.get_hash

range_ field

Range.

RelRange class

```
RelRange(
    *args,
    **kwargs
)
```

Class that represents a relative range.

Superclasses

- [Base](#)
- [DefineMixin](#)
- [Hashable](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [DefineMixin.asdict](#)
- [DefineMixin.assert_field_not_missing](#)
- [DefineMixin.fields](#)
- [DefineMixin.fields_dict](#)
- [DefineMixin.get_field](#)
- [DefineMixin.hash](#)
- [DefineMixin.hash_key](#)
- [DefineMixin.is_field_missing](#)
- [DefineMixin.is_field_optional](#)
- [DefineMixin.is_field_required](#)
- [DefineMixin.merge_over](#)
- [DefineMixin.merge_with](#)
- [DefineMixin.replace](#)
- [DefineMixin.resolve](#)
- [DefineMixin.resolve_field](#)
- [Hashable.get_hash](#)

is_gap field ⓘ

Whether the range acts as a gap.

length field ⓘ

Length.

Floating values between 0 and 1 are considered relative.

Can be negative.

length_space field ⓘ

Length space.

Supported are

- 'all': All space
- 'free': Remaining space after the offset
- 'free_or_prev': Remaining space after the offset or the start/end of the previous range, depending what comes first in the direction of [RelRange.length](#)

Applied only when [RelRange.length](#) is a relative number.

offset field ⓘ

Offset.

Floating values between 0 and 1 are considered relative.

Can be negative.

offset_anchor field

Offset anchor.

Supported are

- 'start': Start of the range
- 'end': End of the range
- 'prev_start': Start of the previous range
- 'prev_end': End of the previous range

offset_space field

Offset space.

Supported are

- 'all': All space
- 'free': Remaining space after the offset anchor
- 'prev': Length of the previous range

Applied only when [RelRange.offset](#) is a relative number.

out_of_bounds field

Check if start and stop are within bounds.

Supported are

- 'keep': Keep out-of-bounds values
- 'ignore': Ignore if out-of-bounds
- 'warn': Emit a warning if out-of-bounds
- 'raise': Raise an error if out-of-bounds

to_slice method

```
RelRange.to_slice(  
    total_len,  
    prev_start=0,  
    prev_end=0,  
    index=None,  
    freq=None  
)
```

Convert the relative range into a slice.

Splitter class

```
Splitter(  
    wrapper,  
    index,  
    splits_arr,  
    **kwargs  
)
```

Base class for splitting.

Superclasses

- [Analyzable](#)
- [AttrResolverMixin](#)
- [Base](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)

- [ExtPandasIndexer](#)
- [HasSettings](#)
- [IndexApplier](#)
- [IndexingBase](#)
- [Itemable](#)
- [PandasIndexer](#)
- [Paramable](#)
- [Pickleable](#)
- [PlotsBuilderMixin](#)
- [Prettified](#)
- [StatsBuilderMixin](#)
- [Wrapping](#)

Inherited members

- [Analyzable.cls_dir](#)
- [Analyzable.column_only_select](#)
- [Analyzable.config](#)
- [Analyzable.group_select](#)
- [Analyzable.iloc](#)
- [Analyzable.indexing_kwargs](#)
- [Analyzable.loc](#)
- [Analyzable.range_only_select](#)
- [Analyzable.rec_state](#)
- [Analyzable.self_aliases](#)
- [Analyzable.wrapper](#)
- [Analyzable.xloc](#)
- [AttrResolverMixin.deep_getattr](#)
- [AttrResolverMixin.post_resolve_attr](#)
- [AttrResolverMixin.pre_resolve_attr](#)
- [AttrResolverMixin.resolve_attr](#)
- [AttrResolverMixin.resolve_shortcut_attr](#)
- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [Cacheable.get_ca_setup](#)
- [Chainable.chain](#)
- [Chainable.pipe](#)
- [Configured.copy](#)
- [Configured.equals](#)
- [Configured.get_writeable_attrs](#)
- [Configured.prettify](#)
- [Configured.replace](#)
- [Configured.resolve_merge_kwargs](#)
- [Configured.update_config](#)
- [HasSettings.get_path_setting](#)
- [HasSettings.get_path_settings](#)
- [HasSettings.get_setting](#)
- [HasSettings.get_settings](#)
- [HasSettings.has_path_setting](#)
- [HasSettings.has_path_settings](#)
- [HasSettings.has_setting](#)
- [HasSettings.has_settings](#)

- [HasSettings.reset_settings](#)
- [HasSettings.resolve_setting](#)
- [HasSettings.resolve_settings_paths](#)
- [HasSettings.set_settings](#)
- [IndexApplier.add_levels](#)
- [IndexApplier.drop_duplicate_levels](#)
- [IndexApplier.drop_levels](#)
- [IndexApplier.drop_redundant_levels](#)
- [IndexApplier.rename_levels](#)
- [IndexApplier.select_levels](#)
- [IndexingBase.indexing_setter_func](#)
- [PandasIndexer.xs](#)
- [Pickleable.decode_config](#)
- [Pickleable.decode_config_node](#)
- [Pickleable.dumps](#)
- [Pickleable.encode_config](#)
- [Pickleable.encode_config_node](#)
- [Pickleable.file_exists](#)
- [Pickleable.getsize](#)
- [Pickleable.load](#)
- [Pickleable.loads](#)
- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)
- [PlotsBuilderMixin.build_subplots_doc](#)
- [PlotsBuilderMixin.override_subplots_doc](#)
- [PlotsBuilderMixin.plots](#)
- [PlotsBuilderMixin.resolve_plots_setting](#)
- [Prettified.pprint](#)
- [StatsBuilderMixin.build_metrics_doc](#)
- [StatsBuilderMixin.override_metrics_doc](#)
- [StatsBuilderMixin.resolve_stats_setting](#)
- [StatsBuilderMixin.stats](#)
- [Wrapping.apply_to_index](#)
- [Wrapping.as_param](#)
- [Wrapping.get_item_keys](#)
- [Wrapping.items](#)
- [Wrapping.regroup](#)
- [Wrapping.resample](#)
- [Wrapping.resolve_self](#)
- [Wrapping.resolve_stack_kwargs](#)
- [Wrapping.select_col](#)
- [Wrapping.select_col_from_obj](#)
- [Wrapping.split](#)
- [Wrapping.split_apply](#)

apply method 

```

Splitter.apply(
    apply_func,
    *apply_args,
    split=None,
    set_=None,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    remap_to_obj=True,
    obj_index=None,

```

```

obj_freq=None,
range_format='slice_or_any',
point_wise=False,
attach_bounds=False,
right_inclusive=False,
template_context=None,
silence_warnings=False,
index_combine_kwargs=None,
freq=None,
iteration='split_wise',
execute_kwargs=None,
filter_results=True,
raise_no_results=True,
merge_func=None,
merge_kwargs=None,
merge_all=True,
wrap_results=True,
eval_id=None,
**apply_kwargs
)

```

Apply a function on each range.

Uses [Splitter.select_indices](#) to get the indices for selected splits and sets. Arguments `split_group_by` and `set_group_by` can be used to group splits and sets respectively. Ranges belonging to the same split and set group will be merged.

For each index pair, in a lazily manner, resolves the source range using [Splitter.select_range](#) and [Splitter.get_ready_range](#). Then, takes each argument from `args` and `kwargs` wrapped with [Takeable](#), remaps the range into each object's index using [Splitter.get_ready_obj_range](#), and takes the slice from that object using [Splitter.take_range](#). The original object will be substituted by this slice. At the end, substitutes any templates in the prepared `args` and `kwargs` and saves the function and arguments for execution.

For substitution, the following information is available:

- `split/set_group_indices`: Indices corresponding to the selected row/column groups
- `split/set_indices`: Indices corresponding to the selected rows/columns
- `n_splits/sets`: Number of the selected rows/columns
- `split/set_labels`: Labels corresponding to the selected row/column groups
- `split/set_idx`: Index of the selected row/column
- `split/set_label`: Label of the selected row/column
- `range_`: Selected range ready for indexing (see [Splitter.get_ready_range](#))
- `range_meta`: Various information on the selected range
- `obj_range_meta`: Various information on the range taken from each takeable argument. Positional arguments are denoted by position, keyword arguments are denoted by keys.
- `args`: Positional arguments with ranges already selected
- `kwargs`: Keyword arguments with ranges already selected
- `bounds`: A tuple of either integer or index bounds. Can be source or target depending on `attach_bounds`.
- `template_context`: Passed template context

Since each range is processed lazily (that is, upon request), there are multiple iteration modes controlled by the argument `iteration`:

- 'split_major': Flatten all ranges in split-major order and iterate over them
- 'set_major': Flatten all ranges in set-major order and iterate over them
- 'split_wise': Iterate over splits, while ranges in each split are processed sequentially
- 'set_wise': Iterate over sets, while ranges in each set are processed sequentially

The execution is done using `execute` with `execute_kwargs`. Once all results have been obtained, attempts to merge them using `merge_func` with `merge_kwargs` (all templates in it will be substituted as well), which can also be a string or a tuple of strings resolved using [resolve_merge_func](#). If `wrap_results` is enabled, packs the results into a Pandas object. If `apply_func` returns something complex, the resulting Pandas object will be of object data type. If `apply_func` returns a tuple (detected by the first returned result), a Pandas object is built for each element of that tuple.

If `merge_all` is True, will merge all results in a flattened manner irrespective of the iteration mode. Otherwise, will merge by split/set.

If [NoResult](#) is returned, will skip the current iteration and remove it from the final index.

Usage

- Get the return of each data range:

```

>>> from vectorbtpro import *

>>> data = vbt.YFData.pull(
...     "BTC-USD",
...     start="2020-01-01 UTC",
...     end="2021-01-01 UTC"
... )
>>> splitter = vbt.Splitter.from_n_rolling(data.wrapper.index, 5)

>>> def apply_func(data):
...     return data.close.iloc[-1] - data.close.iloc[0]

```

```
>>> splitter.apply(apply_func, vbt.Takeable(data))
split
0    -1636.467285
1     3706.568359
2    2944.720703
3    -118.113281
4    17098.916016
dtype: float64
```

- The same but by indexing manually:

```
>>> def apply_func(range_, data):
...     data = data.iloc[range_]
...     return data.close.iloc[-1] - data.close.iloc[0]

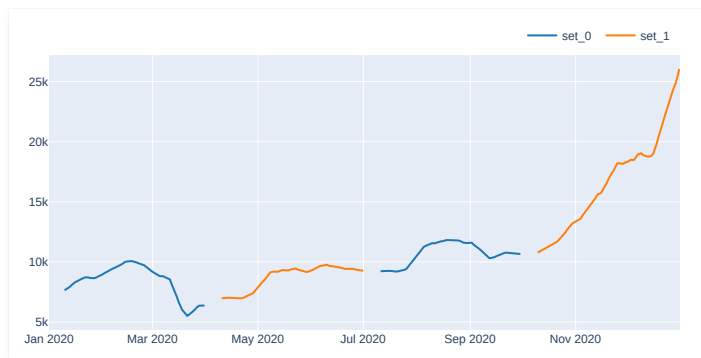
>>> splitter.apply(apply_func, vbt.Rep("range_"), data)
split
0    -1636.467285
1     3706.568359
2    2944.720703
3    -118.113281
4    17098.916016
dtype: float64
```

- Divide into two windows, each consisting of 50% train and 50% test, compute SMA for each range, and row-stack the outputs of each set upon merging:

```
>>> splitter = vbt.Splitter.from_n_rolling(data.wrapper.index, 2, split=0.5)

>>> def apply_func(data):
...     return data.run("SMA", 10).real

>>> splitter.apply(
...     apply_func,
...     vbt.Takeable(data),
...     merge_func="row_stack"
... ).unstack("set").vbt.drop_levels("split", axis=0).vbt.plot().show()
```



bounds class property

[Splitter.get_bounds](#) with default arguments.

bounds_arr class property

[Splitter.get_bounds_arr](#) with default arguments.

break_up_splits method

```
Splitter.break_up_splits(
    new_split,
    sort=False,
    template_context=None,
    wrapper_kwargs=None,
    init_kwargs=None,
    **split_range_kwargs
)
```

Split each split into multiple splits.

If there are multiple sets, make sure to merge them into one beforehand.

Arguments `new_split` and `**split_range_kwargs` are passed to [Splitter.split_range](#).

column_stack class method

```
Splitter.column_stack(
    *objs,
    wrapper_kwargs=None,
    **kwargs
)
```

Stack multiple **Splitter** instances along columns.

Uses **ArrayWrapper.column_stack** to stack the wrappers.

coverage class property

Splitter.get_coverage with default arguments.

duration class property

Splitter.get_duration with default arguments.

from_expanding class method

```
Splitter.from_expanding(
    index,
    min_length,
    offset,
    split=None,
    split_range_kwargs=None,
    range_bounds_kwargs=None,
    template_context=None,
    freq=None,
    **kwargs
)
```

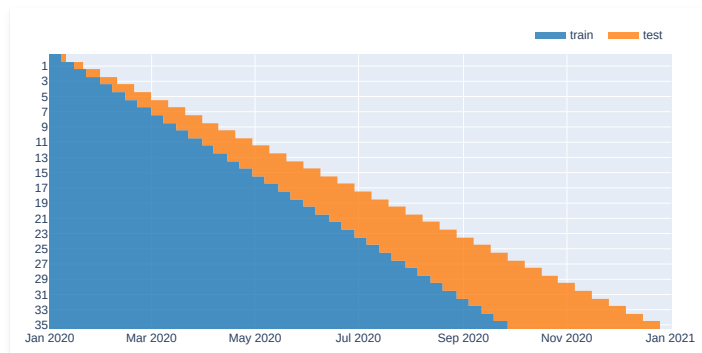
Create a **Splitter** instance from an expanding range.

Argument **min_length** is the minimum length of the expanding range. Provide it as a float between 0 and 1 to make it relative to the length of the index. Argument **offset** is an offset after the right bound of the previous range from which the next range should start. It can also be a float relative to the index length. For other arguments, see **Splitter.from_rolling**.

Usage

- Roll an expanding range with a length of 10 and an offset of 10, and split it into 3/4:

```
>>> from vectorbtpro import *
>>> index = pd.date_range("2020", "2021", freq="D")
>>> splitter = vbt.Splitter.from_expanding(
...     index,
...     10,
...     10,
...     split=3/4,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()
```

**from_grouper** class method

```
Splitter.from_grouper(
```

```

    index,
    by,
    groupby_kwargs=None,
    grouper_kwargs=None,
    split=None,
    split_range_kwargs=None,
    template_context=None,
    split_labels=None,
    freq=None,
    **kwargs
)

```

Create a [Splitter](#) instance from a grouper.

See [BaseIDXAccessor.get_grouper](#).

Uses [Splitter.from_splits](#) to prepare the splits array and labels, and to build the instance.

Usage

- Map each month into a range:

```

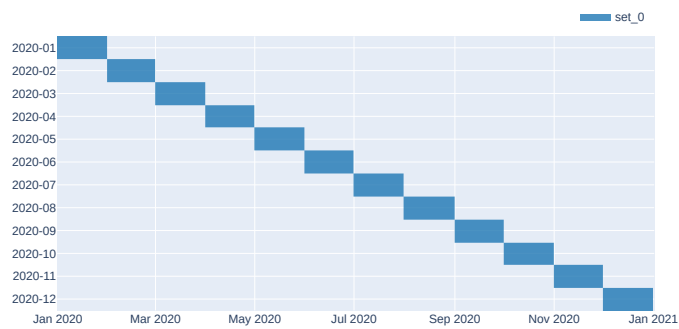
>>> from vectorbtpro import *

>>> index = pd.date_range("2020", "2021", freq="D")

>>> def is_month_end(index, split):
...     last_range = split[-1]
...     return index[last_range][-1].is_month_end

>>> splitter = vbt.Splitter.from_grouper(
...     index,
...     "M",
...     split_check_template=vbt.RepFunc(is_month_end)
... )
>>> splitter.plot().show()

```



from_n_expanding class method

```

Splitter.from_n_expanding(
    index,
    n,
    min_length=None,
    split=None,
    split_range_kwargs=None,
    template_context=None,
    freq=None,
    **kwargs
)

```

Create a [Splitter](#) instance from a number of expanding ranges.

Picks `n` evenly-spaced, expanding ranges. Argument `min_length` defines the minimum length for each range. For other arguments, see [Splitter.from_rolling](#).

Usage

- Roll 10 expanding ranges with a minimum length of 100, while reserving 50 elements for test:

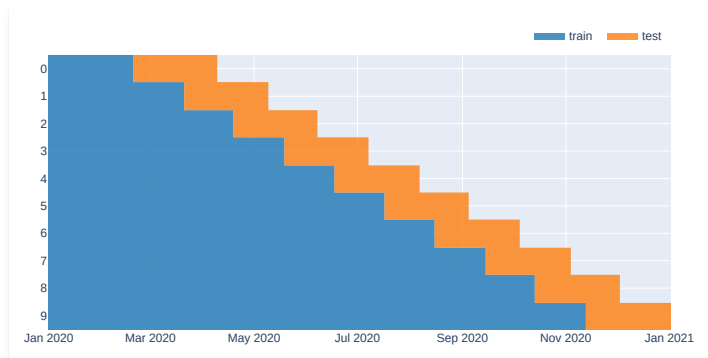
```

>>> from vectorbtpro import *

>>> index = pd.date_range("2020", "2021", freq="D")

>>> splitter = vbt.Splitter.from_n_expanding(
...     index,
...     10,
...     min_length=100,
...     split=-50,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()

```

from_n_random class method

```

Splitter.from_n_random(
    index,
    n,
    min_length,
    max_length=None,
    min_start=None,
    max_end=None,
    length_choice_func=None,
    start_choice_func=None,
    length_p_func=None,
    start_p_func=None,
    seed=None,
    split=None,
    split_range_kwargs=None,
    template_context=None,
    freq=None,
    **kwargs
)

```

Create a **Splitter** instance from a number of random ranges.

Randomly picks the length of a range between `min_length` and `max_length` (including) using `length_choice_func`, which receives an array of possible values and selects one. It defaults to `numpy.random.Generator.choice`. Optional function `length_p_func` takes the same as `length_choice_func` and must return either None or probabilities.

Randomly picks the start position of a range starting at `min_start` and ending at `max_end` (excluding) minus the chosen length using `start_choice_func`, which receives an array of possible values and selects one. It defaults to `numpy.random.Generator.choice`. Optional function `start_p_func` takes the same as `start_choice_func` and must return either None or probabilities.

Note

Each function must take two arguments: the iteration index and the array with possible values.

For other arguments, see **Splitter.from_rolling**.

Usage

- Generate 20 random ranges with a length from [40, 100], and split each into 3/4:

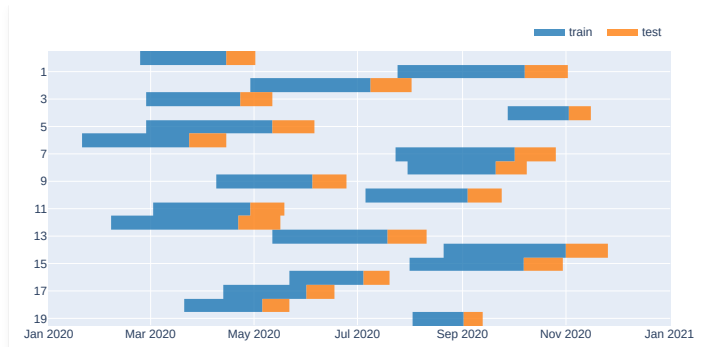
```

>>> from vectorbtpro import *

>>> index = pd.date_range("2020", "2021", freq="D")

>>> splitter = vbt.Splitter.from_n_random(
...     index,
...     20,
...     min_length=40,
...     max_length=100,
...     split=3/4,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()

```



from_n_rolling class method

```

Splitter.from_n_rolling(
    index,
    n,
    length=None,
    optimize_anchor_set=1,
    split=None,
    split_range_kwargs=None,
    template_context=None,
    freq=None,
    **kwargs
)

```

Create a **Splitter** instance from a number of rolling ranges of the same length.

If `length` is `None`, splits the index evenly into `n` non-overlapping ranges using **Splitter.from_rolling**. Otherwise, picks `n` evenly-spaced, potentially overlapping ranges of a fixed length. For other arguments, see **Splitter.from_rolling**.

If `length` is "optimize", searches for a length to cover the most of the index. Use `optimize_anchor_set` to provide the index of a set that should become non-overlapping.

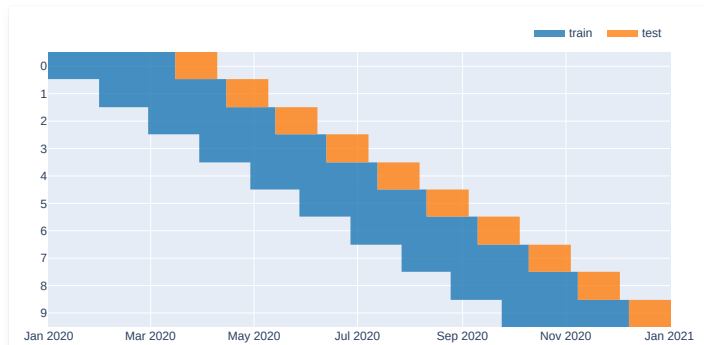
Usage

- Roll 10 ranges with 100 elements, and split it into 3/4:

```

>>> from vectorbtpro import *
>>> index = pd.date_range("2020", "2021", freq="D")
>>> splitter = vbt.Splitter.from_n_rolling(
...     index,
...     10,
...     length=100,
...     split=3/4,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()

```



from_purged class method

```

Splitter.from_purged(
    index,
    purged_splitter,
    pred_times=None,
    eval_times=None,
    split_labels=None,
    set_labels=None,
)

```

```

    **kwargs
)

```

Create a [Splitter](#) instance from a purged splitter.

The splitter must be an instance of [BasePurgedCV](#).

Uses [Splitter.from_splits](#) to prepare the splits array and labels, and to build the instance.

from_purged_kfold class method

```

Splitter.from_purged_kfold(
    index,
    n_folds=10,
    n_test_folds=2,
    purge_td=0,
    embargo_td=0,
    pred_times=None,
    eval_times=None,
    **kwargs
)

```

Create a [Splitter](#) instance from [PurgedKfoldCV](#).

Keyword arguments are passed to [Splitter.from_purged](#).

from_purged_walkforward class method

```

Splitter.from_purged_walkforward(
    index,
    n_folds=10,
    n_test_folds=1,
    min_train_folds=2,
    max_train_folds=None,
    split_by_time=False,
    purge_td=0,
    pred_times=None,
    eval_times=None,
    **kwargs
)

```

Create a [Splitter](#) instance from [PurgedWalkForwardCV](#).

Keyword arguments are passed to [Splitter.from_purged](#).

from_ranges class method

```

Splitter.from_ranges(
    index,
    split=None,
    split_range_kwargs=None,
    template_context=None,
    **kwargs
)

```

Create a [Splitter](#) instance from ranges.

Uses [get_index_ranges](#) to generate start and end indices. Passes only related keyword arguments found in `kwargs`.

Other keyword arguments will be passed to [Splitter.from_splits](#). For details on `split` and `split_range_kwargs`, see [Splitter.from_rolling](#).

Usage

- Translate each quarter into a range:

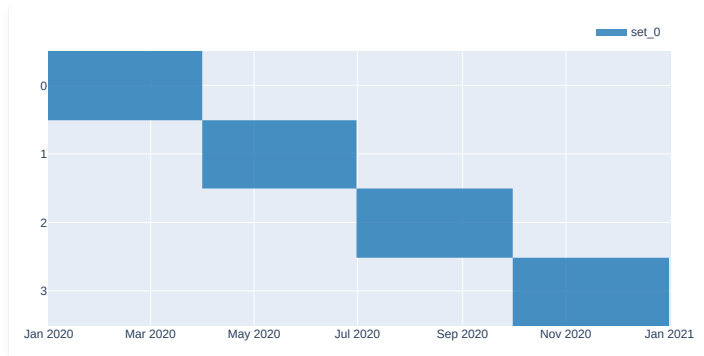
```

>>> from vectorbtpro import *

>>> index = pd.date_range("2020", "2021", freq="D")

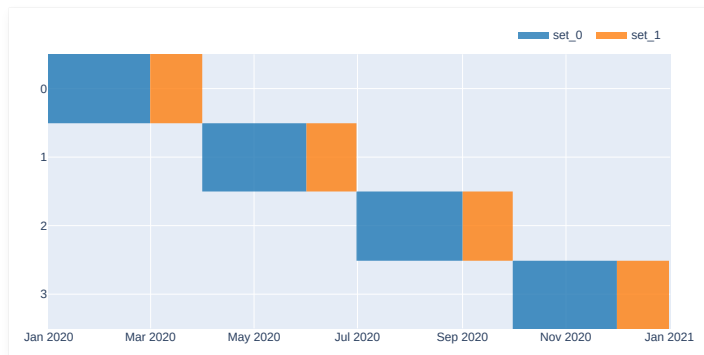
>>> splitter = vbt.Splitter.from_ranges(index, every="QS")
>>> splitter.plot().show()

```



- In addition to the above, reserve the last month for testing purposes:

```
>>> splitter = vbt.Splitter.from_ranges(
...     index,
...     every="QS",
...     split=(1.0, lambda index: index.month == index.month[-1]),
...     split_range_kwargs=dict(backwards=True)
... )
>>> splitter.plot().show()
```



from_rolling class method

```
Splitter.from_rolling(
    index,
    length,
    offset=0,
    offset_anchor='prev_end',
    offset_anchor_set=0,
    offset_space='prev',
    backwards=False,
    split=None,
    split_range_kwargs=None,
    range_bounds_kwargs=None,
    template_context=None,
    freq=None,
    **kwargs
)
```

Create a **Splitter** instance from a rolling range of a fixed length.

Uses **Splitter.from_splits** to prepare the splits array and labels, and to build the instance.

Args

index : index_like

Index.

length : int, float, or timedelta_like

See **RelRange.length**.

offset : int, float, or timedelta_like

See **RelRange.offset**.

offset_anchor : str

See **RelRange.offset_anchor**.

`offset_anchor_set` : int

Offset anchor set.

Selects the set from the previous range to be used as an offset anchor. If None, the whole previous split is considered as a single range. By default, it's the first set.

`offset_space` : str

See [RelRange.offset_space](#).

`backwards` : bool or str

Whether to roll backwards.

If 'sorted', will roll backwards and sort the resulting splits by the start index.

`split` : any

Ranges to split the range into.

If None, will produce the entire range as a single range. Otherwise, will use [Splitter.split_range](#) to split the range into multiple ranges.

`split_range_kwargs` : dict

Keyword arguments passed to [Splitter.split_range](#).

`range_bounds_kwargs` : dict

Keyword arguments passed to [Splitter.get_range_bounds](#).

`template_context` : dict

Context used to substitute templates in ranges.

`freq` : any

Index frequency in case it cannot be parsed from `index`.

If None, will be parsed using [BaseIDXAccessor.get_freq](#).

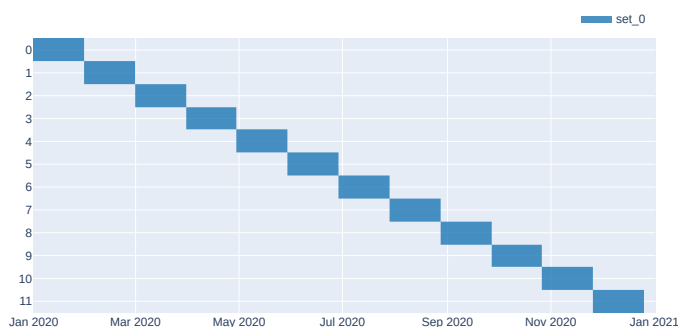
****kwargs**

Keyword arguments passed to the constructor of [Splitter](#).

Usage

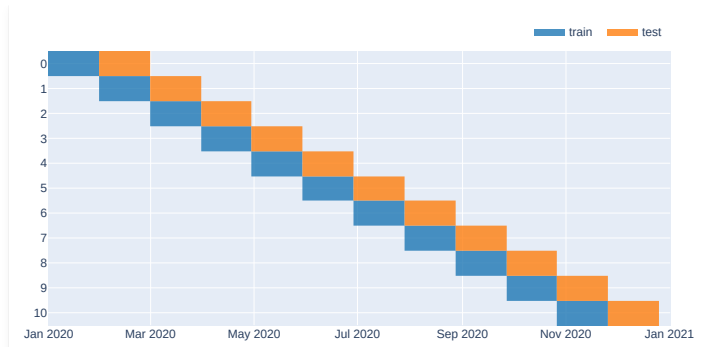
- Divide a range into a set of non-overlapping ranges:

```
>>> from vectorbtpro import *
>>> index = pd.date_range("2020", "2021", freq="D")
>>> splitter = vbt.Splitter.from_rolling(index, 30)
>>> splitter.plot().show()
```



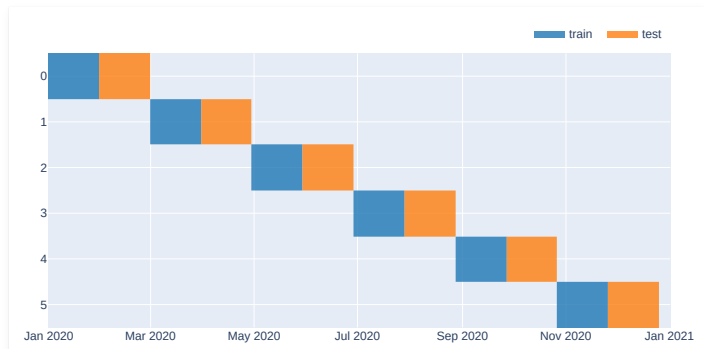
- Divide a range into ranges, each split into 1/2:

```
>>> splitter = vbt.Splitter.from_rolling(
...     index,
...     60,
...     split=1/2,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()
```



- Make the ranges above non-overlapping by using the right bound of the last set as an offset anchor:

```
>>> splitter = vbt.Splitter.from_rolling(
...     index,
...     60,
...     offset_anchor_set=-1,
...     split=1/2,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()
```



from_single class method

```
Splitter.from_single(
    index,
    split,
    split_range_kwargs=None,
    template_context=None,
    **kwargs
)
```

Create a [Splitter](#) instance from a single split.

from_sklearn class method

```
Splitter.from_sklearn(
    index,
    skl_splitter,
    groups=None,
    split_labels=None,
    set_labels=None,
    **kwargs
)
```

Create a [Splitter](#) instance from a scikit-learn's splitter.

The splitter must be an instance of `sklearn.model_selection.BaseCrossValidator`.

Uses [Splitter.from_splits](#) to prepare the splits array and labels, and to build the instance.

from_split_func class method

```
Splitter.from_split_func(
    index,
    split_func,
```

```

split_args=None,
split_kwargs=None,
fix_ranges=True,
split=None,
split_range_kwargs=None,
range_bounds_kwargs=None,
template_context=None,
freq=None,
**kwargs
)

```

Create a [Splitter](#) instance from a custom split function.

In a while-loop, substitutes templates in `split_args` and `split_kwargs` and passes them to `split_func`, which should return either a split (see `new_split` in [Splitter.split_range](#), also supports a single range if it's not an iterable) or None to abrupt the while-loop. If `fix_ranges` is True, the returned split is then converted into a fixed split using [Splitter.split_range](#) and the bounds of its sets are measured using [Splitter.get_range_bounds](#).

Each template substitution has the following information:

- `split_idx`: Current split index, starting at 0
- `splits`: Nested list of splits appended up to this point
- `bounds`: Nested list of bounds appended up to this point
- `prev_start`: Left bound of the previous split
- `prev_end`: Right bound of the previous split
- Arguments and keyword arguments passed to [Splitter.from_split_func](#)

Usage

- Rolling window of 30 elements, 20 for train and 10 for test:

```

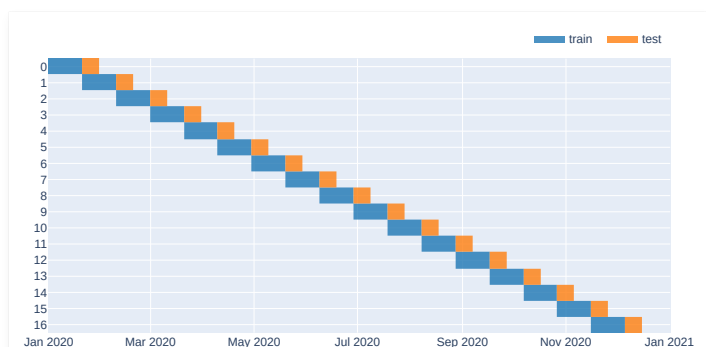
>>> from vectorbtpro import *

>>> index = pd.date_range("2020", "2021", freq="D")

>>> def split_func(splits, bounds, index):
...     if len(splits) == 0:
...         new_split = (slice(0, 20), slice(20, 30))
...     else:
...         # Previous split, first set, right bound
...         prev_end = bounds[-1][0][1]
...         new_split = (
...             slice(prev_end, prev_end + 20),
...             slice(prev_end + 20, prev_end + 30)
...         )
...     if new_split[-1].stop > len(index):
...         return None
...     return new_split

>>> splitter = vbt.Splitter.from_split_func(
...     index,
...     split_func,
...     split_args=(
...         vbt.Rep("splits"),
...         vbt.Rep("bounds"),
...         vbt.Rep("index"),
...     ),
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()

```



from_splits class method

```

Splitter.from_splits(
    index,
    splits,
    squeeze=False,
    fix_ranges=True,
)

```

```

wrap_with_fixrange=False,
split_range_kwargs=None,
split_check_template=None,
template_context=None,
split_labels=None,
set_labels=None,
wrapper_kwargs=None,
**kwargs
)

```

Create a **Splitter** instance from an iterable of splits.

Argument `splits` supports both absolute and relative ranges. To transform relative ranges into the absolute format, enable `fix_ranges`. Arguments `split_range_kwargs` are then passed to **Splitter.split_range**.

Enable `wrap_with_fixrange` to wrap any fixed range with **FixRange**. If the range is an array, it will be wrapped regardless of this argument to avoid building a 3d array.

Pass a template via `split_check_template` to discard splits that do not fulfill certain criteria. The current split will be available as `split`. Should return a boolean (`False` to discard).

Labels for splits and sets can be provided via `split_labels` and `set_labels` respectively. Both arguments can be provided as templates. The split array will be available as `splits`.

get_bounds method

```

Splitter.get_bounds(
    index_bounds=False,
    right_inclusive=False,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    index_combine_kwargs=None,
    **kwargs
)

```

Boolean Series/DataFrame where index are bounds and columns are splits stacked together.

Keyword arguments `**kwargs` are passed to **Splitter.get_bounds_arr**.

get_bounds_arr method

```

Splitter.get_bounds_arr(
    index_bounds=False,
    right_inclusive=False,
    split_group_by=None,
    set_group_by=None,
    template_context=None,
    **range_bounds_kwargs
)

```

Three-dimensional integer array with bounds.

First axis represents splits. Second axis represents sets. Third axis represents bounds.

Each range is getting selected using **Splitter.select_range** and then measured using **Splitter.get_range_bounds**. Keyword arguments `**kwargs` are passed to **Splitter.get_range_bounds**.

get_coverage method

```

Splitter.get_coverage(
    overlapping=False,
    normalize=True,
    split_group_by=None,
    set_group_by=None,
    **kwargs
)

```

Get the coverage of the entire mask.

If `overlapping` is True, returns the number of overlapping True values. If `normalize` is True, returns the number of True values relative to the length of the index. If `overlapping` and `normalize` are True, returns the number of overlapping True values relative to the total number of True values.

Keyword arguments `**kwargs` are passed to **Splitter.get_mask_arr**.

get_duration method


```
Splitter.get_duration(
    **kwargs
)
```

Get duration.

get_iter_set_mask_arrs method

```
Splitter.get_iter_set_mask_arrs(
    split_group_by=None,
    set_group_by=None,
    template_context=None,
    **kwargs
)
```

Generator of two-dimensional boolean arrays, one per set.

First axis represents splits. Second axis represents index.

Keyword arguments `**kwargs` are passed to [Splitter.get_range_mask](#).

get_iter_set_masks method

```
Splitter.get_iter_set_masks(
    split_group_by=None,
    set_group_by=None,
    **kwargs
)
```

Generator of boolean DataFrames, one per set.

Keyword arguments `**kwargs` are passed to [Splitter.get_iter_set_mask_arrs](#).

get_iter_split_mask_arrs method

```
Splitter.get_iter_split_mask_arrs(
    split_group_by=None,
    set_group_by=None,
    template_context=None,
    **kwargs
)
```

Generator of two-dimensional boolean arrays, one per split.

First axis represents sets. Second axis represents index.

Keyword arguments `**kwargs` are passed to [Splitter.get_range_mask](#).

get_iter_split_masks method

```
Splitter.get_iter_split_masks(
    split_group_by=None,
    set_group_by=None,
    **kwargs
)
```

Generator of boolean DataFrames, one per split.

Keyword arguments `**kwargs` are passed to [Splitter.get_iter_split_mask_arrs](#).

get_mask method

```
Splitter.get_mask(
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    index_combine_kwargs=None,
    **kwargs
)
```

Boolean Series/DataFrame where index is [Splitter.index](#) and columns are splits stacked together.

Keyword arguments `**kwargs` are passed to [Splitter.get_mask_arr](#).

Warning

Boolean arrays for a big number of splits may take a considerable amount of memory.

get_mask_arr method

```
Splitter.get_mask_arr(
    split_group_by=None,
    set_group_by=None,
    template_context=None,
    **kwargs
)
```

Three-dimensional boolean array with splits.

First axis represents splits. Second axis represents sets. Third axis represents index.

Keyword arguments `**kwargs` are passed to [Splitter.get_iter_split_mask_arrs](#).

get_n_sets method

```
Splitter.get_n_sets(
    set_group_by=None
)
```

Get number of sets while considering the grouper.

get_n_splits method

```
Splitter.get_n_splits(
    split_group_by=None
)
```

Get number of splits while considering the grouper.

get_obj_index class method

```
Splitter.get_obj_index(
    obj
)
```

Get index from an object.

get_overlap_matrix method

```
Splitter.get_overlap_matrix(
    by='split',
    normalize=True,
    split_group_by=None,
    set_group_by=None,
    jitted=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    index_combine_kwargs=None,
    **kwargs
)
```

Get the overlap between each pair of ranges.

The argument `by` can be one of 'split', 'set', and 'range'.

If `normalize` is True, returns the number of True values in each overlap relative to the total number of True values in both ranges.

Keyword arguments `**kwargs` are passed to [Splitter.get_mask_arr](#).

get_range_bounds class method

```
Splitter.get_range_bounds(
    range_,
    index_bounds=False,
    right_inclusive=False,
    check_constant=True,
)
```

```

        template_context=None,
        index=None,
        freq=None
    )

```

Get the left (inclusive) and right (exclusive) bound of a range.



Note

Even when mapped to the index, the right bound is always exclusive.

get_range_coverage method

```

Splitter.get_range_coverage(
    normalize=True,
    relative=False,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    index_combine_kwargs=None,
    **kwargs
)

```

Get the coverage of each range mask.

If `normalize` is True, returns the number of True values in each range relative to the length of the index. If `normalize` and `relative` are True, returns the number of True values in each range relative to the total number of True values in its split.

Keyword arguments `**kwargs` are passed to [Splitter.get_mask_arr](#).

get_range_mask class method

```

Splitter.get_range_mask(
    range_,
    template_context=None,
    index=None
)

```

Get the mask of a range.

get_ready_obj_range class method

```

Splitter.get_ready_obj_range(
    obj,
    range_,
    remap_to_obj=True,
    obj_index=None,
    obj_freq=None,
    template_context=None,
    jitted=None,
    silence_warnings=False,
    index=None,
    freq=None,
    return_obj_meta=False,
    **ready_range_kwargs
)

```

Get a range that is ready to be mapped into an array-like object.

If the object is Pandas-like and `obj_index` is not None, searches for an index in the object using [Splitter.get_obj_index](#). Once found, uses [Splitter.remap_range](#) to get the range that maps to the object index. Finally, uses [Splitter.get_ready_range](#) to convert the range into the one that can be used directly in indexing.

get_ready_range class method

```

Splitter.get_ready_range(
    range_,
    allow_relative=False,
    allow_zero_len=False,
    range_format='slice_or_any',
    template_context=None,
    index=None,
    return_meta=False
)

```

Get a range that can be directly used in array indexing.

Such a range is either an integer or datetime-like slice (right bound is always exclusive!), a one-dimensional NumPy array with integer indices or datetime-like objects, or a one-dimensional NumPy mask of the same length as the index.

Argument `range_format` accepts the following options:

- 'any': Return any format
- 'indices': Return indices
- 'mask': Return mask of the same length as index
- 'slice': Return slice
- 'slice_or_indices': If slice fails, return indices
- 'slice_or_mask': If slice fails, return mask
- 'slice_or_any': If slice fails, return any format

get_set_coverage method

```
Splitter.get_set_coverage(
    overlapping=False,
    normalize=True,
    relative=False,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_set=True,
    **kwargs
)
```

Get the coverage of each set mask.

If `overlapping` is True, returns the number of overlapping True values between splits in each set. If `normalize` is True, returns the number of True values in each set relative to the length of the index. If `normalize` and `relative` are True, returns the number of True values in each set relative to the total number of True values across all sets.

Keyword arguments `**kwargs` are passed to [Splitter.get_mask_arr](#).

get_set_grouper method

```
Splitter.get_set_grouper(
    set_group_by=None
)
```

Get set grouper.

get_set_labels method

```
Splitter.get_set_labels(
    set_group_by=None
)
```

Get set labels while considering the grouper.

get_split_coverage method

```
Splitter.get_split_coverage(
    overlapping=False,
    normalize=True,
    relative=False,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    **kwargs
)
```

Get the coverage of each split mask.

If `overlapping` is True, returns the number of overlapping True values between sets in each split. If `normalize` is True, returns the number of True values in each split relative to the length of the index. If `normalize` and `relative` are True, returns the number of True values in each split relative to the total number of True values across all splits.

Keyword arguments `**kwargs` are passed to [Splitter.get_mask_arr](#).

get_split_grouper method

```
Splitter.get_split_grouper(  
    split_group_by=None  
)
```

Get split grouper.

get_split_labels method

```
Splitter.get_split_labels(  
    split_group_by=None  
)
```

Get split labels while considering the grouper.

guess_method class method

```
Splitter.guess_method(  
    **kwargs  
)
```

Guess the factory method based on keyword arguments.

Returns None if cannot guess.

index class property

Index.

index_bounds class property

[Splitter.get_bounds](#) with `index_bounds=True`.

index_duration class property

[Splitter.get_duration](#) with `index_bounds=True`.

indexing_func method

```
Splitter.indexing_func(  
    *args,  
    splitter_meta=None,  
    **kwargs  
)
```

Perform indexing on [Splitter](#).

indexing_func_meta method

```
Splitter.indexing_func_meta(  
    *args,  
    wrapper_meta=None,  
    **kwargs  
)
```

Perform indexing on [Splitter](#) and return metadata.

is_range_relative class method

```
Splitter.is_range_relative(  
    range_  
)
```

Return whether a range is relative.

iter_set_mask_arrs class property

[Splitter.get_iter_set_mask_arrs](#) with default arguments.

`iter_set_masks` class property 

[Splitter.get_iter_set_masks](#) with default arguments.

`iter_split_mask_arrs` class property 

[Splitter.get_iter_split_mask_arrs](#) with default arguments.

`iter_split_masks` class property 

[Splitter.get_iter_split_masks](#) with default arguments.

`map_bounds_to_index` class method 

```
Splitter.map_bounds_to_index(
    start,
    stop,
    right_inclusive=False,
    index=None,
    freq=None
)
```

Map bounds to index.

`mask` class property 

[Splitter.get_mask](#) with default arguments.

`mask_arr` class property 

[Splitter.get_mask_arr](#) with default arguments.

`merge_sets` method 

```
Splitter.merge_sets(
    columns=None,
    new_set_label=None,
    insert_at_last=False,
    wrapper_kwargs=None,
    init_kwargs=None,
    **merge_split_kwargs
)
```

Merge multiple sets (columns) into a set (column).

Arguments `**merge_split_kwargs` are passed to [Splitter.merge_split](#).

If columns are not provided, merges all columns. If provided and `insert_at_last` is True, a new column is inserted at the position of the last column.

Use `new_set_label` to specify the label of the new set. To provide final labels, define `columns` in `wrapper_kwargs`.

`merge_split` class method 

```
Splitter.merge_split(
    split,
    range_format=None,
    wrap_with_template=False,
    wrap_with_fixrange=False,
    wrap_with_hslice=False,
    template_context=None,
    index=None
)
```

Merge a split of multiple fixed ranges into a fixed range.

Creates one mask and sets True for each range. If all input ranges are masks, returns that mask. If all input ranges are slices, returns a slice if possible. Otherwise, returns integer indices.

For `range_format`, see [Splitter.get_ready_range](#). Enable `wrap_with_template` to wrap the resulting range with a template of the type [Rep](#).

metrics property 

Metrics supported by [Splitter](#).

```
HybridConfig(
    start=dict(
        title='Index Start',
        calc_func=<function Splitter.<lambda> at 0x14fd10540>,
        agg_func=None,
        tags=[
            'splitter',
            'index'
        ]
    ),
    end=dict(
        title='Index End',
        calc_func=<function Splitter.<lambda> at 0x14fd105e0>,
        agg_func=None,
        tags=[
            'splitter',
            'index'
        ]
    ),
    period=dict(
        title='Index Length',
        calc_func=<function Splitter.<lambda> at 0x14fd10680>,
        agg_func=None,
        tags=[
            'splitter',
            'index'
        ]
    ),
    split_count=dict(
        title='Splits',
        calc_func='n_splits',
        agg_func=None,
        tags=[
            'splitter',
            'splits'
        ]
    ),
    set_count=dict(
        title='Sets',
        calc_func='n_sets',
        agg_func=None,
        tags=[
            'splitter',
            'splits'
        ]
    ),
    coverage=dict(
        title=RepFunc(
            template=<function Splitter.<lambda> at 0x14fd10720>,
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        calc_func='coverage',
        overlapping=False,
        post_calc_func=<function Splitter.<lambda> at 0x14fd107c0>,
        agg_func=None,
        tags=[
            'splitter',
            'splits',
            'coverage'
        ]
    ),
    set_coverage=dict(
        title=RepFunc(
            template=<function Splitter.<lambda> at 0x14fd10860>,
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        check_has_multiple_sets=True,
        calc_func='set_coverage',
        overlapping=False,
        relative=False,
        post_calc_func=<function Splitter.<lambda> at 0x14fd10900>,
        agg_func=None,
        tags=[
            'splitter',
            'splits',
            'coverage'
        ]
    ),
    set_mean_rel_coverage=dict(
        title='Mean Rel Coverage [%]',
        check_has_multiple_sets=True,
        check_normalize=True,

```

```

        calc_func='range_coverage',
        relative=True,
        post_calc_func=<function Splitter.<lambda> at 0x14fd109a0>,
        agg_func=None,
        tags=[
            'splitter',
            'splits',
            'coverage'
        ]
    ),
    overlap_coverage=dict(
        title=RepFunc(
            template=<function Splitter.<lambda> at 0x14fd10a40>,
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        calc_func='coverage',
        overlapping=True,
        post_calc_func=<function Splitter.<lambda> at 0x14fd10ae0>,
        agg_func=None,
        tags=[
            'splitter',
            'splits',
            'coverage'
        ]
    ),
    set_overlap_coverage=dict(
        title=RepFunc(
            template=<function Splitter.<lambda> at 0x14fd10b80>,
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        check_has_multiple_sets=True,
        calc_func='set_coverage',
        overlapping=True,
        post_calc_func=<function Splitter.<lambda> at 0x14fd10c20>,
        agg_func=None,
        tags=[
            'splitter',
            'splits',
            'coverage'
        ]
    )
)

```

Returns `Splitter._metrics`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change metrics, you can either change the config in-place, override this property, or overwrite the instance variable `Splitter._metrics`.

n_sets class property

Number of sets.

n_splits class property

Number of splits.

parse_and_inject_takeables class method

```

Splitter.parse_and_inject_takeables(
    flat_ann_args,
    eval_id=None
)

```

Parse **Takeable** instances from function annotations and inject them into flattened annotated arguments.

plot method

```

Splitter.plot(
    split_group_by=None,
    set_group_by=None,
    mask_kwargs=None,
    trace_kwargs=None,
    add_trace_kwargs=None,
    fig=None,
    **layout_kwargs
)

```

Plot splits as rows and sets as colors.

Args`split_group_by` : anySplit groups. See [BaseIDXAccessor.get_grouper](#).`set_group_by` : anySet groups. See [BaseIDXAccessor.get_grouper](#).`mask_kwargs` : dictKeyword arguments passed to [Splitter.get_iter_set_masks](#).`trace_kwargs` : dictKeyword arguments passed to `plotly.graph_objects.Heatmap`.

Can be a sequence, one per set.

`add_trace_kwargs` : dictKeyword arguments passed to `add_trace`.`fig` : Figure or FigureWidget

Figure to add traces to.

`**layout_kwargs`

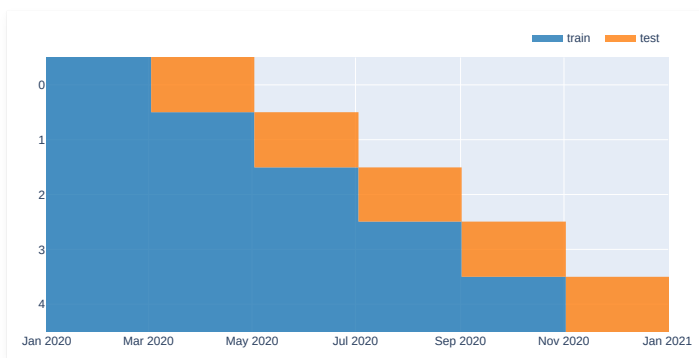
Keyword arguments for layout.

Usage

- Plot a scikit-learn splitter:

```
>>> from vectorbtpro import *
>>> from sklearn.model_selection import TimeSeriesSplit

>>> index = pd.date_range("2020", "2021", freq="D")
>>> splitter = vbt.Splitter.from_sklearn(index, TimeSeriesSplit())
>>> splitter.plot().show()
```

**plot_coverage** method

```
Splitter.plot_coverage(
    stacked=True,
    split_group_by=None,
    set_group_by=None,
    mask_kwargs=None,
    trace_kwargs=None,
    add_trace_kwargs=None,
    fig=None,
    **layout_kwargs
)
```

Plot index as rows and sets as lines.

Args`stacked` : bool

Whether to plot as an area plot.

`split_group_by` : any

Split groups. See [BaseIDXAccessor.get_grouper](#).

`set_group_by` : any

Set groups. See [BaseIDXAccessor.get_grouper](#).

`mask_kwargs` : dict

Keyword arguments passed to [Splitter.get_iter_set_masks](#).

`trace_kwargs` : dict

Keyword arguments passed to `plotly.graph_objects.Scatter`.

Can be a sequence, one per set.

`add_trace_kwargs` : dict

Keyword arguments passed to `add_trace`.

`fig` : Figure or FigureWidget

Figure to add traces to.

`**layout_kwargs`

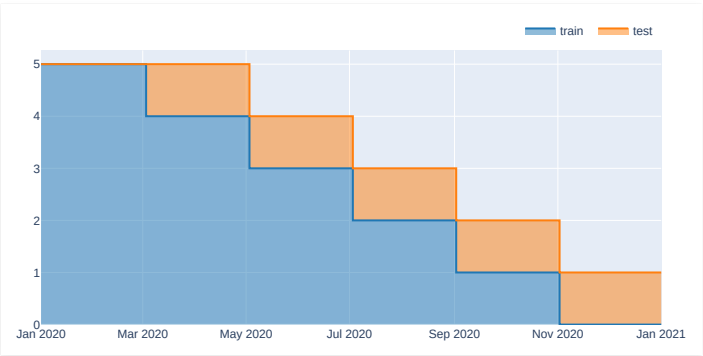
Keyword arguments for layout.

Usage

- Area plot:

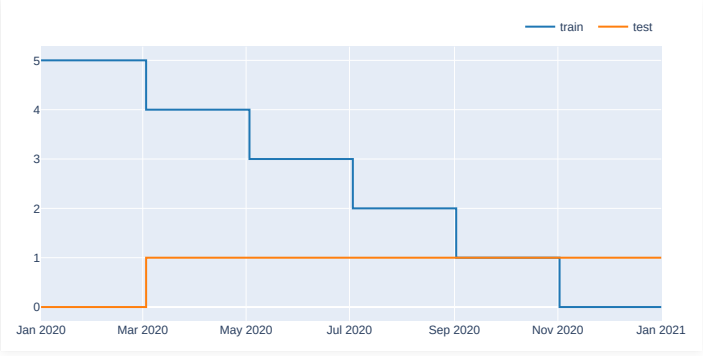
```
>>> from vectorbtpro import *
>>> from sklearn.model_selection import TimeSeriesSplit

>>> index = pd.date_range("2020", "2021", freq="D")
>>> splitter = vbt.Splitter.from_sklearn(index, TimeSeriesSplit())
>>> splitter.plot_coverage().show()
```



- Line plot:

```
>>> splitter.plot_coverage(stacked=False).show()
```



`plots_defaults` class property

Defaults for [PlotsBuilderMixin.plots](#).

Merges [PlotsBuilderMixin.plots_defaults](#) and `plots` from [splitter](#).

range_coverage class property 

[Splitter.get_range_coverage](#) with default arguments.

range_overlap_matrix class property 

[Splitter.get_overlap_matrix](#) with `by="range"`.

remap_range class method 

```
Splitter.remap_range(
    range_,
    target_index,
    target_freq=None,
    template_context=None,
    jitted=None,
    silence_warnings=False,
    index=None,
    freq=None
)
```

Remap a range to a target index.

If `index` and `target_index` are the same, returns the range. Otherwise, uses [Resampler.resample_source_mask](#) to resample the range into the target index. In such a case, `freq` and `target_freq` must be provided.

resolve_column_stack_kwargs class method 

```
Splitter.resolve_column_stack_kwargs(
    *objs,
    reindex_kwargs=None,
    **kwargs
)
```

Resolve keyword arguments for initializing [Splitter](#) after stacking along columns.

resolve_row_stack_kwargs class method 

```
Splitter.resolve_row_stack_kwargs(
    *objs,
    **kwargs
)
```

Resolve keyword arguments for initializing [Splitter](#) after stacking along rows.

row_stack class method 

```
Splitter.row_stack(
    *objs,
    wrapper_kwargs=None,
    **kwargs
)
```

Stack multiple [Splitter](#) instances along rows.

Uses [ArrayWrapper.row_stack](#) to stack the wrappers.

select_indices method 

```
Splitter.select_indices(
    split=None,
    set_=None,
    split_group_by=None,
    set_group_by=None
)
```

Get indices corresponding to selected splits and sets.

Arguments `split` and `set_` can be either integers and labels. Also, multiple values are accepted; in such a case, the corresponding ranges are merged. If split/set labels are of an integer data type, treats the provided values as labels rather than indices, unless the split/set index is not of an integer data type or the values are wrapped with [PosSel](#).

If `split_group_by` and/or `set_group_by` are provided, their groupers get created using [BaseIDXAccessor.get_grouper](#) and arguments `split` and `set_` become relative to the groups.

If `split / set_` is not provided, selects all indices.

Returns four arrays: split group indices, set group indices, split indices, and set indices.

`select_range` method

```
Splitter.select_range(
    merge_split_kwargs=None,
    **select_indices_kwargs
)
```

Select a range.

Passes `**select_indices_kwargs` to [Splitter.select_indices](#) to get the indices for selected splits and sets. If multiple ranges correspond to those indices, merges them using [Splitter.merge_split](#).

`set_coverage` class property

[Splitter.get_set_coverage](#) with default arguments.

`set_labels` class property

Set labels.

`set_overlap_matrix` class property

[Splitter.get_overlap_matrix](#) with `by="set"`.

`shuffle_splits` method

```
Splitter.shuffle_splits(
    size=None,
    replace=False,
    p=None,
    seed=None,
    wrapper_kwargs=None,
    **init_kwargs
)
```

Shuffle splits.

`split_and_apply` class method

```
Splitter.split_and_apply(
    index,
    apply_func,
    *apply_args,
    splitter=None,
    splitter_kwargs=None,
    apply_kwargs=None,
    template_context=None,
    **var_kwargs
)
```

Split an index and apply a function.

Argument `splitter` can be an actual [Splitter](#) instance, the name of a factory method (such as "from_n_rolling"), or the factory method itself. If `splitter` is None, the right method will be guessed based on the supplied arguments using [Splitter.guess_method](#).

Keyword arguments `splitter_kwargs` are passed to the factory method. Keyword arguments `apply_kwargs` are passed to [Splitter.apply](#). If variable keyword arguments are provided, they will be used as `apply_kwargs` if a splitter instance has been built, otherwise, arguments will be distributed based on the signatures of the factory method and [Splitter.apply](#).

split_and_take class method

```

Splitter.split_and_take(
    index,
    obj,
    splitter=None,
    splitter_kwargs=None,
    take_kwargs=None,
    template_context=None,
    **var_kwargs
)

```

Split an index and take from an object.

Argument `splitter` can be an actual [Splitter](#) instance, the name of a factory method (such as "from_n_rolling"), or the factory method itself. If `splitter` is None, the right method will be guessed based on the supplied arguments using [Splitter.guess_method](#).

Keyword arguments `splitter_kwargs` are passed to the factory method. Keyword arguments `take_kwargs` are passed to [Splitter.take](#). If variable keyword arguments are provided, they will be used as `take_kwargs` if a splitter instance has been built, otherwise, arguments will be distributed based on the signatures of the factory method and [Splitter.take](#).

split_coverage class property

[Splitter.get_split_coverage](#) with default arguments.

split_labels class property

Split labels.

split_overlap_matrix class property

[Splitter.get_overlap_matrix](#) with `by="split"`.

split_range class method

```

Splitter.split_range(
    range_,
    new_split,
    backwards=False,
    allow_zero_len=False,
    range_format=None,
    wrap_with_template=False,
    wrap_with_fixrange=False,
    template_context=None,
    index=None,
    freq=None
)

```

Split a fixed range into a split of multiple fixed ranges.

Range must be either a template, a callable, a tuple (start and stop), a slice, a sequence of indices, or a mask. This range will then be re-mapped into the index.

Each sub-range in `new_split` can be either a fixed or relative range, that is, an instance of [RelRange](#) or a number that will be used as a length to create an [RelRange](#). Each sub-range will then be re-mapped into the main range. Argument `new_split` can also be provided as an integer or a float indicating the length; in such a case the second part (or the first one depending on `backwards`) will stretch. If `new_split` is a string, the following options are supported:

- 'by_gap': Split `range_` by gap using [split_range_by_gap_nb](#).

New ranges are returned relative to the index and in the same order as passed.

For `range_format`, see [Splitter.get_ready_range](#). Enable `wrap_with_template` to wrap the resulting ranges with a template of the type [Rep](#).

split_set method

```

Splitter.split_set(
    new_split,
    column=None,
    new_set_labels=None,
    wrapper_kwargs=None,
    init_kwargs=None,
    **split_range_kwargs
)

```

Split a set (column) into multiple sets (columns).

Arguments `new_split` and `**split_range_kwargs` are passed to `Splitter.split_range`.

Column must be provided if there are two or more sets.

Use `new_set_labels` to specify the labels of the new sets; it must have the same length as there are new ranges in the new split. To provide final labels, define `columns` in `wrapper_kwargs`.

splits class property

`Splitter.splits_arr` as a DataFrame.

splits_arr class property

Two-dimensional, object-dtype DataFrame with splits.

First axis represents splits. Second axis represents sets. Elements represent ranges. Range must be either a slice, a sequence of indices, a mask, or a callable that returns such.

stats_defaults class property

Defaults for `StatsBuilderMixin.stats`.

Merges `StatsBuilderMixin.stats_defaults` and `stats` from `splitter`.

subplots property

Subplots supported by `Splitter`.

```
HybridConfig(
    plot=dict(
        title='Splits',
        yaxis_kwargs=dict(
            title='Split'
        ),
        plot_func='plot',
        tags='splitter'
    ),
    plot_coverage=dict(
        title='Coverage',
        yaxis_kwargs=dict(
            title='Count'
        ),
        plot_func='plot_coverage',
        tags='splitter'
    )
)
```

Returns `Splitter._subplots`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change subplots, you can either change the config in-place, override this property, or overwrite the instance variable `Splitter._subplots`.

take method

```
Splitter.take(
    obj,
    split=None,
    set=None,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    into=None,
    remap_to_obj=True,
    obj_index=None,
    obj_freq=None,
    range_format='slice_or_any',
    point_wise=False,
    attach_bounds=False,
    right_inclusive=False,
    template_context=None,
    silence_warnings=False,
    index_combine_kwargs=None,
    stack_axis=1,
    stack_kwargs=None,
    freq=None
)
```

Take all ranges from an array-like object and optionally column-stack them.

Uses `Splitter.select_indices` to get the indices for selected splits and sets. Arguments `split_group_by` and `set_group_by` can be used to group splits and sets respectively. Ranges belonging to the same split and set group will be merged.

For each index pair, resolves the source range using `Splitter.select_range` and `Splitter.get_ready_range`. Then, remaps this range into the object index using `Splitter.get_ready_obj_range` and takes the slice from the object using `Splitter.take_range`. If the object is a custom template, substitutes its instead of calling `Splitter.take_range`. Finally, uses `column_stack_merge` (`stack_axis=1`) or `row_stack_merge` (`stack_axis=0`) with `stack_kwargs` to merge the taken slices.

If `attach_bounds` is enabled, measures the bounds of each range and makes it an additional level in the final index hierarchy. The argument supports the following options:

- True, 'index', 'source', or 'source_index': Attach source (index) bounds
- 'target' or 'target_index': Attach target (index) bounds
- False: Do not attach

Argument `into` supports the following options:

- None: Series of range slices
- 'stacked': Stack all slices into a single object
- 'stacked_by_split': Stack set slices in each split and return a Series of objects
- 'stacked_by_set': Stack split slices in each set and return a Series of objects
- 'split_major_meta': Generator with ranges processed lazily in split-major order. Returns meta with indices and labels, and the generator.
- 'set_major_meta': Generator with ranges processed lazily in set-major order. Returns meta with indices and labels, and the generator.

Prepend any stacked option with "from_start_" (also "reset_") or "from_end_" to reset the index from start and from end respectively.

Usage

- Roll a window and stack it along columns by keeping the index:

```
>>> from vectorbtpro import *

>>> data = vbt.YFData.pull(
...     "BTC-USD",
...     start="2020-01-01 UTC",
...     end="2021-01-01 UTC"
... )
>>> splitter = vbt.Splitter.from_n_rolling(
...     data.wrapper.index,
...     3,
...     length=5
... )
>>> splitter.take(data.close, into="stacked")
split
Date
2020-01-01 00:00:00+00:00    7200.174316    NaN    NaN
2020-01-02 00:00:00+00:00    6985.470215    NaN    NaN
2020-01-03 00:00:00+00:00    7344.884277    NaN    NaN
2020-01-04 00:00:00+00:00    7410.656738    NaN    NaN
2020-01-05 00:00:00+00:00    7411.317383    NaN    NaN
2020-06-29 00:00:00+00:00         NaN    9190.854492    NaN
2020-06-30 00:00:00+00:00         NaN    9137.993164    NaN
2020-07-01 00:00:00+00:00         NaN    9228.325195    NaN
2020-07-02 00:00:00+00:00         NaN    9123.410156    NaN
2020-07-03 00:00:00+00:00         NaN    9087.303711    NaN
2020-12-27 00:00:00+00:00         NaN         NaN    26272.294922
2020-12-28 00:00:00+00:00         NaN         NaN    27084.808594
2020-12-29 00:00:00+00:00         NaN         NaN    27362.437500
2020-12-30 00:00:00+00:00         NaN         NaN    28840.953125
2020-12-31 00:00:00+00:00         NaN         NaN    29001.720703
```

- Disgard the index and attach index bounds to the column hierarchy:

```
>>> splitter.take(
...     data.close,
...     into="reset_stacked",
...     attach_bounds="index"
... )
split
start 2020-01-01 00:00:00+00:00 2020-06-29 00:00:00+00:00
end 2020-01-06 00:00:00+00:00 2020-07-04 00:00:00+00:00
0    7200.174316    9190.854492
1    6985.470215    9137.993164
2    7344.884277    9228.325195
3    7410.656738    9123.410156
4    7411.317383    9087.303711

split
start 2020-12-27 00:00:00+00:00
end 2021-01-01 00:00:00+00:00
0    26272.294922
1    27084.808594
2    27362.437500
3    28840.953125
4    29001.720703
```

take_range class method

```
Splitter.take_range(
    obj,
    ready_range,
    point_wise=False
)
```

Take a ready range from an array-like object.

Set `point_wise` to True to select one range point at a time and return a tuple.

take_range_from_takeable class method

```
Splitter.take_range_from_takeable(
    takeable,
    range_,
    remap_to_obj=True,
    obj_index=None,
    obj_freq=None,
    point_wise=False,
    template_context=None,
    return_obj_meta=False,
    return_meta=False,
    **ready_obj_range_kwargs
)
```

Take a range from a takeable object.

to_fixed method

```
Splitter.to_fixed(
    split_range_kwargs=None,
    **kwargs
)
```

Convert relative ranges into fixed ones and return a new [Splitter](#) instance.

Keyword arguments `split_range_kwargs` are passed to [Splitter.split_range](#).

to_grouped method

```
Splitter.to_grouped(
    split=None,
    set_=None,
    split_group_by=None,
    set_group_by=None,
    merge_split_kwargs=None,
    **kwargs
)
```

Merge all ranges within the same group and return a new [Splitter](#) instance.

Takeable class

```
Takeable(
    *args,
    **kwargs
)
```

Class that represents an object from which a range can be taken.

Superclasses

- [Annotatable](#)
- [Base](#)
- [DefineMixin](#)
- [Evaluable](#)
- [Hashable](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)

- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [DefineMixin.asdict](#)
- [DefineMixin.assert_field_not_missing](#)
- [DefineMixin.fields](#)
- [DefineMixin.fields_dict](#)
- [DefineMixin.get_field](#)
- [DefineMixin.hash](#)
- [DefineMixin.hash_key](#)
- [DefineMixin.is_field_missing](#)
- [DefineMixin.is_field_optional](#)
- [DefineMixin.is_field_required](#)
- [DefineMixin.merge_over](#)
- [DefineMixin.merge_with](#)
- [DefineMixin.replace](#)
- [DefineMixin.resolve](#)
- [DefineMixin.resolve_field](#)
- [Evaluable.meets_eval_id](#)
- [Hashable.get_hash](#)

eval_id field 

One or more identifiers at which to evaluate this instance.

freq field 

Frequency of [Takeable.index](#).

index field 

Index of the object.

If not present, will be accessed using [Splitter.get_obj_index](#).

obj field 

Takeable object.

point_wise field 

Whether to select one range point at a time and return a tuple.

remap_to_obj field 

Whether to remap [Splitter.index](#) to the index of [Takeable.obj](#).

Otherwise, will assume that the object has the same index.

ZeroLengthError class 

```
ZeroLengthError(  
    *args,  
    **kwargs  
)
```



Thrown whenever a range has a length of zero.

Superclasses

- `builtins.BaseException`
- `builtins.Exception`
- `builtins.ValueError`